

R Fundamentals

Vectors

Vector Types and Scalars

- Flavors: **atomic** (all elements same type), **generic** (lists, elements can differ)
- Primary atomic types: logical, integer, double, character; integer + double = numeric
- Scalars are simply vectors of length 1

```
v_logical    <- c(TRUE, FALSE)
v_integer    <- c(1L, 6L, 10L)
v_double     <- c(1, 2.5, 4.5)
v_character  <- c('these', 'are', 'characters')
typeof(v_logical); typeof(v_integer); typeof(v_double); typeof(v_character)
```

```
#> [1] "logical"
```

```
#> [1] "integer"
```

```
#> [1] "double"
```

```
#> [1] "character"
```

```
# concatenation of atomic vectors yields flat atomic vectors
(v <- c(c(1, 2), c(3, 4)))
```

```
#> [1] 1 2 3 4
```

NA and NULL

- NA (Not Applicable): value **exists** but is **unknown**
- NULL: value **does not exist**

```
NA > 5; 20 * NA; !NA           # NA is infectious!
```

```
#> [1] NA
```

```
#> [1] NA
```

```
#> [1] NA
```

```
NA ^ 0; NA | TRUE; NA & FALSE  # some exceptions
```

```
#> [1] 1
```

```
#> [1] TRUE
```

```
#> [1] FALSE
```

```
x <- c(NA, 5, NA, 10)
```

```
x == NA           # WRONG way to test (always NA)
```

```
#> [1] NA NA NA NA
```

```
is.na(x)          # CORRECT way to test
```

```
#> [1] TRUE FALSE TRUE FALSE
```

Coercion Hierarchy

- Automatic coercion: **character** > **double** > **integer** > **logical**

```
typeof(c('a', 1))      # character wins
```

```
#> [1] "character"
```

```
c(1, FALSE)           # logical -> double
```

```
#> [1] 1 0
```

```
c(TRUE, 1L)           # logical -> integer
```

```
#> [1] 1 1
```

```
# explicit coercion with as.*()
```

```
x <- c(TRUE, FALSE, TRUE, TRUE); as.numeric(x)
```

```
#> [1] 1 0 1 1
```

```
as.integer(c('1', '1.5', 'a'))  # 'a' -> NA with warning
```

```
#> [1] 1 1 NA
```

Testing	Coercion
<code>is.logical(x)</code>	<code>as.logical(x)</code>
<code>is.integer(x)</code>	<code>as.integer(x)</code>
<code>is.double(x)</code>	<code>as.double(x)</code>
<code>is.numeric(x)</code>	<code>as.numeric(x)</code>
<code>is.character(x)</code>	<code>as.character(x)</code>
<code>is.na(x), is.null(x)</code>	—
<code>is.vector(x)</code>	<code>as.vector(x)</code>
<code>is.list(x)</code>	<code>as.list(x)</code>
<code>is.data.frame(x)</code>	<code>as.data.frame(x)</code>
<code>is.matrix(x)</code>	<code>as.matrix(x)</code>
<code>is.factor(x)</code>	<code>as.factor(x)</code>

```
x <- c(3, 5, 7, 8)
is.integer(x); is.numeric(x)
```

```
#> [1] FALSE
```

```
#> [1] TRUE
```

```
as.integer(5.7); as.integer(-5.7)
```

```
#> [1] 5
```

```
#> [1] -5
```

Sequences — : and seq()

```
5:10; 5:1
```

```
#> [1] 5 6 7 8 9 10
```

```
#> [1] 5 4 3 2 1
```

```
i <- 5; 1:i-1; 1:(i-1)  # note operator precedence!
```

```
#> [1] 0 1 2 3 4
```

```
#> [1] 1 2 3 4
```

```
seq(from = 11, to = 30, by = 3)
```

```
#> [1] 11 14 17 20 23 26 29
```

```
seq(from = 1.1, to = 2, length = 10)
```

```
#> [1] 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0
```

```
# seq_along and seq_len: safer alternatives for loops
```

```
x <- c(9, 5, 17); seq_along(x)
```

```
#> [1] 1 2 3
```

```
x <- NULL; seq_along(x)  # integer(0), not 1:0 !
```


Repetition: rep()

```
rep(3, 5)
```

```
#> [1] 3 3 3 3 3
```

```
rep(c(10, 15, 3), 4)
```

```
#> [1] 10 15 3 10 15 3 10 15 3 10 15 3
```

```
rep(c(10, 15, 3), each = 4) # interleave with 'each'
```

```
#> [1] 10 10 10 10 15 15 15 15 3 3 3 3
```

```
rep(1:4, 1:4) # variable repetition counts
```

```
#> [1] 1 2 2 3 3 3 4 4 4 4
```

```
rep(c('C', 'G', 'H'), c(4, 5, 2))
```

```
#> [1] "C" "C" "C" "C" "G" "G" "G" "G" "G" "H" "H"
```

Other Generators: `sample()`, `sequence()`

```
set.seed(42)
sample(1:6, 10, replace = TRUE)      # dice rolls
```

```
#> [1] 1 5 1 1 2 4 2 2 1 4
```

```
sample(letters[1:20], 5)              # no replacement
```

```
#> [1] "e" "n" "r" "o" "c"
```

```
sequence(c(6, 7, 4), from = c(8, 1, 9), by = c(-1, 1, -2))
```

```
#> [1] 8 7 6 5 4 3 1 2 3 4 5 6 7 9 7 5 3
```

Numeric Indexing

```
fib <- c(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144)
fib[1]; fib[5]; fib[2:5]; fib[c(2, 4, 8, 1)]
```

```
#> [1] 0
```

```
#> [1] 3
```

```
#> [1] 1 1 2 3
```

```
#> [1] 1 2 13 0
```

```
# 'minus' means 'exclude'; no mixing of positive/negative!
fib[-3]; fib[-(5:9)]
```

```
#> [1] 0 1 2 3 5 8 13 21 34 55 89 144
```

```
#> [1] 0 1 1 2 34 55 89 144
```

Numeric Indexing Cont'd

```
# negative indexing to build new vectors
```

```
x <- c(10, 20, -1, 3, 5, 6, 30, 9)
```

```
(y <- x[-2])           # remove 2nd element
```

```
#> [1] 10 -1 3 5 6 30 9
```

```
(z <- x[c(-2, -3, -6)]) # remove 2nd, 3rd, 6th
```

```
#> [1] 10 3 5 30 9
```

```
(w <- c(x, y, z))       # combine vectors
```

```
#> [1] 10 20 -1 3 5 6 30 9 10 -1 3 5 6 30 9 10 3
```

```
#> [18] 5 30 9
```

```
fib[fib < 10]                                # elements < 10
```

```
#> [1] 0 1 1 2 3 5 8
```

```
fib[fib %% 2 == 0 | fib %% 3 == 0]          # multiples of 2 or 3
```

```
#> [1] 0 2 3 8 21 34 144
```

```
fib[c(TRUE, FALSE)]                          # odd-indexed (recycling!)
```

```
#> [1] 0 1 3 8 21 55 144
```

Logical Indexing Cont'd

```
v <- c(2, 8, 1, 7, -11, 6, 20, 3, 12, 35, 21, 3, 5, 7, -2)
v[v > median(v)]                      # above median
```

```
#> [1]  8  7 20 12 35 21  7
```

```
v[v < quantile(v, 0.05) | v > quantile(v, 0.95)] # extreme 5%
```

```
#> [1] -11  35
```

```
v[abs(v - mean(v)) > sd(v)]          # beyond 1 SD
```

```
#> [1] -11  20  35  21
```

```
which(), which.min(), which.max()
```

```
which(Nile > 1200)          # positions where TRUE
```

```
#> [1]  4  8  9 22 24 25 26
```

```
Nile[Nile > 1200]          # the values themselves
```

```
#> [1] 1210 1230 1370 1210 1250 1260 1220
```

```
x <- c(5, 3, 8, 1, 9, 2)
```

```
which.min(x); which.max(x)  # index of first min/max
```

```
#> [1] 4
```

```
#> [1] 5
```

Removing NA from Vectors

```
v <- c(21, 2, 3, NA, 5, NULL, NA, 32, 55)
v[!is.na(v)]           # NULL is already dropped by c()
```

```
#> [1] 21  2  3  5 32 55
```

```
na.omit(v)             # alternative; returns with attr
```

```
#> [1] 21  2  3  5 32 55
```

```
#> attr(,"na.action")
```

```
#> [1] 4 6
```

```
#> ... [output truncated]
```


Recycling Rule

- In operations involving two vectors of different lengths, R **repeats** the shorter one:

```
c(2, 4, 3) + c(6, 10, -1, 3, 5)
```

```
#> [1] 8 14 2 5 9
```

```
# useful applications
```

```
x <- 1:5
```

```
x + 10          # add a constant (scalar = length-1 vector)
```

```
#> [1] 11 12 13 14 15
```

```
x * c(1, -1)    # alternate signs
```

```
#> [1] 1 -2 3 -4 5
```

all(), any(), and Vector Equality

```
x <- 1:100; any(x > 200); all(x > 0)
```

```
#> [1] FALSE
```

```
#> [1] TRUE
```

```
# == is element-wise; don't use for whole-vector equality!
```

```
x <- 1:3; y0 <- c(1, 2, 3); y1 <- c(1, 3, 4)
```

```
x == y0; x == y1
```

```
#> [1] TRUE TRUE TRUE
```

```
#> [1] TRUE FALSE FALSE
```

identical() and all.equal()

```
x <- 1:3; y0 <- c(1, 2, 3)
identical(x, y0)          # FALSE: integer vs double!
```

```
#> [1] FALSE
```

```
typeof(x); typeof(y0)
```

```
#> [1] "integer"
```

```
#> [1] "double"
```

```
all.equal(x, y0)          # TRUE (with tolerance)
```

```
#> [1] TRUE
```

```
all.equal(x, c(1, 3, 4)) # shows difference
```

```
#> [1] "Mean relative difference: 0.4"
```

Named Vectors

```
# 3 ways to name a vector
x <- c(a = 1, b = 2, c = 3)
x <- 1:3; names(x) <- c('a', 'b', 'c')
x <- setNames(1:3, c('a', 'b', 'c'))
```

```
# access by name
x['b']; x[c('a', 'c')]
```

```
#> b
```

```
#> 2
```

```
#> a c
```

```
#> 1 3
```

- Factors represent **categorical variables** with a fixed set of levels.

```
(x <- factor(c('a', 'b', 'b', 'a')))
```

```
#> [1] a b b a  
#> Levels: a b
```

```
typeof(x); levels(x)
```

```
#> [1] "integer"  
#> [1] "a" "b"
```

```
# ordered factors
```

```
(size <- factor(c('S','M','L','M','S'),  
               levels=c('S','M','L'), ordered=TRUE))
```

```
#> [1] S M L M S  
#> Levels: S < M < L
```

```
size > 'S'
```

```
#> [1] FALSE TRUE TRUE TRUE FALSE
```

Let `v <- c(14, -3, 22, 7, -8, 0, 31, 5, -1, 18)`.

- Extract all elements strictly between 5 and 25
- Replace every negative element with 0 in a copy `w`
- How many elements are negative or exceed 20?
- Find indices where $|v_i|$ exceeds its rank

```
v <- c(14, -3, 22, 7, -8, 0, 31, 5, -1, 18)
v[v > 5 & v < 25]
```

```
#> [1] 14 22 7 18
```

```
w <- v; w[w < 0] <- 0; w
```

```
#> [1] 14 0 22 7 0 0 31 5 0 18
```

```
sum(v < 0 | v > 20)
```

```
#> [1] 5
```

```
which(abs(v) > rank(v))
```

```
#> [1] 1 2 3 4 5 7 10
```

Create `stock <- c(AAPL=189.5, MSFT=415.2, GOOG=172.3, AMZN=185.7)`.

- Access GOOG by name; add NVDA = 875.4
- Compute percentage change from base 150; round to 1 dp
- Return stocks above mean price, sorted descending
- Rename all tickers to lowercase

Sol 2

```
stock <- c(AAPL=189.5, MSFT=415.2, GOOG=172.3, AMZN=185.7)
stock["GOOG"]; stock["NVDA"] <- 875.4
```

```
#> GOOG
#> 172.3
```

```
round((stock - 150) / 150 * 100, 1)
```

```
#> AAPL MSFT GOOG AMZN NVDA
#> 26.3 176.8 14.9 23.8 483.6
```

```
sort(stock[stock > mean(stock)], decreasing=TRUE)
```

```
#> NVDA MSFT
#> 875.4 415.2
```

```
names(stock) <- tolower(names(stock)); names(stock)
```

```
#> [1] "aapl" "msft" "goog" "amzn" "nvda"
```

Ex 3: which, which.min, which.max

Let `x <- c(3, 7, 2, 9, 1, 5, 8, 4, 6, 10)`.

- Indices of elements > 6
- Index of min and max
- Second-largest value (no sort)
- Local maxima: elements greater than both immediate neighbours (ignore endpoints)

```
x <- c(3, 7, 2, 9, 1, 5, 8, 4, 6, 10)
which(x > 6)
```

```
#> [1] 2 4 7 10
```

```
which.min(x); which.max(x)
```

```
#> [1] 5
```

```
#> [1] 10
```

```
max(x[x != max(x)])
```

```
#> [1] 9
```

```
n <- length(x); idx <- 2:(n-1)
x[idx][x[idx] > x[idx-1] & x[idx] > x[idx+1]]
```

```
#> [1] 7 9 8
```

Let $x \leftarrow c(2,5,3,8,1,7,4)$ and $y \leftarrow c(6,2,9,1,4,3,8)$.

- Running maximum of x
- Running sum and product
- Element-wise min and max
- Verify $\sum \text{pmin}(x, y) + \sum \text{pmax}(x, y) = \sum x + \sum y$

Sol 4

```
x <- c(2,5,3,8,1,7,4); y <- c(6,2,9,1,4,3,8)
cummax(x)
```

```
#> [1] 2 5 5 8 8 8 8
```

```
cumsum(x); cumprod(x)
```

```
#> [1] 2 7 10 18 19 26 30
```

```
#> [1] 2 10 30 240 240 1680 6720
```

```
pmin(x,y); pmax(x,y)
```

```
#> [1] 2 2 3 1 1 3 4
```

```
#> [1] 6 5 9 8 4 7 8
```

```
sum(pmin(x,y)) + sum(pmax(x,y)) == sum(x) + sum(y)
```

```
#> [1] TRUE
```

Let `a <- c(1,3,5,7,9,3,5)` and `b <- c(2,3,5,6,7,2)`.

- Unique elements of `a`
- Positions of duplicated values in `a`
- $a \cup b$, $a \cap b$, $a \setminus b$
- `match(c(3,5,4), b)` — explain the NA

Sol 5

```
a <- c(1,3,5,7,9,3,5); b <- c(2,3,5,6,7,2)
unique(a)
```

```
#> [1] 1 3 5 7 9
```

```
which(duplicated(a))
```

```
#> [1] 6 7
```

```
union(a,b); intersect(a,b); setdiff(a,b)
```

```
#> [1] 1 3 5 7 9 2 6
```

```
#> [1] 3 5 7
```

```
#> [1] 1 9
```

```
match(c(3,5,4), b) # 4 not found -> NA
```

```
#> [1] 2 3 NA
```

- What does `c(1,2,3,4,5,6) + c(10,20)` return? Why?
- Let `m <- matrix(1:12, nrow=3)`. What does `m + c(100,200,300)` do?
- Extract odd-indexed and even-indexed elements of `1:12` using `c(TRUE,FALSE)`
- Generate `1 0 1 0 1 0 1 0 1 0` using `rep`

Sol 6

```
c(1,2,3,4,5,6) + c(10,20) # recycled
```

```
#> [1] 11 22 13 24 15 26
```

```
m <- matrix(1:12, nrow=3)
m + c(100,200,300) # adds per column (col-major)
```

```
#>      [,1] [,2] [,3] [,4]
#> [1,]  101  104  107  110
#> [2,]  202  205  208  211
#> ... [output truncated]
```

```
x <- 1:12; x[c(T,F)]; x[c(F,T)]
```

```
#> [1]  1  3  5  7  9 11
#> [1]  2  4  6  8 10 12
```

```
rep(c(1,0), 5)
```

```
#> [1] 1 0 1 0 1 0 1 0 1 0
```

Let `x <- c(40,15,30,10,50,25)`.

- Sort ascending and descending
- Find `order(x)` and verify `x[order(x)] == sort(x)`
- What rank does 30 receive?
- Sort `c("banana","apple","cherry")` alphabetically

```
x <- c(40,15,30,10,50,25)
sort(x); sort(x, decreasing=TRUE)
```

```
#> [1] 10 15 25 30 40 50
```

```
#> [1] 50 40 30 25 15 10
```

```
ord <- order(x); all(x[ord] == sort(x))
```

```
#> [1] TRUE
```

```
rank(x) # 30 -> rank 4
```

```
#> [1] 5 2 4 1 6 3
```

```
sort(c("banana","apple","cherry"))
```

```
#> [1] "apple" "banana" "cherry"
```

- Given `x <- c(3, NA, -1, NaN, Inf, -Inf, 0)`, apply `is.na`, `is.nan`, `is.finite`, `is.infinite`. Why is `is.na(NaN)` TRUE?
- Remove non-finite values and compute mean
- Show `c(1, NULL, 3)` vs `c(1, NA, 3)` lengths
- Compute `NA^0`, `NA | TRUE`, `NA & FALSE`

Sol 8

```
x <- c(3, NA, -1, NaN, Inf, -Inf, 0)
is.na(x); is.nan(x); is.finite(x)
```

```
#> [1] FALSE TRUE FALSE TRUE FALSE FALSE FALSE
```

```
#> [1] FALSE FALSE FALSE TRUE FALSE FALSE FALSE
```

```
#> [1] TRUE FALSE TRUE FALSE FALSE FALSE TRUE
```

```
mean(x[is.finite(x)])
```

```
#> [1] 0.6666667
```

```
length(c(1, NULL, 3)); length(c(1, NA, 3))      # 2 vs 3
```

```
#> [1] 2
```

```
#> [1] 3
```

```
NA^0; NA | TRUE; NA & FALSE
```

```
#> [1] 1
```

```
#> [1] TRUE
```

```
#> [1] FALSE
```

Ex 9: Type Coercion

- What does `c(TRUE, 1L, 2.5, "x")` produce? What type?
- Coerce `c("3", "1.5", "NA", "7")` to numeric. What happens to "NA"?
- Predict `as.integer(c(3.9, -2.7, TRUE, "5"))`
- Why does `1 == "1"` return TRUE?

```
x <- c(TRUE, 1L, 2.5, "x"); x; typeof(x)           # character
```

```
#> [1] "TRUE" "1"    "2.5"  "x"
```

```
#> [1] "character"
```

```
as.numeric(c("3","1.5","NA","7"))                # NA + warning
```

```
#> [1] 3.0 1.5 NA 7.0
```

```
as.integer(c(3.9, -2.7, TRUE, "5"))                # 3,-2,1,5
```

```
#> [1] 3 -2 NA 5
```

```
1 == "1"      # 1 coerced to "1", then string compare
```

```
#> [1] TRUE
```

Ex 10: seq and seq_along

- Generate 1.0 2.5 4.0 5.5 7.0 8.5 10.0 using seq
- Show `1:length(x)` is unsafe when `x <- NULL`; fix with `seq_along`
- 6 equally-spaced values from 0 to 2π
- What does `seq_along(integer(0))` return?


```
seq(1, 10, by=1.5)
```

```
#> [1] 1.0 2.5 4.0 5.5 7.0 8.5 10.0
```

```
x <- NULL; 1:length(x) # c(1,0) WRONG
```

```
#> [1] 1 0
```

```
seq_along(x) # integer(0) SAFE
```

```
#> integer(0)
```

```
seq(0, 2*pi, length.out=6)
```

```
#> [1] 0.000000 1.256637 2.513274 3.769911 5.026548
```

```
#> [6] 6.283185
```

```
seq_along(integer(0)) # integer(0)
```

```
#> integer(0)
```

Using rep, generate:

- 1 1 2 2 3 3 4 4 5 5
- 1 2 3 1 2 3 1 2 3 1 2 3
- "A" "A" "A" "B" "B" "C"
- Alternating TRUE FALSE of length 20

```
rep(1:5, each=2)
```

```
#> [1] 1 1 2 2 3 3 4 4 5 5
```

```
rep(1:3, times=4)
```

```
#> [1] 1 2 3 1 2 3 1 2 3 1 2 3
```

```
rep(c("A","B","C"), c(3,2,1))
```

```
#> [1] "A" "A" "A" "B" "B" "C"
```

```
rep(c(TRUE,FALSE), 10)
```

```
#> [1] TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE
```

```
#> [9] TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE
```

```
#> [17] TRUE FALSE TRUE FALSE
```

```
#> ... [output truncated]
```

```
haystack <- c(3,1,4,1,5,9,2,6,5,3); needles <- c(5,7,2).
```

- Which needles are present?
- First occurrence position of each
- Difference between `match("a", c("b","a","a"))` and `which(c("b","a","a") == "a")`
- Filter `mtcars` rows where `cyl %in% c(4,6)`

Sol 12

```
haystack <- c(3,1,4,1,5,9,2,6,5,3); needles <- c(5,7,2)
```

```
needles %in% haystack
```

```
#> [1] TRUE FALSE TRUE
```

```
match(needles, haystack) # NA for 7
```

```
#> [1] 5 NA 7
```

```
match("a", c("b","a","a")) # 2 (first)
```

```
#> [1] 2
```

```
which(c("b","a","a") == "a") # c(2,3) (all)
```

```
#> [1] 2 3
```

```
nrow(mtcars[mtcars$cyl %in% c(4,6), ])
```

```
#> [1] 18
```

- Compute $H_{100} = \sum_{k=1}^{100} \frac{1}{k}$ in one line
- Compute $\sum_{k=0}^{50} \frac{(-1)^k}{k!}$. Compare with `exp(-1)`
- For `x <- seq(0.1,2,by=0.1)`, compute $e^x \sin x - \ln x$, rounded to 4 dp
- Compute `diff(cumsum(1:10))` and explain

Sol 13

```
sum(1 / 1:100) # H_100
```

```
#> [1] 5.187378
```

```
k <- 0:50; sum((-1)^k / factorial(k)); exp(-1)
```

```
#> [1] 0.3678794
```

```
#> [1] 0.3678794
```

```
x <- seq(0.1, 2, by=0.1)  
round(exp(x)*sin(x) - log(x), 4)
```

```
#> [1] 2.4129 1.8521 1.6029 1.4972 1.4836 1.5397 1.6540
```

```
#> [8] 1.8196 2.0320 2.2874 2.5820 2.9122 3.2732 3.6597
```

```
#> [15] 4.0650 4.4809 4.8977 5.3036 5.6850 6.0257
```

```
#> ... [output truncated]
```

```
diff(cumsum(1:10)) # recovers 2:10
```

```
#> [1] 2 3 4 5 6 7 8 9 10
```

- Create `a <- 1:4`; attach attributes `"unit"="kg"` and `"source"="lab"`. Print
- Recreate using `structure()` in one call
- `x <- 1:3` vs `y <- c(1,2,3)`: are they identical? `all.equal()`? Why?
- When to use `isTRUE(all.equal())` in an if?


```
a <- 1:4; attr(a,"unit") <- "kg"; attr(a,"source") <- "lab"  
attributes(a)
```

```
#> $unit  
#> [1] "kg"  
#>  
#> ... [output truncated]
```

```
b <- structure(1:4, unit="kg", source="lab")  
x <- 1:3; y <- c(1,2,3)  
identical(x, y); all.equal(x, y) # F; T
```

```
#> [1] FALSE  
#> [1] TRUE
```

```
isTRUE(all.equal(0.1+0.2, 0.3)) # TRUE
```

```
#> [1] TRUE
```

- Create factor `f <- factor(c("low","med","high","med","low"))`. What are the levels?
- Create an ordered factor with levels `low < med < high`. Test `f > "low"`
- Use `table(f)` to count occurrences
- What does `as.integer(f)` return and why?

```
f <- factor(c("low","med","high","med","low"))  
levels(f)                                     # alphabetical
```

```
#> [1] "high" "low"  "med"
```

```
fo <- factor(f, levels=c("low","med","high"), ordered=TRUE)  
fo > "low"
```

```
#> [1] FALSE TRUE TRUE TRUE FALSE
```

```
table(f)
```

```
#> f  
#> high low med  
#>    1    2    2  
#> ... [output truncated]
```

```
as.integer(f) # underlying integer codes
```

```
#> [1] 2 3 1 3 2
```

- Is `0.1 + 0.2 == 0.3` TRUE or FALSE? Why?
- Use `all.equal` to compare correctly
- What is `trunc(-3.7)` vs `floor(-3.7)`?
- Demonstrate banker's rounding: `round(0.5)`, `round(1.5)`, `round(2.5)`

```
0.1 + 0.2 == 0.3 # FALSE (floating point)
```

```
#> [1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3) # TRUE
```

```
#> [1] TRUE
```

```
trunc(-3.7); floor(-3.7) # -3 vs -4
```

```
#> [1] -3
```

```
#> [1] -4
```

```
round(0.5); round(1.5); round(2.5) # 0; 2; 2
```

```
#> [1] 0
```

```
#> [1] 2
```

```
#> [1] 2
```

Ex 17: Appending and Inserting

- Start with `v <- 1:5`. Append `c(10,11)` to the end
- Insert 99 after position 3 using `append`
- What happens if you assign `v[10] <- 42` when `length(v)` is 7?
- Remove the 3rd element without using negative indexing (use logical)

Sol 17

```
v <- 1:5; v <- c(v, c(10,11)); v
```

```
#> [1] 1 2 3 4 5 10 11
```

```
append(v, 99, after=3)
```

```
#> [1] 1 2 3 99 4 5 10 11
```

```
v[10] <- 42; v # NAs fill gaps
```

```
#> [1] 1 2 3 4 5 10 11 NA NA 42
```

```
v[seq_along(v) != 3]
```

```
#> [1] 1 2 4 5 10 11 NA NA 42
```

Ex 18: sample and sequence

- Simulate 20 dice rolls with `sample`
- Use `sample` to shuffle `letters[1:10]`
- Using `sequence`, generate 1 2 3 4 1 2 3 1 2 1
- Generate 5 4 3 2 1 6 5 4 3 2 1 using `sequence`


```
set.seed(42)  
sample(1:6, 20, replace=TRUE)
```

```
#> [1] 1 5 1 1 2 4 2 2 1 4 1 5 6 4 2 2 3 1 1 3
```

```
sample(letters[1:10])
```

```
#> [1] "d" "e" "g" "i" "h" "j" "b" "c" "f" "a"
```

```
sequence(4:1)
```

```
#> [1] 1 2 3 4 1 2 3 1 2 1
```

```
sequence(5:6, from=5:6, by=-1)
```

```
#> [1] 5 4 3 2 1 6 5 4 3 2 1
```

Operators

Operators	Definition
<code>+ - * /</code>	plus, minus, times, divide
<code>%/% %% ^</code>	integer quotient, modulo, power
<code>> >= < <=</code>	greater, greater-or-equal, less, less-or-equal
<code>== !=</code>	equals, not equals
<code>! & </code>	not, and (elementwise), or (elementwise)
<code><- -></code>	assignment
<code>\$</code>	list/data.frame element access

Operators	Definition
:	sequence creation
~	model formulae
%*%	matrix product
%in%	test matching
&&	short-circuit and, or (scalar only)
::	namespace access
xor(a, b)	elementwise exclusive OR
<<-	global / parent-env assignment
>	native pipe (R 4.1+)

Operator Precedence (High to Low)

Precedence	Operators
1 (highest)	::, :::
2	\$, @
3	[, [[
4	^ (right-associative)
5	unary -, +
6	:

Operator Precedence (High to Low) Cont'd

Precedence	Operators
7	<code>%any%</code> (including <code>%*%</code> , <code>%/%</code> , <code>%%</code> , <code>%in%</code>)
8	<code>*</code> , <code>/</code>
9	<code>+</code> , <code>-</code>
10	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>==</code> , <code>!=</code>
11	<code>!</code>
12	<code>&</code> , <code>&&</code>
13	<code> </code> , <code> </code>
14	<code> ></code>
15	<code>~</code>
16 (lowest)	<code><-</code> , <code><<-</code> , <code>-></code> , <code>->></code>

%in% and match()

```
x <- c(1, 5, 3, 8, 10)
x %in% c(3, 5, 7)      # which elements of x are in the set?
```

```
#> [1] FALSE  TRUE  TRUE FALSE FALSE
```

```
# match() returns the position of first match (NA if not found)
match(c(3, 5, 99), x)
```

```
#> [1]  3  2 NA
```

%in% and match() Cont'd

```
# practical: filter data frame rows by a set of values
tg <- ToothGrowth
tg[tg$dose %in% c(0.5, 2), ][1:6, ]
```

```
#>      len supp dose
#> 1   4.2   VC   0.5
#> 2  11.5   VC   0.5
#> ... [output truncated]
```


Common Functions

Function	Definition
<code>abs(x)</code>	absolute value
<code>floor(x)</code> , <code>ceiling(x)</code> , <code>trunc(x)</code>	rounding toward $-\infty$, $+\infty$, 0
<code>round(x, digits=0)</code>	round (banker's rounding)
<code>signif(x, digits=6)</code>	significant digits
<code>sqrt(x)</code> , <code>exp(x)</code>	$\sqrt{x}$, e^x
<code>log(x)</code> , <code>log10(x)</code> , <code>log2(x)</code>	natural, base-10, base-2 log
<code>factorial(x)</code> , <code>choose(n, m)</code>	$x!$, $\binom{n}{m}$
<code>sin</code> , <code>cos</code> , <code>tan</code> , <code>asin</code> , <code>acos</code> , <code>atan</code>	trig functions
<code>beta(a, b)</code> , <code>gamma(x)</code>	special functions

Examples: Math Functions

```
abs(-1); floor(5.7); ceiling(-5.7); trunc(-5.7)
```

```
#> [1] 1
```

```
#> [1] 5
```

```
#> [1] -5
```

```
#> [1] -5
```

```
round(5.7); round(5.5); round(2.5) # banker's rounding!
```

```
#> [1] 6
```

```
#> [1] 6
```

```
#> [1] 2
```

```
signif(12345678, 4)
```

```
#> [1] 12350000
```

```
sin(pi / 2); cos(pi / 2) # cos not exactly 0 (floating point)
```

```
#> [1] 1
```

```
#> [1] 6.123234e-17
```

Function	Definition
<code>max(x), min(x), range(x)</code>	max, min, c(min, max)
<code>sum(x), prod(x)</code>	sum, product
<code>mean(x), median(x)</code>	mean, median
<code>quantile(x, p)</code>	quantile at probability <code>p</code>
<code>var(x), sd(x)</code>	sample variance, std deviation
<code>cor(x, y), cov(x, y)</code>	correlation, covariance
<code>sort(x), rev(x)</code>	sorted copy, reversed copy
<code>rank(x), order(x)</code>	ranks, permutation to sort
<code>cumsum, cumprod, cummax, cummin</code>	cumulative operations
<code>diff(x)</code>	lagged differences
<code>pmax(x,y), pmin(x,y)</code>	parallel (elementwise) max/min
<code>table(x)</code>	frequency table
<code>colMeans, colSums</code>	column means/sums
<code>rowMeans, rowSums</code>	row means/sums

Floating Point Surprises

```
0.1 + 0.2 == 0.3          # FALSE!
```

```
#> [1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)  # TRUE (with tolerance)
```

```
#> [1] TRUE
```

```
isTRUE(all.equal(0.1 + 0.2, 0.3)) # safe for if()
```

```
#> [1] TRUE
```

```
# trunc vs floor for negative numbers
```

```
trunc(-3.7); floor(-3.7)    # toward 0 vs toward -Inf
```

```
#> [1] -3
```

```
#> [1] -4
```

Ex 19: Operator Precedence

- Without running, predict: $2 + 3 * 4$, 2^3^2 , $1:3 + 1$, $1:(3+1)$
- Why does -2^2 give -4 instead of 4 ?
- Predict: `TRUE | FALSE & FALSE`
- Write `x <- 5; x %% 3 == 2 & x > 3` and trace operator precedence

Sol 19

```
2 + 3 * 4      # 14: * before +
```

```
#> [1] 14
```

```
2^3^2          # 512: ^ right-associative = 2^9
```

```
#> [1] 512
```

```
1:3 + 1        # 2 3 4: : before +
```

```
#> [1] 2 3 4
```

```
1:(3+1)        # 1 2 3 4
```

```
#> [1] 1 2 3 4
```

```
-2^2           # -4: ^ before unary -
```

```
#> [1] -4
```

```
TRUE | FALSE & FALSE # TRUE: & before |
```

```
#> [1] TRUE
```

```
x <- 5; x %% 3 == 2 & x > 3 # TRUE
```

```
#> [1] TRUE
```

- Compute floor, ceiling, trunc, round of -5.7
- Compute $\binom{10}{3}$ and verify with factorial
- Compute $119 \text{ /\% } 13$ and $119 \text{ \% } 13$. Verify $119 == 13 * (119 \text{ /\% } 13) + 119 \text{ \% } 13$
- `signif(12345678, 4)`


```
floor(-5.7); ceiling(-5.7); trunc(-5.7); round(-5.7)
```

```
#> [1] -6
```

```
#> [1] -5
```

```
#> [1] -5
```

```
#> [1] -6
```

```
choose(10, 3); factorial(10)/(factorial(3)*factorial(7))
```

```
#> [1] 120
```

```
#> [1] 120
```

```
119 %/% 13; 119 %% 13
```

```
#> [1] 9
```

```
#> [1] 2
```

```
119 == 13 * (119 %/% 13) + 119 %% 13          # TRUE
```

```
#> [1] TRUE
```

```
signif(12345678, 4)
```

```
#> [1] 12350000
```

```
v <- c(4, 7, 2, 9, 1, 5, 8, 3, 6, 10).
```

- mean, median, var, sd, range
- `quantile(v, c(0.25, 0.75))` and IQR
- Compute z-scores: $(v - \bar{v})/s$. How many exceed $|z| > 1$?
- Use `diff(v)` to find the largest jump

```
v <- c(4, 7, 2, 9, 1, 5, 8, 3, 6, 10)
mean(v); median(v); var(v); sd(v); range(v)
```

```
#> [1] 5.5
```

```
#> [1] 5.5
```

```
#> [1] 9.166667
```

```
#> [1] 3.02765
```

```
#> [1] 1 10
```

```
quantile(v, c(0.25, 0.75)); IQR(v)
```

```
#> 25% 75%
```

```
#> 3.25 7.75
```

```
#> [1] 4.5
```

```
z <- (v - mean(v)) / sd(v); sum(abs(z) > 1)
```

```
#> [1] 4
```

```
max(abs(diff(v))) # largest jump
```

```
#> [1] 8
```

```
x <- c("A","B","A","C","B","A","C","C","B","A").
```

- Frequency table of x
- Proportion table
- Create `y <- c(1,1,2,1,2,2,1,2,1,2)`. Cross-tabulate x and y; add margins
- Which category of x is most frequent?

```
x <- c("A","B","A","C","B","A","C","C","B","A")  
table(x)
```

```
#> x  
#> A B C  
#> 4 3 3  
#> ... [output truncated]
```

```
prop.table(table(x))
```

```
#> x  
#>  A  B  C  
#> 0.4 0.3 0.3  
#> ... [output truncated]
```

```
y <- c(1,1,2,1,2,2,1,2,1,2)
addmargins(table(x, y))
```

```
#>      y
#> x    1  2 Sum
#>  A    1  3  4
#> ... [output truncated]
```

```
names(which.max(table(x)))
```

```
# "A"
```

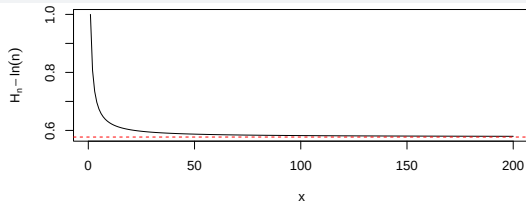
```
#> [1] "A"
```


- Compute $H_n = \sum_{k=1}^n \frac{1}{k}$ for $n \in \{10, 100, 1000, 10000\}$
- Compare each H_n with $\ln(n) + \gamma$ where $\gamma \approx 0.5772$
- Plot $H_n - \ln n$ for $n = 1, \dots, 200$

```
ns <- c(10, 100, 1000, 10000)
Hn <- sapply(ns, function(n) sum(1/1:n))
data.frame(n=ns, Hn=round(Hn,4),
           approx=round(log(ns)+0.5772,4))
```

```
#>      n      Hn approx
#> ... [output truncated]
```

```
x <- 1:200; y <- cumsum(1/x) - log(x)
plot(x, y, type="l", ylab=expression(H[n]-ln(n)))
abline(h=0.5772, col="red", lty=2)
```



- Clip vector $v \leftarrow c(-5, 3, 12, -2, 8, 25)$ to range $[0, 10]$ using `pmin` and `pmax`
- Given two portfolios $a \leftarrow c(5, 8, 3, 9)$ and $b \leftarrow c(7, 2, 6, 4)$, compute the “best of both” portfolio
- Compute $\sum_{k=1}^{100} \min(2^k, k^4)$ without loops

```
v <- c(-5, 3, 12, -2, 8, 25)
pmin(pmax(v, 0), 10)           # clip to [0,10]
```

```
#> [1] 0 3 10 0 8 10
```

```
a <- c(5,8,3,9); b <- c(7,2,6,4)
pmax(a, b)
```

```
#> [1] 7 8 6 9
```

```
k <- 1:100; sum(pmin(2^k, k^4))
```

```
#> [1] 2050220551
```

```
x <- c(3, -1, 4, -1, 5, -9, 2, 6).
```

- Compute `cumsum(x)`. At which index does it first become negative?
- Maximum drawdown: $\max_i (\text{cummax}_i - \text{cumsum}_i)$
- Compute `cumprod(1 + c(0.05, -0.02, 0.03, -0.01))` (portfolio growth)

```
x <- c(3, -1, 4, -1, 5, -9, 2, 6)
cs <- cumsum(x); cs
```

```
#> [1] 3 2 6 5 10 1 3 9
```

```
which(cs < 0)[1] # first negative
```

```
#> [1] NA
```

```
cm <- cummax(cs); max(cm - cs) # max drawdown
```

```
#> [1] 9
```

```
cumprod(1 + c(0.05, -0.02, 0.03, -0.01)) # growth
```

```
#> [1] 1.050000 1.029000 1.059870 1.049271
```

```
d <- data.frame(name=c("Zara","Amy","Mike","Bob"), score=c(88,95,72,95),  
age=c(22,20,25,21)).
```

- Sort by score ascending
- Sort by score descending, ties broken by age ascending
- Find the name of the person with the highest score (youngest if tied)

```
d <- data.frame(name=c("Zara","Amy","Mike","Bob"),
                score=c(88,95,72,95), age=c(22,20,25,21))
d[order(d$score), ]
```

```
#>   name score age
#> 3 Mike    72  25
#> 1 Zara    88  22
#> ... [output truncated]
```

```
d[order(-d$score, d$age), ]
```

```
#>   name score age
#> 2 Amy    95  20
#> 4 Bob    95  21
#> ... [output truncated]
```

```
d[order(-d$score, d$age), ]$name[1]           # Bob
```

```
#> [1] "Amy"
```


Data Frames

What Is a Data Frame?

- A rectangular table: one row per observation, one column per variable.
- **Technically:** a named list of equal-length vectors (we'll revisit this in Lists).

```
tg <- ToothGrowth      # built-in data; used throughout
head(tg, 4)
```

```
#>      len supp dose
#> 1  4.2   VC  0.5
#> 2 11.5   VC  0.5
#> ... [output truncated]
```

```
# Each column is a vector; use $ to extract
tg$len[1:5]
```

```
#> [1]  4.2 11.5  7.3  5.8  6.4
```

```
str(tg)
```

```
#> 'data.frame':    60 obs. of  3 variables:
#> $ len : num  4.2 11.5 7.3 5.8 6.4 10 11.2 11.2 5.2 7 ...
#> $ supp: Factor w/ 2 levels "OJ","VC": 2 2 2 2 2 2 2 2 2 2 ...
#> ... [output truncated]
```

```
nrow(tg); ncol(tg); dim(tg); names(tg)
```

```
#> [1] 60
```

```
#> [1] 3
```

```
#> [1] 60 3
```

```
#> [1] "len" "supp" "dose"
```

```
summary(tg)
```

```
#>      len      supp      dose  
#> Min.   : 4.20    0J:30    Min.   :0.500  
#> 1st Qu.:13.07    VC:30    1st Qu.:0.500  
#> ... [output truncated]
```

Indexing: [row, col]

```
tg[3, 1]          # row 3, column 1
```

```
#> [1] 7.3
```

```
tg$len[3]         # equivalent via $
```

```
#> [1] 7.3
```

```
(z <- tg[2:4, c(1, 3)])  # rows 2-4, columns 1 & 3
```

```
#>    len dose
```

```
#> 2 11.5  0.5
```

```
#> 3  7.3  0.5
```

```
#> ... [output truncated]
```

```
head(tg[, c(1, 3)], 3)  # all rows, columns 1 & 3
```

```
#>   len dose  
#> 1  4.2  0.5  
#> 2 11.5  0.5  
#> ... [output truncated]
```

```
head(tg[, -2], 3)      # all rows, remove column 2
```

```
#>   len dose  
#> 1  4.2  0.5  
#> 2 11.5  0.5  
#> ... [output truncated]
```

Creating Data Frames

```
age <- c(55, 58, 45)
name <- c('Alice', 'Bill', 'Cathy')
(d <- data.frame(age, name))
```

```
#>   age name
#> 1  55 Alice
#> 2  58  Bill
#> ... [output truncated]
• Vectors of different lengths are not allowed!
```

```
blood <- c('O', 'B', 'A', 'AB')
data.frame(age, name, blood)
```

```
#> Error in data.frame(age, name, blood): arguments imply differing number of rows: 3, 4
```

```
# with custom column names
data.frame(p1 = pred1, p2 = pred2)
```

```
# from a named list of vectors
l <- list(p1 = pred1, p2 = pred2)
as.data.frame(l)
```

```
# from row data via rbind
r1 <- data.frame(a=1, b=2, c="X")
r2 <- data.frame(a=3, b=4, c="Y")
rbind(r1, r2)
```

Row Filtering with Logical Conditions

```
tg_vc <- tg[tg$supp == 'VC', ]  
tg_oj <- tg[tg$supp == 'OJ', ]  
mean(tg_vc$len); mean(tg_oj$len)
```

```
#> [1] 16.96333
```

```
#> [1] 20.66333
```


Row Filtering with Logical Conditions Cont'd

```
# AND condition
```

```
tg[tg$supp == 'OJ' & tg$len < 8.8, ]
```

```
#>      len supp dose
```

```
#> 37 8.2   OJ  0.5
```

```
# OR condition
```

```
head(tg[tg$len > 28 | tg$dose == 1.0, ], 6)
```

```
#>      len supp dose
```

```
#> 11 16.5   VC    1
```

```
#> 12 16.5   VC    1
```

```
#> ... [output truncated]
```

- Note the comma (,) and the use of ==, not =!

subset() — Cleaner Filtering

```
# equivalent to tg[tg$supp == 'OJ' & tg$len > 20, ]
```

```
subset(tg, supp == 'OJ' & len > 20)
```

```
#>      len supp dose
#> 32 21.5   OJ  0.5
#> 42 23.3   OJ  1.0
#> ... [output truncated]
```

```
# select specific columns
```

```
subset(tg, dose == 2, select = c(len, supp))
```

```
#>      len supp
#> 21 23.6   VC
#> 22 18.5   VC
#> ... [output truncated]
```

with() and within()

```
# with() evaluates an expression in the data frame environment
with(tg, mean(len[supp == 'VC']))
```

```
#> [1] 16.96333
```

```
# within() modifies and returns the data frame
tg2 <- within(tg, {
  len_cm <- len / 10
  log_len <- log(len)
})
head(tg2, 3)
```

```
#>   len supp dose  log_len len_cm
#> 1  4.2   VC  0.5 1.435085  0.42
#> 2 11.5   VC  0.5 2.442347  1.15
#> ... [output truncated]
```

Adding, Removing, Renaming Columns

```
d <- data.frame(x = 1:3, y = c(10, 20, 30))
d$z <- d$x + d$y          # add column with $
d[['w']] <- c('a', 'b', 'c') # add column with [[]]
d
```

```
#>   x  y  z w
#> 1 1 10 11 a
#> 2 2 20 22 b
#> ... [output truncated]
```

```
d$z <- NULL              # remove column
names(d)[names(d) == 'w'] <- 'label' # rename
d
```

```
#>   x  y label
#> 1 1 10     a
#> 2 2 20     b
#> ... [output truncated]
```

Sorting Data Frames

```
d <- data.frame(name = c('Zara', 'Amy', 'Mike'),  
                score = c(88, 95, 72))  
d[order(d$score), ]           # ascending
```

```
#>   name score  
#> 3 Mike    72  
#> 1 Zara    88  
#> ... [output truncated]
```

```
d[order(d$score, decreasing = TRUE), ]   # descending
```

```
#>   name score  
#> 2 Amy     95  
#> 1 Zara    88  
#> ... [output truncated]
```

Sorting Data Frames Cont'd

```
# sort by multiple columns
d2 <- data.frame(g = c('A','B','A','B'), x = c(3,1,2,4))
d2[order(d2$g, d2$x), ]
```

```
#>   g x
#> 3 A 2
#> 1 A 3
#> ... [output truncated]
```

Handling NAs

```
df <- data.frame(x = c(1, NA, 3, 4, 5), y = c(1, 2, NA, 4, 5))  
na.omit(df)                                # remove rows with any NA
```

```
#>   x y  
#> 1 1 1  
#> 4 4 4  
#> ... [output truncated]
```

Handling NAs Cont'd

```
complete.cases(df)           # logical: which rows complete?
```

```
#> [1]  TRUE FALSE FALSE  TRUE  TRUE
```

```
df[complete.cases(df), ]
```

```
#>   x y
```

```
#> 1 1 1
```

```
#> 4 4 4
```

```
#> ... [output truncated]
```

```
df$x[is.na(df$x)] <- 0        # replace specific column NAs
```

```
df[is.na(df)] <- 0           # replace all NAs
```


Combining Data Frames: cbind, rbind

```
# column-bind (side by side) --- same number of rows
df1 <- data.frame(a = c(1, 2)); df2 <- data.frame(b = c(7, 8))
cbind(df1, df2)
```

```
#>   a b
#> 1 1 7
#> 2 2 8
#> ... [output truncated]
```

```
# row-bind (stack) --- same column names
df1 <- data.frame(x = c("a", "a"), y = c(5, 6))
df2 <- data.frame(x = c("b", "b"), y = c(9, 10))
rbind(df1, df2)
```

```
#>   x y
#> 1 a 5
#> 2 a 6
#> ... [output truncated]
```

The Split–Apply–Combine Paradigm

Step	Base R tool	What it does
Split	<code>split()</code>	partition data by factor → list
Apply	<code>lapply/sapply/tapply/...</code>	run function on each piece
Combine	<code>unsplit/rbind/do.call/...</code>	reassemble results
All-in-one	<code>aggregate()</code>	split + apply + combine in one call
Horizontal combine	<code>merge()</code>	join two tables by key

```
scores <- c(88, 92, 78, 95, 85, 70, 91, 83)
group  <- c('A','B','A','B','A','B','A','B')
(sg <- split(scores, group))
```

```
#> $A
#> [1] 88 78 85 91
#>
#> ... [output truncated]
```

```
sapply(sg, mean)      # per-group mean
```

```
#>      A      B
#> 85.5 85.0
```

```
# split a data frame --- each element is a complete data frame
tg_split <- split(tg, tg$supp)
head(tg_split$OJ, 3)
```

```
#>      len supp dose
#> 31 15.2   OJ  0.5
#> 32 21.5   OJ  0.5
#> ... [output truncated]
```

split() — Multi-Factor and unsplit()

```
tg_split2 <- split(tg, list(tg$supp, tg$dose))  
names(tg_split2)
```

```
#> [1] "OJ.0.5" "VC.0.5" "OJ.1"   "VC.1"   "OJ.2"  
#> [6] "VC.2"
```

```
# unsplit: put pieces back in their original positions  
sg <- split(1:8, rep(c('A','B'), 4))  
sg$A <- sg$A * 10; sg$B <- sg$B * (-1)  
unsplit(sg, rep(c('A','B'), 4))
```

```
#> [1] 10 -2 30 -4 50 -6 70 -8
```

The split + lapply + rbind Pipeline

```
# add a z-score column within each group
standardize <- function(df) {
  df$len_z <- (df$len - mean(df$len)) / sd(df$len)
  df
}
parts <- split(tg, tg$supp)
parts <- lapply(parts, standardize)
tg2 <- do.call(rbind, parts)
head(tg2[order(as.numeric(rownames(tg2)))], ], 4)
```

```
#>      len supp dose      len_z
#> OJ.31 15.2   OJ  0.5 -0.8270809
#> OJ.32 21.5   OJ  0.5  0.1266610
#> ... [output truncated]
```

```
# formula syntax: response ~ grouping  
aggregate(len ~ supp, data = tg, FUN = mean)
```

```
#>   supp      len  
#> 1    OJ 20.66333  
#> 2    VC 16.96333  
#> ... [output truncated]
```

```
aggregate(len ~ supp + dose, data = tg, FUN = mean)
```

```
#>   supp dose    len  
#> 1    OJ  0.5 13.23  
#> 2    VC  0.5  7.98  
#> ... [output truncated]
```

aggregate() — Custom Summary and Multiple Responses

```
# custom function: result is a matrix column!
```

```
aggregate(len ~ supp, data = tg,  
          FUN = function(x) c(mean = mean(x), sd = sd(x)))
```

```
#>   supp  len.mean   len.sd  
#> 1    OJ 20.663333  6.605561  
#> 2    VC 16.963333  8.266029  
#> ... [output truncated]
```

```
# multiple response columns with cbind()
```

```
aggregate(cbind(mpg, hp, wt) ~ cyl, data = mtcars, FUN = mean)
```

```
#>   cyl      mpg      hp      wt  
#> 1    4 26.66364 82.63636 2.285727  
#> 2    6 19.74286 122.28571 3.117143  
#> ... [output truncated]
```


aggregate() — Expand Matrix Column and . Shorthand

```
# expand matrix column into proper data frame
res <- aggregate(len ~ supp, data = tg,
                 FUN = function(x) c(m = mean(x), s = sd(x)))
cbind(res[1], as.data.frame(res$len))
```

```
#>      supp      m      s
#> 1    OJ 20.66333 6.605561
#> 2    VC 16.96333 8.266029
#> ... [output truncated]
```

```
# . means "all other columns"
aggregate(. ~ cyl, data = mtcars[, c('mpg','hp','wt','cyl')],
          FUN = mean)
```

```
#>      cyl      mpg      hp      wt
#> 1     4 26.66364 82.63636 2.285727
#> 2     6 19.74286 122.28571 3.117143
#> ... [output truncated]
```

merge() — Joining Data Frames

```
customers <- data.frame(id = 1:4,  
  name = c('Alice', 'Bob', 'Cathy', 'Dan'))  
orders <- data.frame(cust_id = c(2, 3, 3, 1, 5),  
  product = c('pen', 'book', 'ink', 'paper', 'tape'),  
  amount = c(12, 35, 8, 20, 5))  
# inner join (default): only matched rows  
merge(customers, orders, by.x = 'id', by.y = 'cust_id')
```

```
#>   id  name product amount  
#> 1  1 Alice  paper     20  
#> 2  2  Bob   pen     12  
#> ... [output truncated]
```

merge() — Join Types

```
# left join: keep all customers (NA for no match)
merge(customers, orders, by.x='id', by.y='cust_id', all.x=TRUE)
```

```
#>   id name product amount
#> 1  1 Alice  paper     20
#> 2  2  Bob   pen     12
#> ... [output truncated]
```

SQL	merge() arguments	Keeps
INNER JOIN	default (<code>all=FALSE</code>)	matched rows only
LEFT JOIN	<code>all.x = TRUE</code>	all rows in <code>x</code>
RIGHT JOIN	<code>all.y = TRUE</code>	all rows in <code>y</code>
FULL OUTER JOIN	<code>all = TRUE</code>	all rows in both

merge() — Multi-Column Keys and Suffixes

```
# multi-column key
sales <- data.frame(year=c(2023,2023,2024), qtr=c(1,2,1), rev=c(100,120,110))
targets <- data.frame(year=c(2023,2023,2024), qtr=c(1,2,1), target=c(95,115,105))
merge(sales, targets, by = c('year', 'qtr'))
```

```
#>   year qtr rev target
#> 1 2023   1 100     95
#> 2 2023   2 120    115
#> ... [output truncated]
```

```
# duplicate column names get suffixes
df1 <- data.frame(id = 1:3, value = c(10, 20, 30))
df2 <- data.frame(id = 1:3, value = c(100, 200, 300))
merge(df1, df2, by = 'id', suffixes = c('_old', '_new'))
```

```
#>   id value_old value_new
#> 1  1         10        100
#> 2  2         20        200
#> ... [output truncated]
```

- Create a data frame with columns name (Alice, Bob, Cathy), age (25, 30, 28), score (88.5, 92.3, 76.8)
- Show str, dim, names, summary
- What class is df\$name?
- Access score of row 2 by name and by position

```
df <- data.frame(name=c("Alice","Bob","Cathy"),  
                 age=c(25,30,28), score=c(88.5,92.3,76.8))  
str(df); dim(df); names(df)
```

```
#> 'data.frame':    3 obs. of  3 variables:  
#> $ name : chr  "Alice" "Bob" "Cathy"  
#> $ age  : num  25 30 28  
#> ... [output truncated]  
#> [1] 3 3  
#> [1] "name" "age" "score"
```

```
class(df$name)
```

```
#> [1] "character"
```

```
df$score[2]; df[2, "score"]
```

```
#> [1] 92.3  
#> [1] 92.3
```

Using `tg <- ToothGrowth`:

- Rows where `supp == "OJ"` and `dose >= 1`
- Mean tooth length for VC vs OJ
- Rows where `len` is above its overall median

```
tg <- ToothGrowth  
head(tg[tg$supp == "OJ" & tg$dose >= 1, ], 3)
```

```
#>      len supp dose  
#> 41 19.7   OJ    1  
#> 42 23.3   OJ    1  
#> ... [output truncated]
```

```
tapply(tg$len, tg$supp, mean)
```

```
#>      OJ      VC  
#> 20.66333 16.96333
```

```
head(tg[tg$len > median(tg$len), ], 3)
```

```
#>      len supp dose  
#> 15 22.5   VC    1  
#> 21 23.6   VC    2  
#> ... [output truncated]
```


Ex 29: Adding and Removing Columns

Start with `df <- data.frame(x=1:5, y=c(10,20,30,40,50))`.

- Add column `z = x * y`
- Add column `label` using `paste0("item", 1:5)`
- Remove column `z`
- Rename `y` to `value`

```
df <- data.frame(x=1:5, y=c(10,20,30,40,50))
df$z <- df$x * df$y
df$label <- paste0("item", 1:5)
df$z <- NULL
names(df)[names(df)=="y"] <- "value"; df
```

```
#>   x value label
#> 1 1    10 item1
#> 2 2    20 item2
#> ... [output truncated]
```

Using `tg <- ToothGrowth`:

- Use `with()` to compute `mean(len[dose == 2])`
- Use `within()` to add `log_len <- log(len)` and `len_centered <- len - mean(len)`

```
tg <- ToothGrowth  
with(tg, mean(len[dose == 2]))
```

```
#> [1] 26.1
```

```
tg2 <- within(tg, {  
  log_len <- log(len)  
  len_centered <- len - mean(len)  
})  
head(tg2, 3)
```

```
#>   len supp dose len_centered log_len  
#> 1  4.2   VC  0.5   -14.613333 1.435085  
#> 2 11.5   VC  0.5    -7.313333 2.442347  
#> ... [output truncated]
```

```
df <- data.frame(x=c(1,NA,3,4,NA), y=c(NA,2,3,NA,5), z=c(1,2,3,4,5)).
```

- Which rows are complete?
- Remove all incomplete rows
- Replace NAs in column x with the mean of x (ignoring NAs)
- Count NAs per column

```
df <- data.frame(x=c(1,NA,3,4,NA), y=c(NA,2,3,NA,5), z=1:5)
complete.cases(df)
```

```
#> [1] FALSE FALSE TRUE FALSE FALSE
```

```
df[complete.cases(df), ]
```

```
#>   x y z
```

```
#> 3 3 3 3
```

```
df$x[is.na(df$x)] <- mean(df$x, na.rm=TRUE); df
```

```
#>           x y z
```

```
#> 1 1.000000 NA 1
```

```
#> 2 2.666667  2 2
```

```
#> ... [output truncated]
```

```
colSums(is.na(df))
```

```
#> x y z
```

```
#> 0 2 0
```

Ex 32: merge()

```
students <- data.frame(id=1:4, name=c("A","B","C","D")). grades <- data.frame(sid=c(1,2,2,4),  
course=c("Math","Math","Eng","Eng"), grade=c(85,90,78,92)).
```

- Inner join on id
- Left join (keep all students)
- How many rows in each result? Why?

```
students <- data.frame(id=1:4, name=c("A","B","C","D"))
grades <- data.frame(sid=c(1,2,2,4),
  course=c("Math","Math","Eng","Eng"), grade=c(85,90,78,92))
merge(students, grades, by.x="id", by.y="sid")      # inner
```

```
#>   id name course grade
#> 1  1    A   Math    85
#> 2  2    B   Math    90
#> ... [output truncated]
```

```
merge(students, grades, by.x="id", by.y="sid",
  all.x=TRUE)                                     # left
```

```
#>   id name course grade
#> 1  1    A   Math    85
#> 2  2    B   Math    90
#> ... [output truncated]
```

```
# inner: 4 rows; left: 5 rows (C has NA)
```


Using mtcars:

- Mean mpg by cyl
- Mean and sd of mpg by cyl and am
- Max hp by cyl

```
aggregate(mpg ~ cyl, data=mtcars, FUN=mean)
```

```
#>   cyl      mpg  
#> 1    4 26.66364  
#> 2    6 19.74286  
#> ... [output truncated]
```

```
aggregate(mpg ~ cyl + am, data=mtcars,  
  FUN=function(x) c(m=mean(x), s=sd(x)))
```

```
#>   cyl am      mpg.m      mpg.s  
#> 1   4  0 22.9000000  1.4525839  
#> 2   6  0 19.1250000  1.6317169  
#> ... [output truncated]
```

```
aggregate(hp ~ cyl, data=mtcars, FUN=max)
```

```
#>   cyl  hp  
#> 1   4 113  
#> 2   6 175  
#> ... [output truncated]
```

- Create two 1-column data frames and `cbind` them
- Create two data frames with same columns and `rbind`
- Use `do.call(rbind, list_of_dfs)` with 3 data frames

```
d1 <- data.frame(a=1:3); d2 <- data.frame(b=4:6)
cbind(d1, d2)
```

```
#>   a b
#> 1 1 4
#> ... [output truncated]
```

```
e1 <- data.frame(x="a", y=1); e2 <- data.frame(x="b", y=2)
rbind(e1, e2)
```

```
#>   x y
#> 1 a 1
#> ... [output truncated]
```

```
dfs <- list(data.frame(x=1,y=2), data.frame(x=3,y=4),
            data.frame(x=5,y=6))
do.call(rbind, dfs)
```

```
#>   x y
#> 1 1 2
#> ... [output truncated]
```

- Show `is.list(mtcars)` is TRUE. What does `typeof(mtcars)` return?
- Access the `mpg` column using `[[` notation
- What is the difference between `mtcars["mpg"]` and `mtcars[["mpg"]]`?
- Convert `mtcars[1:3, 1:3]` to a list and back

```
is.list(mtcars); typeof(mtcars)           # TRUE; "list"
```

```
#> [1] TRUE
```

```
#> [1] "list"
```

```
head(mtcars[["mpg"]])
```

```
#> [1] 21.0 21.0 22.8 21.4 18.7 18.1
```

```
class(mtcars[["mpg"]]); class(mtcars[["mpg"]])      # df vs numeric
```

```
#> [1] "data.frame"
```

```
#> [1] "numeric"
```

```
l <- as.list(mtcars[1:3, 1:3]); as.data.frame(l)
```

```
#>   mpg cyl disp
```

```
#> 1 21.0   6  160
```

```
#> 2 21.0   6  160
```

```
#> ... [output truncated]
```

- Split `iris` by `Species`. What type is the result?
- Compute mean `Sepal.Length` per species using `split + sapply`
- Recombine with `do.call(rbind, ...)`


```
sp <- split(iris, iris$Species)
class(sp); names(sp) # list
```

```
#> [1] "list"
```

```
#> [1] "setosa"      "versicolor" "virginica"
```

```
sapply(sp, function(d) mean(d$Sepal.Length))
```

```
#>      setosa versicolor  virginica  
#>      5.006      5.936      6.588
```

```
head(do.call(rbind, sp), 4)
```

```
#>      Sepal.Length Sepal.Width Petal.Length  
#> setosa.1          5.1          3.5          1.4  
#> setosa.2          4.9          3.0          1.4  
#> ... [output truncated]
```

Using mtcars:

- Select only mpg, cyl, hp for cars with mpg > 25
- Exclude drat and wt columns
- Why can you write subset(mtcars, mpg > 25) without quoting mpg?

```
subset(mtcars, mpg > 25, select=c(mpg, cyl, hp))
```

```
#>           mpg cyl  hp
#> Fiat 128    32.4   4  66
#> Honda Civic 30.4   4  52
#> ... [output truncated]
```

```
head(subset(mtcars, select=-c(drat, wt)), 3)
```

```
#>           mpg cyl disp  hp  qsec vs  am gear carb
#> Mazda RX4    21.0   6  160 110 16.46  0   1    4    4
#> Mazda RX4 Wag 21.0   6  160 110 17.02  0   1    4    4
#> ... [output truncated]
```

```
# subset uses non-standard evaluation: it captures
# the expression and evaluates it within the data frame
```

Ex 38: Sorting with Multiple Keys

```
df <- data.frame(dept=c("A","B","A","B","A"), name=c("Zoe","Amy","Bob","Cal","Amy"),  
salary=c(50,60,70,55,65)).
```

- Sort by dept ascending, then salary descending
- Find the highest-paid person in each department (using `order + !duplicated`)

```
df <- data.frame(dept=c("A","B","A","B","A"),  
  name=c("Zoe","Amy","Bob","Cal","Amy"), salary=c(50,60,70,55,65))  
df[order(df$dept, -df$salary), ]
```

```
#>   dept name salary  
#> 3    A  Bob     70  
#> 5    A  Amy     65  
#> ... [output truncated]
```

```
sorted <- df[order(df$dept, -df$salary), ]  
sorted[!duplicated(sorted$dept), ]
```

```
#>   dept name salary  
#> 3    A  Bob     70  
#> 2    B  Amy     60  
#> ... [output truncated]
```

- Use `colMeans` on `mtcars[,1:4]`
- Use `sapply` to find `max` of each column
- Standardize (z-score) all numeric columns using `scale()`. Compare with manual sweep

```
colMeans(mtcars[,1:4])
```

```
#>      mpg      cyl    disp      hp  
#> 20.09062  6.18750 230.72188 146.68750
```

```
sapply(mtcars[,1:4], max)
```

```
#>  mpg  cyl  disp  hp  
#> 33.9  8.0 472.0 335.0
```

```
head(scale(mtcars[,1:2]), 3)
```

```
#>           mpg      cyl  
#> Mazda RX4    0.1508848 -0.1049878  
#> Mazda RX4 Wag 0.1508848 -0.1049878  
#> ... [output truncated]
```



```
m <- as.matrix(mtcars[,1:2])  
head(sweep(sweep(m, 2, colMeans(m)), 2, apply(m,2,sd), "/"), 3)
```

```
#>                mpg        cyl  
#> Mazda RX4      0.1508848 -0.1049878  
#> Mazda RX4 Wag 0.1508848 -0.1049878  
#> ... [output truncated]
```

- Create `grades <- data.frame(name=c("A","B","C","D","E"), hw=c(85,92,78,95,88), exam=c(90,85,92,88,76))`
- Add `final = 0.4*hw + 0.6*exam`
- Add letter using nested `ifelse` (A/B/C/F)
- Sort by `final` descending

```
grades <- data.frame(name=c("A","B","C","D","E"),  
  hw=c(85,92,78,95,88), exam=c(90,85,92,88,76))  
grades$final <- 0.4*grades$hw + 0.6*grades$exam  
grades$letter <- ifelse(grades$final>=90,"A",  
  ifelse(grades$final>=80,"B",  
    ifelse(grades$final>=70,"C","F")))  
grades[order(-grades$final), ]
```

```
#>   name hw exam final letter  
#> 4    D 95   88  90.8     A  
#> 1    A 85   90  88.0     B  
#> ... [output truncated]
```

Using `ToothGrowth`:

- Split by both `supp` and `dose`; how many groups?
- Compute mean `len` per group using `sapply`
- Add a z-score column within each group using `split + lapply + do.call(rbind,...)`
- Use `unsplit` to restore original ordering after modifying split pieces

```
tg <- ToothGrowth  
sp <- split(tg, list(tg$supp, tg$dose))  
length(sp)
```

6

#> [1] 6

```
sapply(sp, \(d) mean(d$len))
```

```
#> OJ.0.5 VC.0.5  OJ.1   VC.1   OJ.2   VC.2  
#> 13.23   7.98  22.70  16.77  26.06  26.14
```

```
parts <- lapply(sp, \(d) {
  d$len_z <- (d$len - mean(d$len)) / sd(d$len); d })
tg2 <- do.call(rbind, parts)
head(tg2[order(as.numeric(rownames(tg2)))], ], 4)
```

```
#>           len supp dose      len_z
#> 0J.0.5.31 15.2   0J  0.5  0.4417329
#> 0J.0.5.32 21.5   0J  0.5  1.8543813
#> ... [output truncated]
```

```
# unsplit example
sg <- split(1:8, rep(c('A','B'), 4))
sg$A <- sg$A * 10; sg$B <- sg$B * (-1)
unsplit(sg, rep(c('A','B'), 4))
```

```
#> [1] 10 -2 30 -4 50 -6 70 -8
```

Using ToothGrowth:

- Use aggregate with FUN = function(x) c(m=mean(x), s=sd(x)). What type is the result column?
- Expand the matrix column into a proper flat data frame
- Use cbind(mpg, hp, wt) ~ cyl to summarize multiple columns of mtcars
- Use . shorthand: aggregate(. ~ cyl, ...) on a subset of mtcars

```
tg <- ToothGrowth
res <- aggregate(len ~ supp, data = tg,
  FUN = function(x) c(m=mean(x), s=sd(x)))
str(res)                                     # matrix col!
```

```
#> 'data.frame':    2 obs. of  2 variables:
#> $ supp: Factor w/ 2 levels "OJ","VC": 1 2
#> $ len : num [1:2, 1:2] 20.66 16.96 6.61 8.27
#> ... [output truncated]
```

```
cbind(res[1], as.data.frame(res$len))      # flatten
```

```
#>   supp      m      s
#> 1   OJ 20.66333 6.605561
#> 2   VC 16.96333 8.266029
#> ... [output truncated]
```



```
aggregate(cbind(mpg, hp, wt) ~ cyl,  
          data = mtcars, FUN = mean)
```

```
#>   cyl      mpg      hp      wt  
#> 1    4 26.66364 82.63636 2.285727  
#> 2    6 19.74286 122.28571 3.117143  
#> ... [output truncated]
```

```
aggregate(. ~ cyl,  
          data = mtcars[, c('mpg','hp','wt','cyl')], FUN = mean)
```

```
#>   cyl      mpg      hp      wt  
#> 1    4 26.66364 82.63636 2.285727  
#> 2    6 19.74286 122.28571 3.117143  
#> ... [output truncated]
```

```
cust <- data.frame(id=1:4, name=c("A","B","C","D")). ord <- data.frame(cid=c(2,3,3,5),  
prod=c("pen","book","ink","tape")).
```

- Inner join. Which rows are dropped and why?
- Left join. What appears for customer D?
- Right join. What appears for cid=5?
- Full outer join

```
cust <- data.frame(id=1:4, name=c("A","B","C","D"))
ord <- data.frame(cid=c(2,3,3,5),
                  prod=c("pen","book","ink","tape"))
merge(cust, ord, by.x="id", by.y="cid") # inner
```

```
#>   id name prod
#> 1  2    B  pen
#> 2  3    C book
#> ... [output truncated]
```

```
merge(cust, ord, by.x="id", by.y="cid", all.x=TRUE) # left
```

```
#>   id name prod
#> 1  1    A <NA>
#> 2  2    B  pen
#> ... [output truncated]
```

```
merge(cust, ord, by.x="id", by.y="cid", all.y=TRUE) # right
```

```
#>   id name prod  
#> 1  2    B  pen  
#> 2  3    C book  
#> ... [output truncated]
```

```
merge(cust, ord, by.x="id", by.y="cid", all=TRUE) # full
```

```
#>   id name prod  
#> 1  1    A <NA>  
#> 2  2    B  pen  
#> ... [output truncated]
```

Ex 44: Multi-Table Merge with Reduce

- Create `products(pid, name)`, `orders(oid, pid, qty)`, `prices(pid, unit_price)`
- Inner-join all three; compute `total = qty * unit_price`
- Add `regions(pid, region)`. Use Reduce to merge all four

```

products <- data.frame(pid=1:5,
  name=c('Pen','Book','Ink','Paper','Tape'))
orders <- data.frame(oid=101:106,
  pid=c(2,1,3,2,5,1), qty=c(3,10,2,5,1,8))
prices <- data.frame(pid=1:5,
  unit_price=c(1.5, 12, 8, 3, 2.5))
m <- merge(merge(products, orders, by='pid'),
  prices, by='pid')
m$total <- m$qty * m$unit_price; m[, c('oid','name','total')]

```

```

#>   oid name total
#> ... [output truncated]

```

```

regions <- data.frame(pid=1:5, region=c('N','S','N','S','N'))
Reduce(\(x, y) merge(x, y, by='pid', all=TRUE),
  list(products, orders, prices, regions))

```

```

#>   pid  name oid qty unit_price region
#> ... [output truncated]

```

Ex 45: vapply vs sapply Safety

- `sapply(list(a=1:3, b=4:6), identity)` returns a matrix. `sapply(list(a=1:3, b=4:5), identity)` returns a list. Demonstrate
- Show that `vapply(list(a=1:3, b=4:6), identity, numeric(1))` throws an error
- Use `vapply` correctly with `FUN.VALUE = numeric(3)` for same-length vectors
- Use `vapply(iris[,1:4], median, numeric(1))` to safely compute medians

```
sapply(list(a=1:3, b=4:6), identity)      # matrix
```

```
#>      a b  
#> [1,] 1 4  
#> [2,] 2 5  
#> ... [output truncated]
```

```
sapply(list(a=1:3, b=4:5), identity)      # list!
```

```
#> $a  
#> [1] 1 2 3  
#>  
#> ... [output truncated]
```

```
vapply(list(a=1:3, b=4:6), identity, numeric(1))  # error!
```

```
#> Error in vapply(list(a = 1:3, b = 4:6), identity, numeric(1)): values must be length 1,  
#> but FUN(X[[1]]) result is length 3
```

```
vapply(list(a=1:3, b=4:6), identity, numeric(3))  # OK
```

```
#>      a b  
#> [1,] 1 4  
#> [2,] 2 5  
#> ... [output truncated]
```



```
nested <- list(a=1:3, b=list(c=4:6, d=list(e=7:9, f="text"))).
```

- Use `rapply` with `sum`, `classes="integer"` to sum only integer leaves
- Same with `how="list"` to preserve structure
- Use `how="replace"` to multiply all integer leaves by 10
- What happens to the "text" element in each case?

Sol 46

```
nested <- list(a=1:3,  
  b=list(c=4:6, d=list(e=7:9, f="text")))  
rapply(nested, sum, classes="integer")           # unlist
```

```
#>      a    b.c b.d.e  
#>      6     15     24
```

```
rapply(nested, sum, classes="integer", how="list")  # structure
```

```
#> $a  
#> [1] 6  
#>  
#> ... [output truncated]
```

```
rapply(nested, \(x) x*10, classes="integer",  
  how="replace")           # replace
```

```
#> $a  
#> [1] 10 20 30  
#>  
#> ... [output truncated]
```

```
# "text" is left unchanged (not class "integer")
```

```
r <- c(0.02, -0.01, 0.03, -0.04, 0.01, -0.02, 0.05).
```

- Use Reduce with accumulate=TRUE to compute cumulative wealth $W_t = W_{t-1}(1 + r_t)$, starting from $W_0 = 1$
- Compute running max of cumulative wealth using Reduce and max
- Compute drawdown: $DD_t = 1 - W_t / \max_{s \leq t} W_s$
- Verify with vectorized $1 - \text{cumprod}(1+r) / \text{cummax}(\text{cumprod}(1+r))$

```
r <- c(0.02, -0.01, 0.03, -0.04, 0.01, -0.02, 0.05)
W <- Reduce\(w, ri) w*(1+ri), r, init=1,
      accumulate=TRUE)[-1]
Wmax <- Reduce\(mx, w) max(mx, w), W,
      accumulate=TRUE)
DD <- 1 - W / Wmax; round(DD, 6)
```

```
#> [1] 0.000000 0.010000 0.000000 0.040000 0.030400
#> [6] 0.049792 0.002282
```

```
DD2 <- 1 - cumprod(1+r) / cummax(cumprod(1+r))
all.equal(DD, DD2) # TRUE
```

```
#> [1] TRUE
```

```
words <- c("apple","Banana","cherry","Date","123","fig","GRAPE","hi").
```

- Use `Filter` to keep only all-lowercase words
- Use `Filter + Negate` to remove strings containing digits
- Find first word with `> 5` characters using `Find`; find its position with `Position`
- Chain with `|>`: filter lowercase $\rightarrow$ filter length $> 3 \rightarrow$ sort

```
words <- c("apple","Banana","cherry","Date","123",  
          "fig","GRAPE","hi")  
Filter(\(w) w == tolower(w), words)
```

```
#> [1] "apple" "cherry" "123" "fig" "hi"
```

```
Filter(Negate(\(w) grepl("[0-9]", w)), words)
```

```
#> [1] "apple" "Banana" "cherry" "Date" "fig"  
#> [6] "GRAPE" "hi"
```

```
Find(\(w) nchar(w) > 5, words)
```

```
#> [1] "Banana"
```

```
Position(\(w) nchar(w) > 5, words)
```

```
#> [1] 2
```

```
words |>  
  Filter(f = \(w) w == tolower(w)) |>  
  Filter(f = \(w) nchar(w) > 3) |>  
  sort()
```

```
#> [1] "apple" "cherry"
```

Lists

What Is a List?

- A **generic vector**: elements can be of **different types and lengths**.

```
vn <- c(1, 1.2, 2.3, 3.4, 4.5)
vb <- c(TRUE, TRUE, FALSE)
vc <- c('limestone', 'marl', 'oolite', 'CaCO3')
```

```
data.frame(vn, vb, vc)  # ERROR: different lengths
```

```
#> Error in data.frame(vn, vb, vc): arguments imply differing number of rows: 5, 3, 4
```


Creating Lists

```
# unnamed list
l <- list(3.14, 'Moe', c(1, 3, 5, 2), mean)
str(l)
```

```
#> List of 4
#> $ : num 3.14
#> $ : chr "Moe"
#> ... [output truncated]
```

```
# named list + incremental construction
l <- list(max = 5, right = 0.6, mean = 20)
l <- list(); l$left <- 0.1; l$right <- 0.52; l$mid <- 0.33
l
```

```
#> $left
#> [1] 0.1
#>
#> ... [output truncated]
```

[[vs [— The Most Important Distinction

```
l <- list(a = 1:3, b = 'hello',  
          c = TRUE)
```

```
l[[1]]      # the element itself
```

```
#> [1] 1 2 3
```

```
l[1]        # a sub-list
```

```
#> $a
```

```
#> [1] 1 2 3
```

```
class(l[[1]]); class(l[1])
```

```
#> [1] "integer"
```

```
#> [1] "list"
```

```
l$a      # same as l[['a']]
```

```
#> [1] 1 2 3
```

```
l$b
```

```
#> [1] "hello"
```

Analogy: `[[` reaches into the box and pulls out the item; `[` selects boxes (still wrapped).

Modifying, Removing, Concatenating

```
l <- list(x = 1, y = 2, z = 3)
l$x <- 100; l[['y']] <- 200; l$w <- 'new'    # modify / add
l[c('x', 'w')] <- NULL; l                  # remove
```

```
#> $y
#> [1] 200
#>
#> ... [output truncated]
```

```
l1 <- list(a = 1, b = 2); l2 <- list(c = 3, d = 4)
c(l1, l2)    # concatenate lists
```

```
#> $a
#> [1] 1
#>
#> ... [output truncated]
```

Modifying, Removing, Concatenating Cont'd

```
# nested lists
nested <- list(student = list(name = 'Alice', age = 20),
               scores  = list(math = 95, english = 88))
nested$student$name; nested[['scores']][['math']]

#> [1] "Alice"
#> [1] 95
```

unlist() — Flatten a List

```
iq <- list(100, 120, 140, 112, 105)
mean(unlist(iq))      # must unlist before mean()
```

```
#> [1] 115.4
```

```
l <- list(a = 1:3, b = 4:5)
unlist(l)              # names preserved with prefix
```

```
#> a1 a2 a3 b1 b2
```

```
#> 1  2  3  4  5
```

```
unlist(l, use.names = FALSE)  # drop names
```

```
#> [1] 1 2 3 4 5
```

split() — Split a Vector into a List

```
(mtl <- split(mtcars$mpg, mtcars$cyl))
```

```
#> $`4`  
#> [1] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4  
#> [11] 21.4  
#> ... [output truncated]
```

```
mtl$`6`; mtl[[2]] # access by name or position
```

```
#> [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7  
#> [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
```

- `tapply(x, f, FUN)` is equivalent to `sapply(split(x, f), FUN)`

A Data Frame Is a List

```
df <- data.frame(x = 1:3, y = c('a', 'b', 'c'))  
is.list(df); typeof(df)
```

```
#> [1] TRUE
```

```
#> [1] "list"
```

```
df$x; df[['x']]      # extracts column vector (like [])
```

```
#> [1] 1 2 3
```

```
#> [1] 1 2 3
```

```
df['x']              # returns 1-column data.frame (like [])
```

```
#>    x
```

```
#> 1 1
```

```
#> 2 2
```

```
#> ... [output truncated]
```

```
# split() a data frame -> list of data frames
```

```
split(tg, tg$supp)  # returns list of data frames
```

Conversions Between Data Structures

From	To	How	Note
vector	list	<code>as.list(v)</code>	Don't use <code>list(v)</code> !
vector	matrix	<code>matrix(v, n, m)</code>	
vector	data.frame	<code>as.data.frame(v)</code>	
list	vector	<code>unlist(l)</code>	Not <code>as.vector</code>
list	data.frame	<code>as.data.frame(l)</code>	Equal-length only
matrix	vector	<code>as.vector(m)</code>	
matrix	data.frame	<code>as.data.frame(m)</code>	
data.frame	list	<code>as.list(df)</code>	Removes class attr
data.frame	matrix	<code>as.matrix(df)</code>	Coercion!

- Create `l <- list(name="Alice", scores=c(85,92,78), passed=TRUE)`
- Access scores with `$`, `[[ ]]`, and `[ ]`. Show the different classes
- Add element `grade = "B"`. Remove `passed`
- What is `length(l)` before and after?

```
l <- list(name="Alice", scores=c(85,92,78), passed=TRUE)
l$scores; l[["scores"]]; l["scores"]
```

```
#> [1] 85 92 78
```

```
#> [1] 85 92 78
```

```
#> $scores
```

```
#> [1] 85 92 78
```

```
class(l$scores); class(l["scores"])           # vector vs list
```

```
#> [1] "numeric"
```

```
#> [1] "list"
```

```
l$grade <- "B"; l$passed <- NULL
length(l)  # now 3
```

```
#> [1] 3
```

- `l <- list(a=1:3, b=4:5)`. What does `unlist(l)` produce? Explain the names
- `unlist(l, use.names=FALSE)`
- `l2 <- list(x="hello", y=1:3)`. What type is `unlist(l2)`? Why?
- Mean of a list of numbers: `list(10, 20, 30)`

```
l <- list(a=1:3, b=4:5)
unlist(l)                # named: a1 a2 a3 b1 b2
```

```
#> a1 a2 a3 b1 b2
#>  1  2  3  4  5
```

```
unlist(l, use.names=FALSE)
```

```
#> [1] 1 2 3 4 5
```

```
unlist(list(x="hello", y=1:3)) # all character
```

```
#>      x      y1      y2      y3
#> "hello" "1"    "2"    "3"
```

```
mean(unlist(list(10, 20, 30))) # 20
```

```
#> [1] 20
```

- Create `student <- list(info=list(name="Bob", age=20), grades=list(math=90, eng=85))`
- Access Bob's math grade
- Add `grades$sci = 92`
- Use `str()` to visualize the structure

```
student <- list(info=list(name="Bob", age=20),  
               grades=list(math=90, eng=85))  
student$grades$math # 90
```

```
#> [1] 90
```

```
student$grades$sci <- 92  
str(student)
```

```
#> List of 2  
#> $ info :List of 2  
#> ..$ name: chr "Bob"  
#> ... [output truncated]
```

- `split(mtcars$mpg, mtcars$cyl)` — what type?
- Compute mean per group using `sapply`
- Same using `tapply` in one line
- `split(1:10, rep(c("odd", "even"), 5))`

```
sp <- split(mtcars$mpg, mtcars$cyl); class(sp)    # list
```

```
#> [1] "list"
```

```
sapply(sp, mean)
```

```
#>      4      6      8  
#> 26.66364 19.74286 15.10000
```

```
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
#>      4      6      8  
#> 26.66364 19.74286 15.10000
```

```
split(1:10, rep(c("odd","even"), 5))
```

```
#> $even  
#> [1]  2  4  6  8 10  
#>  
#> ... [output truncated]
```


- Concatenate `list(a=1)` and `list(b=2)` with `c()`
- What happens with `c(list(a=1), 5)`?
- Build a list incrementally in a loop: for `i=1:5`, append `i^2`

```
c(list(a=1), list(b=2))
```

```
#> $a  
#> [1] 1  
#>  
#> ... [output truncated]
```

```
c(list(a=1), 5)
```

5 becomes element

```
#> $a  
#> [1] 1  
#>  
#> ... [output truncated]
```

```
l <- list()
```

```
for (i in 1:5) l[[i]] <- i^2; l
```

```
#> [[1]]  
#> [1] 1  
#>  
#> ... [output truncated]
```

- Vector to list: `as.list(1:3)` vs `list(1:3)`. Explain difference
- List to data frame: `as.data.frame(list(a=1:3, b=4:6))`
- Data frame to matrix: what happens with `as.matrix(data.frame(x=1:3, y=c("a","b","c")))`?
- List to vector: why use `unlist` not `as.vector`?

Sol 54

```
as.list(1:3)      # 3 elements
```

```
#> [[1]]  
#> [1] 1  
#>  
#> ... [output truncated]
```

```
list(1:3)         # 1 element (the vector)
```

```
#> [[1]]  
#> [1] 1 2 3
```

```
as.data.frame(list(a=1:3, b=4:6))
```

```
#>   a b  
#> 1 1 4  
#> 2 2 5  
#> ... [output truncated]
```

```
as.matrix(data.frame(x=1:3, y=c("a","b","c"))) # all char!
```

```
#>      x  y  
#> [1,] "1" "a"  
#> [2,] "2" "b"
```

```
#> ... [output truncated]
```

- Create a lookup: `grade_points <- list(A=4.0, B=3.0, C=2.0, D=1.0, F=0)`
- Look up GPA for grades `c("A","B","F","A","C")` using `sapply + [[`
- Compute mean GPA

```
grade_points <- list(A=4.0, B=3.0, C=2.0, D=1.0, F=0)
g <- c("A","B","F","A","C")
gpa <- sapply(g, function(x) grade_points[[x]])
gpa
```

```
#> A B F A C
```

```
#> 4 3 0 4 2
```

```
mean(gpa)
```

```
# 2.6
```

```
#> [1] 2.6
```

```
l <- list(a=1:10, b=rnorm(20), c=c(TRUE,FALSE,TRUE,TRUE)).
```

- Compute mean of each element with lapply and sapply
- Compute length of each
- Apply custom function: range (max - min)

```
set.seed(1)
l <- list(a=1:10, b=rnorm(20), c=c(TRUE,FALSE,TRUE,TRUE))
lapply(l, mean); sapply(l, mean)
```

```
#> $a
#> [1] 5.5
#>
#> ... [output truncated]
#>           a           b           c
#> 5.5000000 0.1905239 0.7500000
```

```
sapply(l, length)
```

```
#>  a  b  c
#> 10 20 4
```

```
sapply(l, function(x) diff(range(x)))
```

```
#>           a           b           c
#> 9.0000000 3.809981 1.0000000
```



```
l <- list(1:3, "hello", 4:6, TRUE, NULL, c(1.5, 2.5)).
```

- Keep only numeric elements
- Find first character element
- Position of first character element
- Filter integers from 1:30 that are divisible by both 3 and 7

```
l <- list(1:3, "hello", 4:6, TRUE, NULL, c(1.5, 2.5))  
Filter(is.numeric, l)
```

```
#> [[1]]  
#> [1] 1 2 3  
#>  
#> ... [output truncated]
```

```
Find(is.character, l) # "hello"
```

```
#> [1] "hello"
```

```
Position(is.character, l) # 2
```

```
#> [1] 2
```

```
Filter(\(x) x%%3==0 & x%%7==0, 1:30) # 21
```

```
#> [1] 21
```

- Compute $1+2+3+4+5$ using `Reduce("+", 1:5)`. Show accumulated steps
- Find intersection of 3 vectors via `Reduce(intersect, ...)`
- Merge 3 data frames pairwise using `Reduce`

```
Reduce("+", 1:5) # 15
```

```
#> [1] 15
```

```
Reduce("+", 1:5, accumulate=TRUE) # steps
```

```
#> [1] 1 3 6 10 15
```

```
vecs <- list(c(1,2,3,4), c(2,3,4,5), c(3,4,5,6))  
Reduce(intersect, vecs) # 3 4
```

```
#> [1] 3 4
```

```
dfs <- list(data.frame(id=1:3, a=10:12),  
            data.frame(id=2:4, b=20:22))  
Reduce(\(x,y) merge(x,y,by="id",all=TRUE), dfs)
```

```
#>   id  a  b  
#> 1   1 10 NA  
#> 2   2 11 20  
#> ... [output truncated]
```

- Given `l <- list(x=1, y=2, z=3)`, remove `x` and `z` in one operation
- Replace all numeric elements with their squares using `lapply`
- Deep copy a nested list and modify without affecting original

```
l <- list(x=1, y=2, z=3)
l[c("x","z")] <- NULL; l # only y
```

```
#> $y
#> [1] 2
```

```
l2 <- list(a=3, b="hi", c=4)
lapply(l2, function(x) if(is.numeric(x)) x^2 else x)
```

```
#> $a
#> [1] 9
#>
#> ... [output truncated]
```

```
orig <- list(inner=list(val=10))
copy <- orig; copy$inner$val <- 99
orig$inner$val # still 10 (R copies on modify)
```

```
#> [1] 10
```

```
l <- list(a=1:3, b=list(c=4:6, d=list(e=7:9))).
```

- Apply sum to all leaf elements using `rapply(l, sum)`
- Use `rapply(l, sum, how="unlist")` vs `how="list"`
- Double all values with `rapply(l, function(x) x*2, how="replace")`

Sol 60

```
l <- list(a=1:3, b=list(c=4:6, d=list(e=7:9)))  
rapply(l, sum)
```

```
#>      a    b.c b.d.e  
#>      6     15     24
```

```
rapply(l, sum, how="unlist")           # named vector
```

```
#>      a    b.c b.d.e  
#>      6     15     24
```

```
rapply(l, sum, how="list")            # list structure
```

```
#> $a  
#> [1] 6  
#>  
#> ... [output truncated]
```

```
str(rapply(l, \(x) x*2, how="replace")) # doubled
```

```
#> List of 2  
#> $ a: num [1:3] 2 4 6  
#> $ b:List of 2  
#> ... [output truncated]
```


Matrices

Creating Matrices

```
v <- 1:6; matrix(v, nrow = 2, ncol = 3)
```

```
#>      [,1] [,2] [,3]  
#> [1,]    1    3    5  
#> [2,]    2    4    6  
#> ... [output truncated]
```

```
# named rows and columns
```

```
m <- matrix(c(1, .56, .38, .56, 1, .44, .38, .44, 1), 3, 3)  
colnames(m) <- rownames(m) <- c('AAPL', 'MSFT', 'GOOG')  
m; m['MSFT', 'GOOG']
```

```
#>      AAPL MSFT GOOG  
#> AAPL 1.00 0.56 0.38  
#> MSFT 0.56 1.00 0.44  
#> ... [output truncated]  
#> [1] 0.44
```

Matrix Operations

```
A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2)
```

```
A * B           # element-wise
```

```
#>      [,1] [,2]  
#> [1,]    5  21  
#> [2,]   12  32  
#> ... [output truncated]
```

```
A %*% B         # matrix multiplication
```

```
#>      [,1] [,2]  
#> [1,]   23  31  
#> [2,]   34  46  
#> ... [output truncated]
```

```
t(A); det(A); solve(A)  # transpose, determinant, inverse
```

```
#>      [,1] [,2]  
#> [1,]    1    2  
#> [2,]    3    4  
#> ... [output truncated]  
  
#> [1] -2
```

```
#>      [,1] [,2]
```

Matrix Operations Cont'd

```
# row/column selection: drop=FALSE preserves matrix class  
A[1, ]; A[1, , drop = FALSE]
```

```
#> [1] 1 3
```

```
#>      [,1] [,2]
```

```
#> [1,]    1    3
```

Matrix Functions Summary

```
nrow(m); ncol(m); dim(m)
diag(m)                # diagonal elements
diag(3)                # 3x3 identity matrix
crossprod(A, B)        # t(A) %*% B (faster)
tcrossprod(A, B)       # A %*% t(B)
outer(1:3, 1:3, '+' )  # outer product
```

- Create 3×4 matrix filled column-wise from 1:12
- Create same matrix filled row-wise
- Name rows `r1,r2,r3` and columns `c1,...,c4`. Access `m["r2","c3"]`
- Create a 3×3 identity matrix with `diag`

Sol 61

```
matrix(1:12, nrow=3)
```

```
#>      [,1] [,2] [,3] [,4]  
#> [1,]     1     4     7    10  
#> [2,]     2     5     8    11  
#> ... [output truncated]
```

```
matrix(1:12, nrow=3, byrow=TRUE)
```

```
#>      [,1] [,2] [,3] [,4]  
#> [1,]     1     2     3     4  
#> [2,]     5     6     7     8  
#> ... [output truncated]
```

```
m <- matrix(1:12, 3, 4)
```

```
rownames(m) <- paste0("r",1:3); colnames(m) <- paste0("c",1:4)  
m["r2","c3"]                                     # 8
```

```
#> [1] 8
```

```
diag(3)
```

```
#>      [,1] [,2] [,3]  
#> [1,]     1     0     0
```

```
A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2).
```

- Element-wise `*` vs matrix `%*%`
- Compute `t(A)`, `det(A)`, `solve(A)`
- Verify `A %*% solve(A)` is the identity
- `crossprod(A,B)` vs `t(A) %*% B`

Sol 62

```
A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2)
```

```
A * B; A %*% B
```

```
#>      [,1] [,2]  
#> [1,]    5  21  
#> ... [output truncated]  
#>      [,1] [,2]  
#> [1,]   23  31  
#> ... [output truncated]
```

```
t(A); det(A); solve(A)
```

```
#>      [,1] [,2]  
#> [1,]    1    2  
#> ... [output truncated]  
#> [1] -2  
#>      [,1] [,2]  
#> [1,]   -2  1.5  
#> ... [output truncated]
```

```
round(A %*% solve(A))
```

```
# identity
```

```
m <- matrix(1:12, nrow=3).
```

- Row sums and column sums via `apply`
- Per-column range
- Standardize each column (subtract mean, divide by sd)
- Compare `apply(m,2,sum)` speed with `colSums(m)`

Sol 63

```
m <- matrix(1:12, nrow=3)
apply(m, 1, sum); apply(m, 2, sum)
```

```
#> [1] 22 26 30
```

```
#> [1] 6 15 24 33
```

```
apply(m, 2, range)
```

```
#>      [,1] [,2] [,3] [,4]
```

```
#> [1,]     1     4     7    10
```

```
#> [2,]     3     6     9    12
```

```
#> ... [output truncated]
```

```
apply(m, 2, \(x) (x-mean(x))/sd(x))
```

```
#>      [,1] [,2] [,3] [,4]
```

```
#> [1,]    -1    -1    -1    -1
```

```
#> [2,]     0     0     0     0
```

```
#> ... [output truncated]
```

```
all.equal(apply(m,2,sum), colSums(m))           # TRUE
```

```
#> [1] TRUE
```

- Multiplication table $1..5 \times 1..5$ using `outer`
- Compute $f(x, y) = x^2 + y^2$ for `x=1:4`, `y=1:3`
- All pairwise paste of `c("a", "b")` and `c("1", "2", "3")`

```
outer(1:5, 1:5, "*")
```

```
#>      [,1] [,2] [,3] [,4] [,5]
#> [1,]     1     2     3     4     5
#> [2,]     2     4     6     8    10
#> ... [output truncated]
```

```
outer(1:4, 1:3, \(x,y) x^2 + y^2)
```

```
#>      [,1] [,2] [,3]
#> [1,]     2     5    10
#> [2,]     5     8    13
#> ... [output truncated]
```

```
outer(c("a","b"), c("1","2","3"), paste0)
```

```
#>      [,1] [,2] [,3]
#> [1,] "a1" "a2" "a3"
#> [2,] "b1" "b2" "b3"
#> ... [output truncated]
```

```
m <- matrix(1:12, 3, 4).
```

- Extract row 2 as a vector vs as a 1-row matrix (drop)
- Extract column 3 both ways
- Select submatrix: rows 1-2, columns 2-3
- Replace all elements > 8 with NA

Sol 65

```
m <- matrix(1:12, 3, 4)
m[2,]; m[2,,drop=FALSE]
```

```
#> [1]  2  5  8 11
#>      [,1] [,2] [,3] [,4]
#> [1,]    2    5    8   11
```

```
m[,3]; m[,3,drop=FALSE]
```

```
#> [1] 7 8 9
#>      [,1]
#> [1,]    7
#> [2,]    8
#> ... [output truncated]
```

```
m[1:2, 2:3]
```

```
#>      [,1] [,2]
#> [1,]    4    7
#> [2,]    5    8
#> ... [output truncated]
```

```
m[m > 8] <- NA; m
```

Solve $2x + 3y = 7$, $4x - y = 1$.

- Set up as $A \cdot x = b$ and solve with `solve(A, b)`
- Verify by multiplying back
- Compute eigenvalues of A


```
A <- matrix(c(2,4,3,-1), 2, 2); b <- c(7,1)
x <- solve(A, b); x
```

```
#> [1] 0.7142857 1.8571429
```

```
A %*% x # should be c(7,1)
```

```
#>      [,1]
#> [1,]    7
#> [2,]    1
#> ... [output truncated]
```

```
eigen(A)$values
```

```
#> [1] 4.274917 -3.274917
```

Programming

Defining Functions

```
function_name <- function(arg1, arg2 = default_value, ...) {  
  # code  
  return(some_object)  
}
```

```
# functions can read global variables  
y_squared <- function() { return(y^2) }  
y <- 2; y_squared()
```

```
#> [1] 4
```

```
# but local assignment doesn't change globals  
add_to_y <- function(n) { y <- y + n }  
y <- 1; add_to_y(2); y      # y is still 1
```

```
#> [1] 1
```

```
# <<- modifies globals (use sparingly!)  
add_to_y_global <- function(n) { y <<- y + n }  
y <- 1; add_to_y_global(2); y  # y is now 3
```

```
#> [1] 3
```

return() and Implicit Return

```
avg <- function(x) { return(sum(x) / length(x)) }
```

```
avg(c(2, 3, 6))
```

```
#> [1] 3.666667
```

```
avg(c(TRUE, FALSE, TRUE))    # TRUE=1, FALSE=0
```

```
#> [1] 0.666667
```

```
# implicit return: the last evaluated expression is returned
```

```
avg_bad <- function(x) { r <- sum(x) / length(x) }
```

```
avg_ok  <- function(x) { sum(x) / length(x) }
```

```
avg_bad(1:10)    # silent! (assignment returns invisibly)
```

```
avg_ok(1:10)     # returns result
```

```
#> [1] 5.5
```

Default Arguments and Named Arguments

```
power_n <- function(x, n = 2) { return(x^n) }  
power_n(3)                # default n = 2
```

```
#> [1] 9
```

```
power_n(3, 3)              # supplied n = 3
```

```
#> [1] 27
```

```
power_n(x = 5, n = 2)     # named arguments
```

```
#> [1] 25
```

```
power_n(n = 2, x = 5)     # order doesn't matter with names
```

```
#> [1] 25
```

```
apply_to_first2 <- function(x, func, ...) {  
  func(x[1:2], ...)  
}  
x <- c(NA, 2, 3)  
apply_to_first2(x, sum, na.rm = TRUE)  
  
#> [1] 2
```

Anonymous Functions and \(\x\) Shorthand

```
sapply(1:5, function(x) x^2 + 1)
```

```
#> [1]  2  5 10 17 26
```

```
sapply(1:5, \(x) x^2 + 1)    # R 4.1+ shorthand
```

```
#> [1]  2  5 10 17 26
```

```
# useful in apply family
```

```
l <- list(a = 1:5, b = 6:10)
```

```
lapply(l, \(v) c(mean = mean(v), sd = sd(v)))
```

```
#> $a
```

```
#>      mean      sd
```

```
#> 3.000000 1.581139
```

```
#> ... [output truncated]
```

Lexical Scoping Rule

When R encounters a free variable inside a function, it searches the **environment where the function was defined** (not where it was called):

```
x <- 10
f <- function() x          # f is born in global env
g <- function() {
  x <- 999
  f()                       # calls f, but f ignores this x
}
g()                         # 10, not 999
```

```
#> [1] 10
```

- **Lexical** (static): determined by where the code is **written**
- R, Python, JavaScript all use lexical scoping

What Is a Closure?

Closure = a function **bundled together** with the environment in which it was created.

When function A returns function B, B retains a **live reference** to A's local environment — even after A has finished executing.

```
power_factory <- function(n) {  
  function(x) x^n          # captures n  
}  
square <- power_factory(2)  
cube   <- power_factory(3)  
square(5); cube(5)
```

```
#> [1] 25
```

```
#> [1] 125
```

```
# each call creates a new, independent environment  
environment(square)$n; environment(cube)$n
```

```
#> [1] 2
```

```
#> [1] 3
```

R does not distinguish “closure” as a special type. Every function stores formals, body, and environment:

```
f <- function(x) x + 1
typeof(f)           # "closure"
```

```
#> [1] "closure"
```

```
formals(f); body(f); environment(f)
```

```
#> $x
```

```
#> x + 1
```

```
#> <environment: R_GlobalEnv>
```

<- vs <<- and Mutable State

- <- assigns **only** in the current (local) environment
- <<- walks **up** the enclosing environment chain to modify an existing variable

```
make_counter <- function() {  
  n <- 0  
  list(  
    increment = function() { n <<- n + 1; n },  
    get       = function() n,  
    reset     = function() { n <<- 0 }  
  )  
}
```

<- vs <<- and Mutable State Cont'd

```
make_counter <- function() {  
  n <- 0  
  list(  
    increment = function() { n <<- n + 1; n },  
    get       = function() n,  
    reset     = function() { n <<- 0 }  
  )  
}  
ctr <- make_counter()  
ctr$increment(); ctr$increment(); ctr$get()
```

```
#> [1] 1
```

```
#> [1] 2
```

```
#> [1] 2
```

```
ctr$reset(); ctr$get()
```

```
#> [1] 0
```

Independent Instances and local()

```
ctr1 <- make_counter(); ctr2 <- make_counter()
ctr1$increment(); ctr1$increment(); ctr2$increment()
```

```
#> [1] 1
```

```
#> [1] 2
```

```
#> [1] 1
```

```
cat("ctr1:", ctr1$get(), " ctr2:", ctr2$get(), "\n")
```

```
#> ctr1: 2  ctr2: 1
```

```
# local() creates a single-instance closure
```

```
counter <- local({
  n <- 0
  list(
    increment = function() { n <-< n + 1; n },
    get       = function() n,
    reset     = function() { n <-< 0 })
})
counter$increment(); counter$increment(); counter$get()
```

```
#> [1] 1
```

Classic Pitfall: Loop Capture

```
funs <- list()
for (i in 1:5) funs[[i]] <- function() i
sapply(funs, \(f) f()) # all 5!
```

```
#> [1] 5 5 5 5 5
```

All five functions share the **same** environment; after the loop, `i = 5`.

```
# Fix 1: force() defeats lazy evaluation
make_fun <- function(val) { force(val); function() val }
funs <- list()
for (i in 1:5) funs[[i]] <- make_fun(i)
sapply(funs, \(f) f())
```

```
#> [1] 1 2 3 4 5
```

```
# Fix 2 (recommended): lapply creates separate environments
funs <- lapply(1:5, \(i) function() i)
sapply(funs, \(f) f())
```

```
#> [1] 1 2 3 4 5
```

Base R Closures: ecdf, approxfun, Negate

```
x <- c(1, 3, 3, 5, 7, 9); Fn <- ecdf(x)
Fn(4); Fn(7)          # P(X<=4)=3/6; P(X<=7)=5/6
```

```
#> [1] 0.5
```

```
#> [1] 0.8333333
```

```
typeof(Fn)            # "closure"
```

```
#> [1] "closure"
```

```
xp <- 1:5; yp <- c(2, 5, 3, 8, 6)
linear_interp <- approxfun(xp, yp)
linear_interp(2.5)    # returns a closure over the knot data
```

```
#> [1] 4
```

```
is_even <- function(x) x %% 2 == 0
is_odd  <- Negate(is_even)          # closure combinator
Filter(is_odd, 1:10)
```

```
#> [1] 1 3 5 7 9
```

if / else

```
reciprocal <- function(x) {  
  if (is.na(x)) {  
    cat("Error! Division by NA.\n")  
  } else if (is.infinite(x)) {  
    cat("Error! Division by infinity.\n")  
  } else if (x == 0) {  
    cat("Error! Division by zero.\n")  
  } else { 1 / x }  
}  
reciprocal(2); reciprocal(0); reciprocal(NA)
```

#> [1] 0.5

#> Error! Division by zero.

#> Error! Division by NA.

ifelse() — Vectorized Conditional

```
x <- c(-1, 2, -100, 20, 50)
ifelse(x > 0, 'positive', 'negative')
```

```
#> [1] "negative" "positive" "negative" "positive"
#> [5] "positive"
```

```
# nested ifelse
v <- c(20, 18, 45, 33, 25, 9, 17, 55, 69, 81, 44, 32)
ifelse(v >= 20 & v < 40, 'young adult',
ifelse(v >= 40 & v < 60, 'middle age', 'other'))
```

```
#> [1] "young adult" "other"      "middle age"
#> [4] "young adult" "young adult" "other"
#> [7] "other"      "middle age"  "other"
#> ... [output truncated]
```

switch()

```
day_type <- function(day) {  
  switch(day,  
    Monday=, Tuesday=, Wednesday=,  
    Thursday=, Friday= "Weekday",  
    Saturday=, Sunday= "Weekend",  
    "Unknown")  
}  
day_type("Monday"); day_type("Sunday"); day_type("Holiday")
```

```
#> [1] "Weekday"
```

```
#> [1] "Weekend"
```

```
#> [1] "Unknown"
```

&& vs & (Short-Circuit Evaluation)

```
# first, ensure 'aa' does not exist from earlier chunks  
if (exists("aa")) rm(aa)
```

```
# && is short-circuited: stops at first FALSE  
if (exists("aa") && aa > 0) {  
  cat("aa exists and is positive.\n")  
} else {  
  cat("aa doesn't exist or is negative.\n")  
}
```

```
#> aa doesn't exist or is negative.
```

&& vs & (Short-Circuit Evaluation) Cont'd

```
# & evaluates BOTH sides -- error because aa doesn't exist!  
if (exists("aa") & aa > 0) {  
  cat("aa exists and is positive.\n")  
} else {  
  cat("aa doesn't exist or is negative.\n")  
}
```

#> Error: object 'aa' not found

- **Rule:** Use && / || in if (scalar); use & / | for vectors.

if (FALSE) { } — Code Block Toggle

```
# wrap example/test code that shouldn't run
if (FALSE) {
  # these lines are "commented out" but remain valid R code
  book(date = "2026/04/02", headless = FALSE)
  cancel(order_id = "01901879")
}
```

- Unlike # comments, the code inside is still checked by R's parser

for Loop

```
# sum 1 to 10000
ans <- 0
for (i in 1:10000) { ans <- ans + i }
ans
```

```
#> [1] 50005000
```

```
# sum of min(2^k, k^4) for k = 1..100
ans <- 0
for (k in 1:100) { ans <- ans + min(2^k, k^4) }
ans
```

```
#> [1] 2050220551
```

```
# vectorized equivalent (much faster)
v <- 1:100; sum(pmin(2^v, v^4))
```

```
#> [1] 2050220551
```

while, repeat, break, next

```
# while: find smallest n such that sum(1:n) > 1000
n <- 0; s <- 0
while (s <= 1000) { n <- n + 1; s <- s + n }
cat("n =", n, ", sum =", s, "\n")
```

```
#> n = 45 , sum = 1035
```

```
# repeat + break
n <- 1
repeat { if (n^2 > 100) break; n <- n + 1 }
cat("First n with n^2 > 100:", n, "\n")
```

```
#> First n with n^2 > 100: 11
```

```
# next: skip iterations
for (i in 1:10) {
  if (i %% 3 == 0) next      # skip multiples of 3
  cat(i, ' ')
}
```

```
#> 1 2 4 5 7 8 10
```

Vectorization vs Loops: Speed Comparison

```
my_sum <- function(x) {  
  ans <- 0  
  for (i in seq_along(x)) { ans <- ans + x[i] }  
  ans  
}  
v <- 1:1000000; system.time(my_sum(v)); system.time(sum(v))
```

```
#>      user  system elapsed  
#>  0.030    0.000    0.029  
  
#>      user  system elapsed  
#>         0         0         0
```


Vectorization vs Loops: Speed Comparison Cont'd

```
# pre-allocate vs grow-in-loop vs vectorized
```

```
n <- 1e5
```

```
system.time({v <- c(); for (i in 1:n) v <- c(v, i^2)})
```

```
#>      user  system elapsed
```

```
#>  6.136    0.168    6.303
```

```
system.time({v <- numeric(n); for (i in 1:n) v[i] <- i^2})
```

```
#>      user  system elapsed
```

```
#>  0.005    0.000    0.004
```

```
system.time({v <- (1:n)^2})
```

```
#>      user  system elapsed
```

```
#>  0.001    0.000    0.000
```

Native Pipe |> (R 4.1+)

```
# without pipe: nested calls are hard to read  
round(mean(abs(rnorm(100))), 3)
```

```
#> [1] 0.698
```

```
# with pipe: read left to right  
set.seed(1)  
rnorm(100) |> abs() |> mean() |> round(3)
```

```
#> [1] 0.717
```

Native Pipe |> (R 4.1+) Cont'd

```
# pipe with data frame operations
```

```
mtcars |> subset(cyl == 6) |> head(3)
```

```
#>           mpg cyl disp  hp drat   wt  qsec vs  
#> Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0  
#> Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0  
#> ... [output truncated]
```

```
# use _ as placeholder for non-first argument (R 4.2+)
```

```
mtcars |> lm(mpg ~ wt, data = _) |> summary()
```

- Write `power(x, n=2)` that computes x^n . Test with 1 and 2 arguments
- Write `greet(name, greeting="Hello")` that returns "Hello, Alice!"
- What happens if you call `power(n=3)` without `x`?

```
power <- function(x, n=2) x^n  
power(3); power(3, 3); power(x=5, n=2)
```

```
#> [1] 9
```

```
#> [1] 27
```

```
#> [1] 25
```

```
greet <- function(name, greeting="Hello")  
  paste0(greeting, ", ", name, "!")  
greet("Alice"); greet("Bob", "Hi")
```

```
#> [1] "Hello, Alice!"
```

```
#> [1] "Hi, Bob!"
```

```
power(n=3) # error: x is missing
```

```
#> Error in power(n = 3): argument "x" is missing, with no default
```

- Write `apply_fn(x, fn)` that applies `fn` to `x`. Test with `sqrt`, `log`, `abs`
- Extend to `apply_fn(x, fn, ...)` supporting extra arguments. Test: `apply_fn(c(1,NA,3), mean, na.rm=TRUE)`

```
apply_fn <- function(x, fn, ...) fn(x, ...)  
apply_fn(c(4,9,16), sqrt)
```

```
#> [1] 2 3 4
```

```
apply_fn(c(1,NA,3), mean, na.rm=TRUE) # 2
```

```
#> [1] 2
```

- Write two versions of `average`: one with explicit `return()`, one without
- Write `bad_avg` that assigns to a local variable as last line. Show it returns silently
- Capture the invisible result with `x <- bad_avg(1:10)`


```
avg_explicit <- function(x) { return(sum(x)/length(x)) }  
avg_implicit <- function(x) { sum(x)/length(x) }  
avg_explicit(1:10); avg_implicit(1:10)           # same
```

```
#> [1] 5.5
```

```
#> [1] 5.5
```

```
bad_avg <- function(x) { result <- sum(x)/length(x) }  
bad_avg(1:10) # no output!  
x <- bad_avg(1:10); x # but value is captured
```

```
#> [1] 5.5
```

- Use `sapply(1:5, function(x) x^2 + 1)`
- Rewrite with `\(x)` shorthand
- Use anonymous function in `Filter` to keep even numbers from `1:20`
- Nested: `sapply(1:3, \(i) sapply(1:3, \(j) i*j))`

```
sapply(1:5, function(x) x^2+1)
```

```
#> [1]  2  5 10 17 26
```

```
sapply(1:5, \(x) x^2+1)
```

```
#> [1]  2  5 10 17 26
```

```
Filter(\(x) x%%2==0, 1:20)
```

```
#> [1]  2  4  6  8 10 12 14 16 18 20
```

```
sapply(1:3, \(i) sapply(1:3, \(j) i*j)) # 3x3 matrix
```

```
#>      [,1] [,2] [,3]
```

```
#> [1,]    1    2    3
```

```
#> [2,]    2    4    6
```

```
#> ... [output truncated]
```

- Write `make_adder(n)` that returns a function adding `n`. Test `add5 <- make_adder(5); add5(3)`
- Write `make_counter()` returning a list with `increment`, `get`, `reset` functions using `<<-`
- Verify the counter works across multiple calls

Sol 71

```
make_adder <- function(n) function(x) x + n
add5 <- make_adder(5); add5(3) # 8
```

```
#> [1] 8
```

```
make_counter <- function() {
  n <- 0
  list(inc = function() {n <- n+1; n},
       get = function() n,
       reset = function() {n <- 0})
}
ctr <- make_counter()
ctr$inc(); ctr$inc(); ctr$get() # 2
```

```
#> [1] 1
```

```
#> [1] 2
```

```
#> [1] 2
```

```
ctr$reset(); ctr$get() # 0
```

```
#> [1] 0
```

- Define `.helper <- function(x) x+1` and `public <- function(x) .helper(x)*2`
- Show `.helper` doesn't appear in `ls()` but does in `ls(all.names=TRUE)`
- Name 3 built-in R objects starting with `.`

```
e <- new.env()
e$.helper <- function(x) x+1
e$public <- function(x) e$.helper(x)*2
e$public(5) # 12
```

```
#> [1] 12
```

```
ls(e); ls(e, all.names=TRUE)
```

```
#> [1] "public"
```

```
#> [1] ".helper" "public"
```

```
# .Machine, .Platform, .GlobalEnv
```

- Wrap code in `if (FALSE) \{ ... \}` and show it doesn't execute
- Change to `if (TRUE)` and show it runs
- Why is this better than commenting with `#` for multi-line blocks?


```
if (FALSE) {  
  cat("This won't print\n")  
  x <- 999  
}  
exists("x") # from previous chunks, may be TRUE
```

```
#> [1] TRUE
```

```
if (TRUE) { cat("This prints\n") }
```

```
#> This prints
```

```
# Unlike #, the code inside is still parsed by R,  
# so syntax errors are caught immediately.
```

- Write `outer_fn(x)` that defines inner helper `validate(v)` checking `is.numeric(v) && length(v) > 0`, then uses it
- Show that `validate` is not accessible outside `outer_fn`
- Why is this pattern useful?

```
outer_fn <- function(x) {  
  validate <- function(v) is.numeric(v) && length(v) > 0  
  if (!validate(x)) stop("invalid input")  
  mean(x)  
}  
outer_fn(1:10) # 5.5
```

```
#> [1] 5.5
```

```
outer_fn("abc") # error
```

```
#> Error in outer_fn("abc"): invalid input
```

```
# validate is local, not in global env  
# keeps global namespace clean; helper has access to  
#   outer function's variables via lexical scoping
```

Ex 75: if/else and ifelse

- Write scalar `classify(x)` returning "pos", "neg", or "zero" using if/else
- Vectorize: `classify c(-3,0,5,2,-1)` using ifelse
- Nested ifelse to map scores to grades: ≥ 90 A, ≥ 80 B, ≥ 70 C, else F

```
classify <- function(x) {  
  if (x > 0) "pos" else if (x < 0) "neg" else "zero"  
}  
classify(5); classify(-1); classify(0)
```

```
#> [1] "pos"
```

```
#> [1] "neg"
```

```
#> [1] "zero"
```

```
ifelse(c(-3,0,5,2,-1)>0, "pos",  
  ifelse(c(-3,0,5,2,-1)<0, "neg", "zero"))
```

```
#> [1] "neg" "zero" "pos" "pos" "neg"
```

```
s <- c(95,82,71,65,90)  
ifelse(s>=90,"A",ifelse(s>=80,"B",ifelse(s>=70,"C","F")))
```

```
#> [1] "A" "B" "C" "F" "A"
```

- Write `calc(op, a, b)` using `switch(op, "+"=a+b, "- "=a-b, "*" =a*b, "/" =a/b, stop("unknown"))`
- Test with all 4 operators and an invalid one
- Write `day_type(day)` classifying weekdays vs weekends

```
calc <- function(op, a, b)
  switch(op, "+"=a+b, "-"=a-b, "*"=a*b, "/"=a/b,
         stop("unknown op"))
calc("+",3,4); calc("*",5,6); calc("/",10,3)
```

```
#> [1] 7
```

```
#> [1] 30
```

```
#> [1] 3.333333
```

```
calc("^",2,3) # error
```

```
#> Error in calc("^", 2, 3): unknown op
```

```
day_type <- function(d) switch(d, Monday=,Tuesday=,
  Wednesday=,Thursday=,Friday="Weekday",
  Saturday=,Sunday="Weekend","Unknown")
day_type("Monday"); day_type("Sunday")
```

```
#> [1] "Weekday"
```

```
#> [1] "Weekend"
```

- Show `&&` short-circuits: `FALSE && stop("boom")` doesn't error
- Show `&` doesn't short-circuit: `c(FALSE,TRUE) & c(TRUE,TRUE)` is vectorized
- Write safe null-check: `if (!is.null(x) && x > 0)` and explain why `&&` is required


```
FALSE && stop("boom") # no error: short-circuit
```

```
#> [1] FALSE
```

```
c(FALSE,TRUE) & c(TRUE,TRUE) # vectorized
```

```
#> [1] FALSE TRUE
```

```
x <- NULL  
if (!is.null(x) && x > 0) cat("pos") else cat("safe")
```

```
#> safe
```

```
# && stops after !is.null(x) is FALSE; & would try x>0
```

- Classic FizzBuzz 1-20: print “Fizz” if divisible by 3, “Buzz” if by 5, “FizzBuzz” if both, else the number
- Vectorized version using `ifelse` (no loop)
- Compare with `paste0` trick

```
for (i in 1:20) {  
  if (i%%15==0) cat("FizzBuzz ")  
  else if (i%%3==0) cat("Fizz ")  
  else if (i%%5==0) cat("Buzz ")  
  else cat(i, "  
}
```

```
#> 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 17 Fizz 19 Buzz
```

```
cat("\n")
```

```
x <- 1:20  
ifelse(x%%15==0,"FizzBuzz",ifelse(x%%3==0,"Fizz",  
  ifelse(x%%5==0,"Buzz",as.character(x))))
```

```
#> [1] "1"      "2"      "Fizz"   "4"  
#> [5] "Buzz"   "Fizz"   "7"      "8"  
#> [9] "Fizz"   "Buzz"   "11"     "Fizz"  
#> ... [output truncated]
```

Implement Newton's method for $\sqrt{a}$: $x_{n+1} = \frac{1}{2}(x_n + a/x_n)$.

- Write `my_sqrt(a, tol=1e-10)` using `while`
- Test on 2, 9, 100
- Count iterations needed for `a=2`

```
my_sqrt <- function(a, tol=1e-10) {  
  x <- a; n <- 0  
  while (TRUE) {  
    x_new <- 0.5 * (x + a/x); n <- n + 1  
    if (abs(x_new - x) < tol) break  
    x <- x_new  
  }  
  list(root=x_new, iterations=n)  
}
```

```
my_sqrt(2); my_sqrt(9)$root; my_sqrt(100)$root
```

```
#> $root  
#> [1] 1.414214  
#>  
#> ... [output truncated]  
#> [1] 3  
#> [1] 10
```

```
my_sqrt(2)$iterations
```

```
#> [1] 5
```

Collatz: if n even, $n/2$; if odd, $3n + 1$. Conjecture: always reaches 1.

- Write `collatz(n)` returning the sequence using `repeat/break`
- Find the starting number 1–100 with the longest sequence
- Plot the sequence for $n=27$

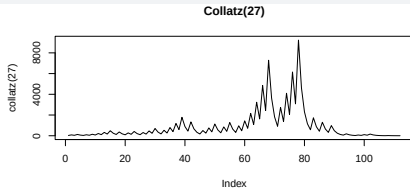
```
collatz <- function(n) {  
  seq <- n  
  repeat { if (n==1) break; n <- if(n%%2==0) n/2 else 3*n+1  
    seq <- c(seq, n) }  
  
  seq  
}  
lengths <- sapply(1:100, \(n) length(collatz(n)))  
which.max(lengths) # starting number
```



```
lengths <- sapply(1:100, \(n) length(collatz(n)))  
which.max(lengths) # starting number
```

```
#> [1] 97
```

```
plot(collatz(27), type="l", main="Collatz(27)")
```



- Sum integers 1-100 that are NOT multiples of 7, using `for+next`
- Same with vectorized filtering (no loop)
- Sum only prime numbers 1-50. Write `is_prime(n)` helper

```
s <- 0; for(i in 1:100) {if(i%%7==0) next; s<-s+i}; s
```

```
#> [1] 4315
```

```
sum((1:100)[(1:100)%%7 != 0])           # same
```

```
#> [1] 4315
```

```
is_prime <- function(n) {  
  if (n < 2) return(FALSE)  
  if (n == 2) return(TRUE)  
  all(n %% 2:floor(sqrt(n)) != 0)  
}  
sum(Filter(is_prime, 1:50))
```

```
#> [1] 325
```

- Write loop version of `sum(x^2)` for `x <- 1:1e5`
- Compare `system.time` with vectorized `sum(x^2)`
- Pre-allocate vs grow-in-loop: time `v <- c(); for(...) v <- c(v,i)` vs `v <- numeric(n); for(...)`
`v[i] <- i`

```
n <- 1e5; x <- 1:n
loop_sum <- function(x) {s<-0; for(v in x) s<-s+v^2; s}
system.time(loop_sum(x))                # slow
```

```
#>      user  system elapsed
#> 0.003    0.000    0.004
```

```
system.time(sum(x^2))                    # fast
```

```
#>      user  system elapsed
#>      0      0      0
```

```
system.time({v<-c(); for(i in 1:n) v<-c(v,i)}) # very slow
```

```
#>      user  system elapsed
#> 4.653    0.083    4.737
```

```
system.time({v<-numeric(n); for(i in 1:n) v[i]<-i}) # faster
```

```
#>      user  system elapsed
#> 0.004    0.000    0.004
```

- Implement `sieve(n)` returning all primes up to `n` using a logical vector and nested loops
- Test with `sieve(50)`
- How many primes below 1000?

```
sieve <- function(n) {  
  is_p <- rep(TRUE, n); is_p[1] <- FALSE  
  for (i in 2:floor(sqrt(n)))  
    if (is_p[i]) is_p[seq(i*2, n, i)] <- FALSE  
  which(is_p)  
}  
sieve(50)
```

```
#> [1]  2  3  5  7 11 13 17 19 23 29 31 37 41 43 47
```

```
length(sieve(1000)) # 168
```

```
#> [1] 168
```

- Write `my_fact(n)` using `Recall`
- Write `fib(n)` using `Recall`
- Compute `sapply(1:10, fib)`
- Why is naive recursive `fib` slow for large `n`?


```
my_fact <- function(n) {  
  if (n <= 1) return(1); n * Recall(n-1)  
}  
my_fact(5); my_fact(10)
```

```
#> [1] 120
```

```
#> [1] 3628800
```

```
fib <- function(n) {  
  if (n <= 2) return(1); Recall(n-1) + Recall(n-2)  
}  
sapply(1:10, fib)
```

```
#> [1] 1 1 2 3 5 8 13 21 34 55
```

```
# exponential time: each call branches into 2
```

- Write `summary_stat(x, type=c("mean", "median", "sd"))` using `match.arg`. Test partial matching
- Write `f(x, y)` that checks `missing(y)` and uses default `y=0`
- Show `nargs()` inside a function

```
summary_stat <- function(x, type=c("mean","median","sd")) {  
  type <- match.arg(type)  
  switch(type, mean=mean(x), median=median(x), sd=sd(x))  
}  
summary_stat(1:10, "mea") # mean
```

```
#> [1] 5.5
```

```
f <- function(x, y) {  
  if (missing(y)) y <- 0; x + y  
}  
f(5); f(5, 3)
```

```
#> [1] 5
```

```
#> [1] 8
```

```
g <- function(a,b,c) nargs(); g(1); g(1,2,3)
```

```
#> [1] 1
```

```
#> [1] 3
```

- Write `my_print(x)` that calls and returns `invisible(x)`. Show it doesn't print the return
- Capture the invisible return
- Write `log_and_return(x)` that prints a message and returns `invisible(NULL)`

```
my_print <- function(x) { cat("Value:", x, "\n"); invisible(x) }  
my_print(42)           # prints message, no [1] 42
```

```
#> Value: 42
```

```
val <- my_print(42); val           # captured
```

```
#> Value: 42
```

```
#> [1] 42
```

```
log_return <- function(x) {  
  cat("Processed:", x, "\n"); invisible(NULL)  
}  
r <- log_return(5); is.null(r)     # TRUE
```

```
#> Processed: 5
```

```
#> [1] TRUE
```

- Write function that sets `options(digits=3)`, uses `on.exit` to restore
- Write function with two `on.exit(add=TRUE)` calls. Show both run
- Show cleanup runs even on error

```
show_digits <- function() {  
  old <- options(digits=3)  
  on.exit(options(old))  
  cat("Inside:", getOption("digits"), "\n")  
}
```

```
show_digits(); cat("Outside:", getOption("digits"), "\n")
two_exits <- function() {
  on.exit(cat("exit1\n"), add=TRUE)
  on.exit(cat("exit2\n"), add=TRUE)
  cat("body\n")
}
two_exits()
tryCatch({
  f <- function() { on.exit(cat("cleanup!\n")); stop("err") }
  f()
}, error = function(e) cat("caught:", e$message, "\n"))
```

```
#> body
#> ... [output truncated]
#> cleanup!
#> ... [output truncated]
```


- Write scalar `classify(x)` using `if/else` (not vectorized)
- Use `Vectorize` to create vector version. Test on `c(-2, 0, 5)`
- Why is `Vectorize` just a wrapper around `mapply`?

```
classify <- function(x) {  
  if (x > 0) "pos" else if (x < 0) "neg" else "zero"  
}  
classify_v <- Vectorize(classify)  
classify_v(c(-2, 0, 5))
```

```
#> [1] "neg" "zero" "pos"
```

```
# Vectorize calls mapply internally, applying the  
# scalar function element-wise. Not truly vectorized (still loops).
```

- Rewrite `round(mean(abs(c(-3,4,-1,2))), 2)` using `|>`
- Pipe `mtcars` through `subset` then `head`
- Use `_` placeholder: `mtcars |> lm(mpg ~ wt, data = _)`

```
c(-3,4,-1,2) |> abs() |> mean() |> round(2)
```

```
#> [1] 2.5
```

```
mtcars |> subset(cyl == 4) |> head(3)
```

```
#>           mpg cyl  disp hp drat   wt  qsec vs am
#> Datsun 710 22.8   4 108.0 93 3.85 2.32 18.61  1  1
#> Merc 240D 24.4   4 146.7 62 3.69 3.19 20.00  1  0
#> ... [output truncated]
```

```
mtcars |> lm(mpg ~ wt, data = _) |>
  coef()
```

```
#> (Intercept)          wt
#>  37.285126   -5.344472
```

- Use `do.call(paste, c(as.list(c("a","b","c")), sep="-"))`
- Use `do.call(rbind, list_of_dfs)` to stack 3 data frames
- Build a call to `seq` dynamically from a list of arguments

```
do.call(paste, c(as.list(c("a","b","c")), sep="-"))
```

```
#> [1] "a-b-c"
```

```
dfs <- list(data.frame(x=1,y=2), data.frame(x=3,y=4))  
do.call(rbind, dfs)
```

```
#>   x y  
#> 1 1 2  
#> 2 3 4  
#> ... [output truncated]
```

```
do.call(seq, list(from=1, to=10, by=2))
```

```
#> [1] 1 3 5 7 9
```

- Use `tryCatch(log("a"), error=function(e) NA)`
- Loop over `list(1,"a",4)`, wrapping each in `try`. Keep only successes
- Write `safe_div(a,b)` that stops on `b==0` and test with `tryCatch`

```
tryCatch(log("a"), error=function(e) NA)
```

```
#> [1] NA
```

```
inputs <- list(1, "a", 4)
results <- lapply(inputs, \(x) try(sqrt(x), silent=TRUE))
Filter(\(r) !inherits(r,"try-error"), results)
```

```
#> [[1]]
```

```
#> [1] 1
```

```
#>
```

```
#> ... [output truncated]
```

```
safe_div <- function(a,b) {if(b==0) stop("div0!"); a/b}
tryCatch(safe_div(10,0), error=function(e) Inf)
```

```
#> [1] Inf
```


- Write `my_mean(x)` with `stopifnot(is.numeric(x), length(x)>0)`
- Write `safe_sqrt(x)` that issues warning for negative input and returns `sqrt(abs(x))`
- Use `suppressWarnings` to silence it

```
my_mean <- function(x) {  
  stopifnot(is.numeric(x), length(x) > 0)  
  sum(x)/length(x)  
}  
my_mean(1:5); my_mean("abc")
```

```
#> [1] 3  
#> Error in my_mean("abc"): is.numeric(x) is not TRUE
```

```
safe_sqrt <- function(x) {  
  if (any(x < 0)) warning("Negative values!")  
  sqrt(abs(x))  
}  
safe_sqrt(c(4, -9, 16))
```

```
#> [1] 2 3 4
```

```
suppressWarnings(safe_sqrt(-1))
```

```
#> [1] 1
```

- Write `power_factory(n)` that returns a function computing x^n
- Create `square` and `cube` from it; test on 5
- Inspect the captured `n` with `environment(square)$n`
- Verify `square` and `cube` have different environments

```
power_factory <- function(n) function(x) x^n  
square <- power_factory(2); cube <- power_factory(3)  
square(5); cube(5)
```

```
#> [1] 25
```

```
#> [1] 125
```

```
environment(square)$n; environment(cube)$n      # 2; 3
```

```
#> [1] 2
```

```
#> [1] 3
```

```
identical(environment(square), environment(cube)) # FALSE
```

```
#> [1] FALSE
```

Write `make_counter()` returning `list(increment, get, reset)`.

- `increment()` adds 1 to internal `n` and returns it
- `get()` returns current `n`; `reset()` sets `n` back to 0
- Create two independent counters; show they don't share state
- Rewrite using `local()` for a single-instance counter

```
make_counter <- function() {  
  n <- 0  
  list(increment = function() { n <- n + 1; n },  
       get = function() n,  
       reset = function() { n <- 0 })  
}  
c1 <- make_counter(); c2 <- make_counter()  
c1$increment(); c1$increment(); c2$increment()
```

```
#> [1] 1
```

```
#> [1] 2
```

```
#> [1] 1
```

```
cat("c1:", c1$get(), " c2:", c2$get(), "\n")
```

```
#> c1: 2 c2: 1
```

```
c1$reset(); c1$get()
```

```
#> [1] 0
```

```
# local() single-instance  
counter <- local({
```

- Create 5 functions in a for loop: `funcs[[i]] <- function() i`. Call all 5. What happens?
- Fix using `force()` inside a wrapper function
- Fix using `lapply`
- Explain *why* the bug occurs (lazy evaluation + shared environment)

```
funcs <- list()
for (i in 1:5) funcs[[i]] <- function() i
sapply(funcs, \(f) f())           # all 5!
# Bug: all closures share the same env; i=5 after loop
```



```
make_fun <- function(val) { force(val); function() val }  
funcs <- list()  
for (i in 1:5) funcs[[i]] <- make_fun(i)  
sapply(funcs, \(f) f())
```

```
#> [1] 1 2 3 4 5
```

```
funcs <- lapply(1:5, \(i) function() i)  
sapply(funcs, \(f) f())
```

```
#> [1] 1 2 3 4 5
```

```
# Lazy eval: function() i captures a promise to i,  
# not its value. By call time, loop finished, i=5.
```

Write `make_memo_fib()` returning `list(fib, cache_size, clear_cache)`.

- Use a named list as cache; `fib(n)` checks cache before computing
- Time `fib(35)` first call vs second call
- How large is the cache after `fib(35)`?
- Clear cache and verify `fib(10)` still works

```
make_memo_fib <- function() {  
  cache <- list()  
  fib <- function(n) {  
    key <- as.character(n)  
    if (!is.null(cache[[key]])) return(cache[[key]])  
    result <- if (n <= 2) 1 else fib(n-1) + fib(n-2)  
    cache[[key]] <- result; result  
  }  
  list(fib = fib,  
       cache_size = function() length(cache),  
       clear_cache = function() { cache <- list() })  
}
```

```
mf <- make_memo_fib()
system.time(mf$fib(35))      # first
system.time(mf$fib(35))      # second: instant
mf$cache_size()             # 35
mf$clear_cache(); mf$fib(10)
```

```
#>      user  system elapsed
```

```
#>         0         0         0
```

```
#>      user  system elapsed
```

```
#>         0         0         0
```

```
#> [1] 35
```

```
#> [1] 55
```

Write `with_logging(f)` that wraps `f` to print call count on each invocation.

- Return a closure that increments a counter and calls `f(...)`
- Test: `logged_mean <- with_logging(mean)`. Call it 3 times
- Extend to also print elapsed time per call

```
with_logging <- function(f, fname = deparse(substitute(f))) {  
  count <- 0L  
  function(...) {  
    count <-< count + 1L  
    t0 <- proc.time()["elapsed"]  
    cat(sprintf("[%s] call #%d\n", fname, count))  
    result <- f(...)  
    cat(sprintf("  elapsed: %.4fs\n",  
      proc.time()["elapsed"] - t0))  
    result  
  }  
}
```

```
logged_mean <- with_logging(mean)
logged_mean(1:100); logged_mean(rnorm(1e5))
```

```
#> [mean] call #1
#>   elapsed: 0.0000s
#> [1] 50.5
#> [mean] call #2
#>   elapsed: 0.0030s
#> [1] -0.002356695
```

Write `make_account(owner, balance)` returning `list(deposit, withdraw, statement, balance)`.

- `deposit(amount)` adds and logs. `withdraw(amount)` subtracts; `stop()` if insufficient
- `statement()` returns a data frame of all transactions
- Create two accounts; verify independence


```
make_account <- function(owner, init = 0) {  
  bal <- init  
  log <- data.frame(time=Sys.time()[0], type=character(0),  
                    amount=numeric(0), balance=numeric(0))  
  record <- function(type, amount) {  
    log <- rbind(log, data.frame(  
      time=Sys.time(), type=type, amount=amount, balance=bal))  
  }  
  list(  
    deposit = function(amt) {  
      stopifnot(amt > 0); bal <- bal + amt  
      record("deposit", amt); invisible(bal)  
    }  
  )  
}
```

```

    balance = function() bal,
    statement = function() log)
}
alice <- make_account("Alice", 1000)
bob    <- make_account("Bob", 500)
alice$deposit(200); alice$withdraw(50); bob$deposit(100)
alice$balance(); bob$balance()
alice$statement()

```

```
#> [1] 1150
```

```
#> [1] 600
```

```

#>
#> 1 2026-04-01 14:03:54 deposit    200    1200
#> ... [output truncated]

```

Write `make_ema(alpha)` returning a function that computes an exponential moving average:

$$\text{EMA}_t = \alpha x_t + (1 - \alpha) \text{EMA}_{t-1}.$$

- First call initializes EMA to the input value
- Test with prices `c(100, 102, 101, 105, 103, 107, 106)` and $\alpha = 0.2$
- Create two EMAs with different α ; verify independence

```
make_ema <- function(alpha = 0.3) {  
  value <- NULL  
  function(x) {  
    if (is.null(value)) value <- x  
    else value <- alpha * x + (1 - alpha) * value  
    value  
  }  
}  
  
ema <- make_ema(0.2)  
sapply(c(100, 102, 101, 105, 103, 107, 106), ema)
```

```
#> [1] 100.0000 100.4000 100.5200 101.4160 101.7328  
#> [6] 102.7862 103.4290
```

```
ema_fast <- make_ema(0.5); ema_slow <- make_ema(0.1)  
sapply(c(100, 110, 90), ema_fast)
```

```
#> [1] 100.0 105.0 97.5
```

```
sapply(c(100, 110, 90), ema_slow)
```

```
#> [1] 100.0 101.0 99.9
```

Write `make_fib_iterator()` returning `list(next_val, reset)`.

- `next_val()` returns the next Fibonacci number on each call
- Use `replicate(12, fib_it$next_val())` to get first 12 values
- Reset and verify the sequence restarts

```
make_fib_iterator <- function() {  
  a <- 0; b <- 1  
  list(  
    next_val = function() {  
      val <- a; new <- a + b; a <-- b; b <-- new; val  
    },  
    reset = function() { a <-- 0; b <-- 1 })  
  }  
fib_it <- make_fib_iterator()  
replicate(12, fib_it$next_val())
```

```
#> [1] 0 1 1 2 3 5 8 13 21 34 55 89
```

```
fib_it$reset(); replicate(5, fib_it$next_val())
```

```
#> [1] 0 1 1 2 3
```

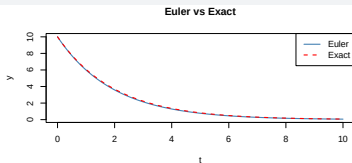
Write `make_ode_solver(f, y0, dt)` where $f(t, y)$ defines dy/dt .

- `step()` advances one time step; `run(n)` returns trajectory as a data frame
- Solve $dy/dt = -0.5y$, $y(0) = 10$, $dt = 0.1$ for 100 steps
- Plot Euler vs exact $y = 10e^{-0.5t}$

```
make_ode_solver <- function(f, y0, dt = 0.01) {  
  t <- 0; y <- y0  
  list(  
    run = function(n) {  
      traj <- matrix(NA, n+1, 2, dimnames=list(NULL, c("t","y")))  
      traj[1, ] <- c(t, y)  
      for (i in seq_len(n)) {  
        y <-< y + f(t, y)*dt; t <-< t + dt  
        traj[i+1, ] <- c(t, y)  
      }  
      as.data.frame(traj)  
    },  
    state = function() c(t=t, y=y),  
    reset = function(y0_new=y0) { t <-< 0; y <-< y0_new }  
  )  
}
```



```
solver <- make_ode_solver(\(t, y) -0.5*y, y0=10, dt=0.1)
traj <- solver$run(100)
plot(traj$t, traj$y, type='l', main='Euler vs Exact')
curve(10*exp(-0.5*x), add=TRUE, col='red', lty=2)
```



Write `make_rate_limiter(f, max_calls, period)` wrapping `f` so at most `max_calls` invocations are allowed per period-second window.

- If limit exceeded, return NULL with a warning
- Track timestamps in a closure variable
- Test: limiter allowing 3 calls per 2 seconds; call 5 times rapidly

```
make_rate_limiter <- function(f, max_calls=3, period=2) {  
  timestamps <- numeric(0)  
  function(...) {  
    now <- as.numeric(Sys.time())  
    timestamps <- timestamps[timestamps > now - period]  
    if (length(timestamps) >= max_calls) {  
      warning("Rate limit exceeded"); return(NULL)  
    }  
    timestamps <- c(timestamps, now)  
    f(...)  
  }  
}
```

```
limited_sqrt <- make_rate_limiter(sqrt, 3, 2)
for (i in 1:5) { r <- limited_sqrt(i*10)
  cat(sprintf("Call %d: %s\n", i,
    if (is.null(r)) "BLOCKED" else round(r, 4))) }
```

```
#> Call 1: 3.1623
#> Call 2: 4.4721
#> Call 3: 5.4772
#> ... [output truncated]
#> Call 4: BLOCKED
#> Call 5: BLOCKED
```

Write `make_portfolio(weights, tickers)` returning:

- `returns(price_matrix)` — weighted portfolio returns
- `sharpe(price_matrix, rf)` — annualized Sharpe ratio
- `drawdown(price_matrix)` — maximum drawdown
- Test with simulated 252-day prices for 3 assets

```
make_portfolio <- function(w, tickers) {  
  w <- w / sum(w); names(w) <- tickers  
  list(  
    returns = function(P) {  
      r <- apply(P, 2, \(x) diff(x)/head(x,-1)); drop(r %*% w)  
    },  
    sharpe = function(P, rf=0) {  
      r <- apply(P, 2, \(x) diff(x)/head(x,-1))
```

```
pr <- drop(r %*% w)
(mean(pr) - rf/252) / sd(pr) * sqrt(252)
},
drawdown = function(P) {
  r <- apply(P, 2, \(x) diff(x)/head(x,-1))
  cum <- cumprod(1 + drop(r %*% w))
  max(1 - cum / cummax(cum))
})
}
```

Sol 103 (Test)

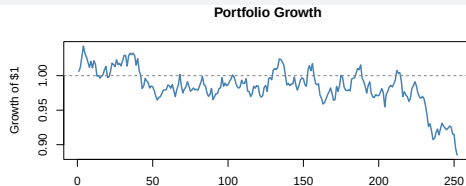
```
set.seed(42); P <- matrix(NA, 253, 3); P[1,] <- c(100,50,200)
for (i in 2:253) P[i,] <- P[i-1,]*exp(rnorm(3, 3e-4,
                                         c(.015,.02,.01)))
pf <- make_portfolio(c(.4,.3,.3), c("A","G","M"))
cat("Sharpe:", round(pf$sharpe(P), 3), "\n")
```

```
#> Sharpe: -0.828
```

```
cat("MaxDD:", round(pf$drawdown(P)*100, 2), "%\n")
```

```
#> MaxDD: 15.15 %
```

```
r <- pf$returns(P)
plot(cumprod(1+r), type='l', lwd=2, col='steelblue',
     main='Portfolio Growth', ylab='Growth of $1')
abline(h=1, lty=2, col='grey50')
```



Using `mtcars`:

- Use `by(mtcars, mtcars$cyl, \(d) cor(d$mpg, d$wt))` to compute per-cylinder correlation
- Mean, sd, min, max of `mpg` by `cyl` in one `aggregate` call; expand the matrix column
- Merge with group counts from `aggregate(..., FUN=length)`
- Sort by mean `mpg` descending

```
by(mtcars, mtcars$cyl, \(d) cor(d$mpg, d$wt))
```

```
#> mtcars$cyl: 4
```

```
#> [1] -0.7131848
```

```
#> -----
```

```
#> ... [output truncated]
```

```
res <- aggregate(mpg ~ cyl, data=mtcars,
```

```
  FUN=\(x) c(mean=mean(x), sd=sd(x), min=min(x), max=max(x)))
```

```
flat <- cbind(res[1], as.data.frame(res$mpg))
```

```
counts <- aggregate(mpg ~ cyl, data=mtcars, FUN=length)
```

```
names(counts)[2] <- 'n'
```

```
flat <- merge(flat, counts, by='cyl')
```

```
flat[order(-flat$mean), ]
```

```
#>   cyl      mean      sd  min  max  n
```

```
#> 1   4 26.66364 4.509828 21.4 33.9 11
```

```
#> 2   6 19.74286 1.453567 17.8 21.4  7
```

```
#> ... [output truncated]
```

The Apply Family

apply() — Over Matrix / Data Frame Margins

```
m <- matrix(1:12, nrow = 3,  
            dimnames = list(paste0('r',1:3), paste0('c',1:4)))
```

```
m
```

```
#>      c1 c2 c3 c4  
#> r1   1  4  7 10  
#> r2   2  5  8 11  
#> ... [output truncated]
```

```
apply(m, 1, sum)      # MARGIN=1: rows
```

```
#> r1 r2 r3  
#> 22 26 30
```

```
apply(m, 2, sum)      # MARGIN=2: columns
```

```
#> c1 c2 c3 c4  
#>  6 15 24 33
```

apply() — When FUN Returns a Vector

```
# when FUN returns a vector, result is k x n matrix
```

```
apply(m, 2, range)
```

```
#>      c1 c2 c3 c4  
#> [1,]  1  4  7 10  
#> [2,]  3  6  9 12  
#> ... [output truncated]
```

```
apply(m, 2, quantile, probs = c(0.25, 0.75))
```

```
#>      c1  c2  c3   c4  
#> 25% 1.5 4.5 7.5 10.5  
#> 75% 2.5 5.5 8.5 11.5  
#> ... [output truncated]
```

apply() on Data Frames — Beware of Coercion!

```
df <- data.frame(x = 1:3, y = c('a','b','c'), z = 4:6)
apply(df, 2, class)      # all "character"!
```

```
#>           x           y           z
#> "character" "character" "character"
```

- apply() converts data frames to matrices first — **all columns become character** if any is character. Use sapply/lapply for mixed types.

```
# faster alternatives for row/column sums/means:
colSums(m); rowSums(m); colMeans(m); rowMeans(m)
```

lapply() and sapply()

```
x <- list(a = 1:5, b = rnorm(10), c = c(TRUE, FALSE, TRUE))  
lapply(x, mean)      # always returns a list
```

```
#> $a  
#> [1] 3  
#>  
#> ... [output truncated]
```

```
sapply(x, mean)      # simplifies to named vector
```

```
#>          a          b          c  
#> 3.0000000 0.3021327 0.6666667
```

```
# sapply can be unpredictable; vapply is safer  
vapply(x, mean, numeric(1))
```

```
#>          a          b          c  
#> 3.0000000 0.3021327 0.6666667
```

sapply() Is Unpredictable — Use vapply() in Production

```
# when all lengths equal: matrix. When they differ: list!
```

```
sapply(list(a = 1:3, b = 4:6), identity)
```

```
#>      a b  
#> [1,] 1 4  
#> [2,] 2 5  
#> ... [output truncated]
```

```
sapply(list(a = 1:3, b = 4:5), identity)
```

```
#> $a  
#> [1] 1 2 3  
#>  
#> ... [output truncated]
```

```
# vapply: explicit type template; error if mismatch
```

```
vapply(iris[,1:4], median, numeric(1))
```

```
#> Sepal.Length Sepal.Width Petal.Length  Petal.Width  
#>           5.80           3.00           4.35           1.30
```



```
# mean mpg by cylinder count
```

```
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
#>      4      6      8
```

```
#> 26.66364 19.74286 15.10000
```

```
# two-way grouping -> matrix
```

```
tapply(mtcars$mpg, list(mtcars$cyl, mtcars$am), mean)
```

```
#>      0      1
```

```
#> 4 22.900 28.07500
```

```
#> 6 19.125 20.56667
```

```
#> ... [output truncated]
```

- `tapply(x, f, FUN) ≡ sapply(split(x, f), FUN)`

mapply() and Map()

```
# apply to corresponding elements of multiple vectors  
mapply(function(x, y) x + y, 1:5, 10:14)
```

```
#> [1] 11 13 15 17 19
```

```
mapply(rep, 1:3, 3:1) # variable-length results
```

```
#> [[1]]
```

```
#> [1] 1 1 1
```

```
#>
```

```
#> ... [output truncated]
```

```
# Map: always returns a list (no simplification)
```

```
Map('+', 1:3, 10:12)
```

```
#> [[1]]
```

```
#> [1] 11
```

```
#>
```

```
#> ... [output truncated]
```

mapply() and Map() Cont'd

```
# MoreArgs: constant arguments not varied across elements
mapply(function(x, y, suffix) paste0(x, y, suffix),
        x = c('Alice', 'Bob'), y = c(25, 30),
        MoreArgs = list(suffix = '!'))
```

```
#>      Alice      Bob
#> "Alice25!"  "Bob30!"
```

rapply() — Recursive Apply

```
nested <- list(a = 1:3,  
  b = list(c = 4:6, d = list(e = 7:9, f = 'text')))  
rapply(nested, sum, classes = 'integer')          # unlist
```

```
#>      a    b.c b.d.e  
#>      6     15     24  
#> ... [output truncated]
```

```
rapply(nested, sum, classes = 'integer', how = 'list')
```

```
#> $a  
#> [1] 6  
#> ... [output truncated]
```

```
rapply(nested, \(x) x * 10, classes = 'integer', how = 'replace')
```

```
#> $a  
#> [1] 10 20 30  
#> ... [output truncated]
```

Reduce() — Sequential Accumulation

```
Reduce('+', 1:5) # 15
```

```
#> [1] 15
```

```
Reduce('+', 1:5, accumulate = TRUE) # running total
```

```
#> [1] 1 3 6 10 15
```

```
# merge multiple data frames
```

```
dfs <- list(data.frame(id=1:3, a=10:12),  
            data.frame(id=2:4, b=20:22),  
            data.frame(id=1:3, c=30:32))  
Reduce(\(x, y) merge(x, y, by='id', all=TRUE), dfs)
```

```
#>   id  a  b  c
```

```
#> 1  1 10 NA 30
```

```
#> 2  2 11 20 31
```

```
#> ... [output truncated]
```

Filter(), Find(), Position(), Negate()

```
x <- list(1:3, "hello", 4:6, TRUE, NULL)
Filter(is.numeric, x)          # keep numeric elements
```

```
#> [[1]]
#> [1] 1 2 3
#>
#> ... [output truncated]
```

```
Find(is.character, x)        # first character element
```

```
#> [1] "hello"
```

```
Position(is.character, x)    # its position
```

```
#> [1] 2
```

```
Filter(\(x) x %% 15 == 0, 1:50) # multiples of 15
```

```
#> [1] 15 30 45
```

```
# Find/Position with direction
```

```
Find(\(x) x > 5, c(2, 8, 3, 9, 1), right = TRUE)
```

```
#> [1] 9
```

```
Position(\(x) x > 5, c(2, 8, 3, 9, 1), right = TRUE)
```

Decision Table: Choosing the Right Tool

Function	Input	Output	Use case
<code>apply</code>	matrix/array	vector/matrix	row/column ops
<code>lapply</code>	vector/list	list	general iteration
<code>sapply</code>	vector/list	simplified	interactive only
<code>vapply</code>	vector/list	typed output	production code
<code>tapply</code>	vector + factor	array	group summaries
<code>mapply/Map</code>	multiple vectors	simplified/list	parallel iteration
<code>rapply</code>	nested list	various	recursive ops
<code>Reduce</code>	vector/list	single value	sequential fold
<code>Filter</code>	vector/list	subset	keep matching
<code>Find/Position</code>	vector/list	one element	first match
<code>Negate</code>	function	function	invert predicate

Common Equivalences

```
tapply(x, f, FUN)      <==> sapply(split(x, f), FUN)
sapply(X, FUN)         <==> unlist(lapply(X, FUN))    # scalar FUN
mapply(FUN, x, y)      <==> sapply(seq_along(x),
                                   \ (i) FUN(x[i], y[i]))
Map(FUN, x, y)         <==> mapply(FUN, x, y, SIMPLIFY = FALSE)
Reduce('+', 1:5)       <==> (((1+2)+3)+4)+5
Filter(f, x)           <==> x[sapply(x, f)]
aggregate(y~g, d, mean) <==> tapply(d$y, d$g, mean)  # approx
```



```
m <- matrix(c(3,1,4,1,5,9,2,6,5,3,5,8), nrow=3).
```

- Column means via `apply`
- Row ranges
- Apply custom function returning a named vector per column
- Apply on a data frame's numeric columns

```
m <- matrix(c(3,1,4,1,5,9,2,6,5,3,5,8), nrow=3)
apply(m, 2, mean)
```

```
#> [1] 2.666667 5.000000 4.333333 5.333333
```

```
apply(m, 1, range) # 2x3 matrix
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    1    4
#> [2,]    3    6    9
#> ... [output truncated]
```

```
apply(m, 2, \(x) c(mu=mean(x), sd=sd(x)))
```

```
#>      [,1] [,2]      [,3]      [,4]
#> mu 2.666667    5 4.333333 5.333333
#> sd 1.527525    4 2.081666 2.516611
#> ... [output truncated]
```

```
apply(mtcars[1:3, 1:4], 2, mean)
```

```
#>      mpg      cyl      disp      hp
#> 21.600000  5.333333 142.666667 104.333333
```

```
l <- list(a=1:10, b=seq(0,1,by=0.1), c=c(T,F,T,T,F)).
```

- `sapply(l, length)`
- `sapply(l, range)` — what shape?
- `vapply(l, mean, numeric(1))`
- Why prefer `vapply` over `sapply`?

```
l <- list(a=1:10, b=seq(0,1,by=0.1), c=c(T,F,T,T,F))  
sapply(l, length)                                # named vector
```

```
#>  a  b  c  
#> 10 11 5
```

```
sapply(l, range)                                # 2x3 matrix
```

```
#>      a b c  
#> [1,] 1 0 0  
#> [2,] 10 1 1  
#> ... [output truncated]
```

```
vapply(l, mean, numeric(1))                    # named vector
```

```
#>  a  b  c  
#> 5.5 0.5 0.6
```

```
# vapply guarantees return type; sapply may return list  
# unexpectedly if elements have different lengths
```

- Mean mpg by cyl in mtcars
- Two-way: mean mpg by cyl $\times$ am
- Use `tapply` on ToothGrowth: mean len by supp and dose

```
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
#>      4      6      8  
#> 26.66364 19.74286 15.10000
```

```
tapply(mtcars$mpg, list(mtcars$cyl, mtcars$am), mean)
```

```
#>      0      1  
#> 4 22.900 28.07500  
#> 6 19.125 20.56667  
#> ... [output truncated]
```

```
tapply(ToothGrowth$len,  
       list(ToothGrowth$dose, ToothGrowth$dose), mean)
```

```
#>      0.5      1      2  
#> 0J 13.23 22.70 26.06  
#> VC  7.98 16.77 26.14  
#> ... [output truncated]
```

- `mapply(function(x,y) x+y, 1:5, 10:14)`
- `mapply(rep, 1:4, 4:1)`
- `Map("+", 1:3, 10:12)` — what type?
- Use `Map` to apply paste element-wise to two vectors

```
mapply(function(x,y) x+y, 1:5, 10:14)
```

```
#> [1] 11 13 15 17 19
```

```
mapply(rep, 1:4, 4:1) # list
```

```
#> [[1]]
```

```
#> [1] 1 1 1 1
```

```
#>
```

```
#> ... [output truncated]
```

```
Map("+", 1:3, 10:12) # list
```

```
#> [[1]]
```

```
#> [1] 11
```

```
#>
```

```
#> ... [output truncated]
```

```
Map(paste, c("a","b","c"), 1:3, sep="-")
```

```
#> $a
```

```
#> [1] "a-1"
```

```
#>
```

```
#> ... [output truncated]
```


- Factorial via `Reduce("*", 1:6)`
- Cumulative products via `accumulate=TRUE`
- Find the common elements of 4 vectors using `Reduce(intersect, ...)`

```
Reduce("*", 1:6) # 720
```

```
#> [1] 720
```

```
Reduce("*", 1:6, accumulate=TRUE)
```

```
#> [1] 1 2 6 24 120 720
```

```
Reduce(intersect, list(1:10, 3:12, 5:15, 7:20)) # 7:10
```

```
#> [1] 7 8 9 10
```

Process `x <- c(-5, 3, -1, 8, -2, 7)`:

- Filter positive $\rightarrow$ square $\rightarrow$ sum. Do with nested calls
- Same with pipe `|>`
- Same with Reduce composing 3 functions

```
x <- c(-5, 3, -1, 8, -2, 7)
sum(x[x>0]^2) # nested
```

```
#> [1] 122
```

```
x[x>0] |> (\(v) v^2)() |> sum() # pipe
```

```
#> [1] 122
```

```
fns <- list(\(v) v[v>0], \(v) v^2, sum)
Reduce(\(v,f) f(v), fns, init=x)
```

```
#> [1] 122
```

Using `iris`:

- Mean `Sepal.Length` by `Species` with `tapply`
- Same with `aggregate`
- Mean of all numeric columns by `Species` with `aggregate(. ~ Species, ...)`

```
tapply(iris$Sepal.Length, iris$Species, mean)
```

```
#>      setosa versicolor  virginica  
#>      5.006      5.936      6.588
```

```
aggregate(Sepal.Length ~ Species, data=iris, mean)
```

```
#>      Species Sepal.Length  
#> 1      setosa      5.006  
#> 2 versicolor      5.936  
#> ... [output truncated]
```

```
aggregate(. ~ Species, data=iris, mean)
```

```
#>      Species Sepal.Length Sepal.Width Petal.Length  
#> 1      setosa      5.006      3.428      1.462  
#> 2 versicolor      5.936      2.770      4.260  
#> ... [output truncated]
```

- Write `make_memo(fn)` that returns a memoized version storing results in a list environment
- Memoize `fib`
- Show second call is instant

```
make_memo <- function(fn) {  
  cache <- list()  
  function(...) {  
    key <- paste(..., sep=",")  
    if (is.null(cache[[key]])) cache[[key]] <- fn(...)  
    cache[[key]]  
  }  
}  
  
slow_fn <- function(x) { Sys.sleep(0.1); x^2 }  
fast_fn <- make_memo(slow_fn)  
system.time(fast_fn(5))                # slow
```

```
#>      user  system elapsed  
#>  0.000    0.000    0.101
```

```
system.time(fast_fn(5))                # instant
```

```
#>      user  system elapsed  
#>         0         0         0
```


- Given `l <- list(a=1:3, b=4:6)`, use `lapply(seq_along(l), ...)` to create "a: 1 2 3", "b: 4 5 6"
- Same with `Map` and `names(l)`
- Same with `mapply`

```
l <- list(a=1:3, b=4:6)
lapply(seq_along(l), \(i)
  paste0(names(l)[i], ": ", paste(l[[i]], collapse=" ")))
```

```
#> [[1]]
#> [1] "a: 1 2 3"
#> ... [output truncated]
```

```
Map(\(nm, v) paste0(nm, ": ", paste(v, collapse=" ")),
    names(l), l)
```

```
#> $a
#> [1] "a: 1 2 3"
#> ... [output truncated]
```

```
mapply(\(nm,v) paste0(nm,": ",paste(v,collapse=" ")),
       names(l), l)
```

```
#>           a           b
#> "a: 1 2 3" "b: 4 5 6"
#> ... [output truncated]
```

- Generate a 3×3 multiplication table using `lapply(1:3, \ (i) sapply(1:3, \ (j) i*j))`
- Convert to a matrix with `do.call(rbind, ...)`
- Compare with `outer(1:3, 1:3)`

```
l <- lapply(1:3, \(i) sapply(1:3, \(j) i*j))  
do.call(rbind, l)
```

```
#>      [,1] [,2] [,3]  
#> [1,]    1    2    3  
#> [2,]    2    4    6  
#> ... [output truncated]
```

```
outer(1:3, 1:3) # same
```

```
#>      [,1] [,2] [,3]  
#> [1,]    1    2    3  
#> [2,]    2    4    6  
#> ... [output truncated]
```

```
prices <- list(c(100,102,98), c(50,55,52), c(200,210,205)). names_v <- c("AAPL", "GOOG",  
"MSFT").
```

- Compute $\text{return} = \text{diff}(p)/p[-\text{length}(p)]$ for each using Map
- Compute mean return per stock

```
prices <- list(c(100,102,98), c(50,55,52), c(200,210,205))
names_v <- c("AAPL","GOOG","MSFT")
returns <- Map(\(p) diff(p)/p[-length(p)], prices)
names(returns) <- names_v
sapply(returns, mean)
```

```
#>           AAPL           GOOG           MSFT
#> -0.009607843  0.022727273  0.013095238
```

- From `1:100`, filter multiples of 3, then sum with Reduce
- From a list of vectors, keep only those with `length > 3`, then find their common intersection
- Compare with direct `sum(Filter(...))`

```
Reduce("+", Filter(\(x) x%%3==0, 1:100))
```

```
#> [1] 1683
```

```
vecs <- list(1:10, 3:8, 2:15, 5:12, 4:20)
```

```
long <- Filter(\(v) length(v)>3, vecs)
```

```
Reduce(intersect, long) # 5:8
```

```
#> [1] 5 6 7 8
```

```
sum(Filter(\(x) x%%3==0, 1:100)) # same as (a)
```

```
#> [1] 1683
```


String Operations

Basic String Functions

```
nchar('hello world')
```

```
#> [1] 11
```

```
toupper('hello'); tolower('HELLO')
```

```
#> [1] "HELLO"
```

```
#> [1] "hello"
```

```
trimws(' hello '); trimws(' hello ', which = 'left')
```

```
#> [1] "hello"
```

```
#> [1] "hello "
```

Basic String Functions Cont'd

```
substr('abcdefgh', 3, 5)
```

```
#> [1] "cde"
```

```
startsWith('hello world', 'hello')
```

```
#> [1] TRUE
```

```
endsWith('hello world', 'world')
```

```
#> [1] TRUE
```

```
chartr('abc', 'ABC', 'a big cat')
```

```
#> [1] "A Big CAAt"
```

paste() and paste0()

```
paste('a', 'b', 'c', sep = '-')
```

```
#> [1] "a-b-c"
```

```
paste0('item', 1:5)
```

```
#> [1] "item1" "item2" "item3" "item4" "item5"
```

```
(labels <- paste(c('a', 'b'), 1:6, sep = '-'))
```

```
#> [1] "a-1" "b-2" "a-3" "b-4" "a-5" "b-6"
```

```
# collapse: join multiple strings into one
```

```
paste('item', 1:5, sep = '_', collapse = ', ')
```

```
#> [1] "item_1, item_2, item_3, item_4, item_5"
```

sprintf() — Formatted Strings

%d: integer; %.2f: 2 decimal places; %s: string

```
sprintf('Name: %s, Age: %d, Score: %.2f', 'Alice', 25, 95.678)
```

```
#> [1] "Name: Alice, Age: 25, Score: 95.68"
```

```
sprintf('%05d', 42)          # zero-padded
```

```
#> [1] "00042"
```

```
sprintf('%.2f%%', 12.345)    # percentage
```

```
#> [1] "12.35%"
```

```
sprintf('%-10s|', 'left')    # left-align
```

```
#> [1] "left      |"
```

cat() — Print to Console

```
v <- c(20, 30, 3 * pi)
cat("The content of v is:", v, "...\\n")
```

```
#> The content of v is: 20 30 9.424778 ...
```

```
for (i in 50 * (0:2)) {
  cat(sprintf('exp(sin(%3d)) = %.6f', i, exp(sin(i))), '\\n')
}
```

```
#> exp(sin( 0)) = 1.000000
```

```
#> exp(sin( 50)) = 0.769223
```

```
#> exp(sin(100)) = 0.602682
```

```
#> ... [output truncated]
```

strsplit() — Split Strings

```
strsplit('a.b.c.d', split = '\\.')
```

```
#> [[1]]  
#> [1] "a" "b" "c" "d"  
#> ... [output truncated]
```

```
strsplit('hello world foo', split = ' ')
```

```
#> [[1]]  
#> [1] "hello" "world" "foo"  
#> ... [output truncated]
```

```
strsplit(c('a-b', 'c-d-e'), split = '-')
```

```
#> [[1]]  
#> [1] "a" "b"  
#> ... [output truncated]
```

```
strsplit('hello', split = '') # every character
```

```
#> [[1]]  
#> [1] "h" "e" "l" "l" "o"  
#> ... [output truncated]
```

```
x <- c('apple', 'banana', 'cherry', 'apricot', 'blueberry')
grep('an', x)                                # indices
```

```
#> [1] 2
```

```
grep('an', x, value = TRUE)                  # values
```

```
#> [1] "banana"
```

```
grepl('an', x)                               # logical
```

```
#> [1] FALSE TRUE FALSE FALSE FALSE
```

```
grep('APPLE', x, ignore.case = TRUE, value = TRUE)
```

```
#> [1] "apple"
```


sub(), gsub() — Pattern Replacement

```
x <- 'Hello World Hello'
sub('Hello', 'Hi', x)      # replace FIRST match
```

```
#> [1] "Hi World Hello"
```

```
gsub('Hello', 'Hi', x)    # replace ALL matches
```

```
#> [1] "Hi World Hi"
```

```
gsub('[0-9]', '', 'abc123def456')  # remove all digits
```

```
#> [1] "abcdef"
```

```
gsub(' +', ' ', 'too many spaces') # compress spaces
```

```
#> [1] "too many spaces"
```

`regexpr()`, `gregexpr()`, `regmatches()`

```
x <- 'Phone: 02-1234-5678, Fax: 04-8765-4321'
m <- gregexpr('[0-9]+-[0-9]+-[0-9]+', x)
regmatches(x, m)      # extract all matches
```

```
#> [[1]]
```

```
#> [1] "02-1234-5678" "04-8765-4321"
```

```
x <- c("  Hello World  ", "R Programming", "data SCIENCE").
```

- Trim whitespace; count chars of each
- Convert to uppercase, lowercase
- `startsWith` / `endsWith` tests
- Extract first 4 chars using `substr`

```
x <- c(" Hello World ", "R Programming", "data SCIENCE")  
xt <- trimws(x); nchar(xt)
```

```
#> [1] 11 13 12
```

```
toupper(xt); tolower(xt)
```

```
#> [1] "HELLO WORLD" "R PROGRAMMING" "DATA SCIENCE"
```

```
#> [1] "hello world" "r programming" "data science"
```

```
startsWith(xt, "Hello"); endsWith(xt, "SCIENCE")
```

```
#> [1] TRUE FALSE FALSE
```

```
#> [1] FALSE FALSE TRUE
```

```
substr(xt, 1, 4)
```

```
#> [1] "Hell" "R Pr" "data"
```

- `paste("item", 1:5, sep="_")`
- Same with `collapse=", "`
- `sprintf("Name: %s, Score: %.1f", "Alice", 95.678)`
- Zero-pad: `sprintf("%03d", 1:5)`

```
paste("item", 1:5, sep="_")
```

```
#> [1] "item_1" "item_2" "item_3" "item_4" "item_5"
```

```
paste("item", 1:5, sep="_", collapse=", ")
```

```
#> [1] "item_1, item_2, item_3, item_4, item_5"
```

```
sprintf("Name: %s, Score: %.1f", "Alice", 95.678)
```

```
#> [1] "Name: Alice, Score: 95.7"
```

```
sprintf("%03d", 1:5)
```

```
#> [1] "001" "002" "003" "004" "005"
```

- Split "a.b.c.d" by "." (note the escape!)
- Split "hello world foo" by space
- Split each character of "ABCDE"
- Split `c("a-b", "c-d-e")` — what shape is the result?

```
strsplit("a.b.c.d", "\\.")
```

```
#> [[1]]  
#> [1] "a" "b" "c" "d"
```

```
strsplit("hello world foo", " ")
```

```
#> [[1]]  
#> [1] "hello" "world" "foo"
```

```
strsplit("ABCDE", "")
```

```
#> [[1]]  
#> [1] "A" "B" "C" "D" "E"
```

```
strsplit(c("a-b","c-d-e"), "-")
```

```
# list of 2
```

```
#> [[1]]  
#> [1] "a" "b"  
#>  
#> ... [output truncated]
```


Regular Expressions

What Are Regular Expressions?

- Patterns for matching text. Used in `grep`, `grep1`, `sub`, `gsub`, `regexpr`, `strsplit`, etc.
- In R strings, backslash must be **doubled**: write `\\d` for the regex `\d`.

Pattern	Meaning
.	any single character (except newline)
^	start of string
\$	end of string
*	0 or more of preceding
+	1 or more of preceding
?	0 or 1 of preceding
{n}, {n,m}	exactly n; between n and m
	alternation (or)
()	grouping and capture
[]	character class
\	escape next character

Character Classes

Class	Meaning
<code>[abc]</code>	matches a, b, or c
<code>[^abc]</code>	matches anything except a, b, c
<code>[a-z], [A-Z], [0-9]</code>	ranges
<code>\d / \D</code>	digit / non-digit
<code>\w / \W</code>	word char / non-word char
<code>\s / \S</code>	whitespace / non-whitespace
<code>\b</code>	word boundary

```
x <- c('hello123', 'world', '456', 'test!', 'a1b2')
grep('[0-9]', x, value = TRUE)      # contains a digit
```

```
#> [1] "hello123" "456"      "a1b2"
```

```
grep('^[0-9]+$ ', x, value = TRUE) # all digits
```

```
#> [1] "456"
```

Anchors, Quantifiers, Alternation

```
x <- c('cat', 'concatenate', 'scat', 'catalog')  
grep('^cat', x, value = TRUE)      # starts with 'cat'
```

```
#> [1] "cat"      "catalog"
```

```
grep('cat$', x, value = TRUE)      # ends with 'cat'
```

```
#> [1] "cat" "scat"
```

```
x <- c('cat', 'dog', 'catfish', 'dogfish', 'bird')  
grep('cat|dog', x, value = TRUE)    # cat OR dog
```

```
#> [1] "cat"      "dog"      "catfish" "dogfish"
```

```
grep('^(cat|dog)$', x, value = TRUE) # exactly cat or dog
```

```
#> [1] "cat" "dog"
```

Capture Groups and Backreferences

```
# swap first and last name
x <- c('John Smith', 'Jane Doe', 'Bob Lee')
sub('(\\w+) (\\w+)', '\\2, \\1', x)

#> [1] "Smith, John" "Doe, Jane"  "Lee, Bob"
```

```
# extract parts of a date
dates <- c('2024-01-15', '2023-12-31')
sub('(\\d{4})-(\\d{2})-(\\d{2})', 'Year:\\1, Month:\\2', dates)

#> [1] "Year:2024, Month:01" "Year:2023, Month:12"
```

Greedy vs Lazy Matching

```
x <- '<b>bold</b> and <i>italic</i>'
regmatches(x, gregexpr('<.*>', x))    # greedy
```

```
#> [[1]]
#> [1] "<b>bold</b> and <i>italic</i>"
```

```
regmatches(x, gregexpr('<.*?>', x))    # lazy (non-greedy)
```

```
#> [[1]]
#> [1] "<b>" "</b>" "<i>" "</i>"
```

Lookahead and Lookbehind (Perl)

```
x <- c('$100', '200', '$300', '400')  
regmatches(x, gregexpr('(?!<=\\$)\\d+', x, perl = TRUE))
```

```
#> [[1]]  
#> [1] "100"  
#>  
#> ... [output truncated]
```

```
# negative lookahead: files NOT ending in .txt  
x <- c('file.txt', 'image.png', 'doc.txt', 'photo.jpg')  
grep('\\.(?!txt)', x, value = TRUE, perl = TRUE)
```

```
#> [1] "image.png" "photo.jpg"
```



```
x <- c('2024-01-15 14:30', '2023-12-31 09:00')
pattern <- '(\d{4})-(\d{2})-(\d{2}) (\d{2}):(\d{2})'
m <- regexec(pattern, x)
regmatches(x, m)
```

```
#> [[1]]
#> [1] "2024-01-15 14:30" "2024"
#> [3] "01"                "15"
#> ... [output truncated]
```

Practical Regex Examples

```
# validate email (simplified)
emails <- c('user@example.com', 'bad@@email',
           'test@domain.org', 'no_at_sign')
grep('^[\\w.]+@[\\w.]+\\. [a-z]{2,}$', emails,
     value = TRUE, perl = TRUE)
```

```
#> [1] "user@example.com" "test@domain.org"
```

```
# clean column names: replace non-alphanumeric with _
col_names <- c('First Name', 'Last.Name', 'Age (years)')
gsub('[^a-zA-Z0-9]', '_', col_names)
```

```
#> [1] "First_Name" "Last_Name" "Age__years_"
```

```
w <- c("statistics","computation","algorithm","distribution","estimation").
```

- Words containing "tion"
- Words starting with vowel
- Words of ≥ 12 characters
- grepl to create logical mask

```
w <- c("statistics","computation","algorithm",  
      "distribution","estimation")  
grep("tion", w, value=TRUE)  
  
#> [1] "computation" "distribution" "estimation"
```

```
grep("^[aeiou]", w, value=TRUE)
```

```
#> [1] "algorithm" "estimation"
```

```
w[nchar(w) >= 12]
```

```
#> [1] "distribution"
```

```
grepl("tion", w)
```

```
#> [1] FALSE TRUE FALSE TRUE TRUE
```

- Replace first space with "_" in "hello world foo"
- Replace all spaces
- Remove all digits from "abc123def456"
- Compress multiple spaces to one

```
sub(" ", "_", "hello world foo")           # first only
```

```
#> [1] "hello_world foo"
```

```
gsub(" ", "_", "hello world foo")          # all
```

```
#> [1] "hello_world_foo"
```

```
gsub("[0-9]", "", "abc123def456")
```

```
#> [1] "abcdef"
```

```
gsub(" +", " ", "too  many  spaces")
```

```
#> [1] "too many spaces"
```

- Swap first/last name: `sub("(\\w+) (\\w+)", "\\2, \\1", "John Smith")`
- Extract domain from email: `sub(".*@", "", "user@example.com")`
- Extract year from dates using capture group

```
sub("(\\w+) (\\w+)", "\\2, \\1",  
     c("John Smith","Jane Doe"))
```

```
#> [1] "Smith, John" "Doe, Jane"
```

```
sub(".*@", "", c("user@example.com","a@b.org"))
```

```
#> [1] "example.com" "b.org"
```

```
sub("(\\d{4})-\\d{2}-\\d{2}", "\\1",  
     c("2024-01-15","2023-12-31"))
```

```
# year
```

```
#> [1] "2024" "2023"
```



```
x <- "Phone: 02-1234-5678, Fax: 04-8765-4321".
```

- Extract all phone numbers with `gregexpr + regmatches`
- Extract first match only with `regexpr`
- Extract all words with `\\b\\w+\\b`

```
x <- "Phone: 02-1234-5678, Fax: 04-8765-4321"  
m <- gregexpr("[0-9]+-[0-9]+-[0-9]+", x)  
regmatches(x, m)
```

```
#> [[1]]  
#> [1] "02-1234-5678" "04-8765-4321"
```

```
m1 <- regexpr("[0-9]+-[0-9]+-[0-9]+", x)  
regmatches(x, m1)
```

```
#> [1] "02-1234-5678"
```

```
regmatches(x, gregexpr("\\b\\w+\\b", x))
```

```
#> [[1]]  
#> [1] "Phone" "02"    "1234"  "5678"  "Fax"   "04"  
#> [7] "8765"  "4321"  
#> ... [output truncated]
```

```
x <- "<b>bold</b> and <i>italic</i>".
```

- `gregexpr("<.*>", x) + regmatches` — what matches? (greedy)
- `gregexpr("<.*?>", x)` — what matches? (lazy)
- Explain the difference

```
x <- "<b>bold</b> and <i>italic</i>"  
regmatches(x, gregexpr("<.*>", x))           # one big match
```

```
#> [[1]]  
#> [1] "<b>bold</b> and <i>italic</i>"
```

```
regmatches(x, gregexpr("<.*?>", x))           # 4 tags
```

```
#> [[1]]  
#> [1] "<b>" "</b>" "<i>" "</i>"
```

```
# greedy: .* matches as much as possible (all the way  
# to last >); lazy: .*? matches as little as possible
```

```
x <- c("cat", "concatenate", "scat", "catalog").
```

- Starts with cat
- Ends with cat
- Exactly cat
- `c("a", "ab", "abb", "abbb")`: match `ab+` vs `ab*` vs `ab?`

```
x <- c("cat","concatenate","scat","catalog")  
grep("^cat", x, value=TRUE)
```

```
#> [1] "cat"      "catalog"
```

```
grep("cat$", x, value=TRUE)
```

```
#> [1] "cat" "scat"
```

```
grep("^cat$", x, value=TRUE)
```

```
#> [1] "cat"
```

```
y <- c("a","ab","abb","abbb")  
grep("ab+", y, value=TRUE); grep("ab*", y, value=TRUE)
```

```
#> [1] "ab" "abb" "abbb"
```

```
#> [1] "a" "ab" "abb" "abbb"
```

```
x <- c("Hello!", "123", "abc", "A1!").
```

- Contains punctuation
- All digits
- Contains only alphabetic
- All alphanumeric

```
x <- c("Hello!", "123", "abc", "A1!")  
grep("[[:punct:]]", x, value=TRUE)
```

```
#> [1] "Hello!" "A1!"
```

```
grep("^[[[:digit:]]+$", x, value=TRUE)
```

```
#> [1] "123"
```

```
grep("^[[[:alpha:]]+$", x, value=TRUE)
```

```
#> [1] "abc"
```

```
grep("^[[[:alnum:]]+$", x, value=TRUE)
```

```
#> [1] "123" "abc"
```


- Extract numbers preceded by \$: `c("$100", "200", "$300")`
- Find files NOT ending in .txt: `c("a.txt", "b.png", "c.txt", "d.jpg")`
- Explain why `perl=TRUE` is needed

```
x <- c("$100","200","$300")  
regmatches(x, gregexpr("(?<=\\$)\\d+", x, perl=TRUE))
```

```
#> [[1]]  
#> [1] "100"  
#>  
#> ... [output truncated]
```

```
y <- c("a.txt","b.png","c.txt","d.jpg")  
grep("\\.(?!txt$)", y, value=TRUE, perl=TRUE)
```

```
#> [1] "b.png" "d.jpg"
```

```
# lookahead/lookbehind are Perl-compatible regex  
# extensions not available in base (POSIX) regex
```

```
dates <- c("2024-01-15 14:30", "2023-12-31 09:00").
```

- Write pattern to capture year, month, day, hour, minute separately
- Extract with `regexec` + `regmatches`
- Build a data frame from the results

```
dates <- c("2024-01-15 14:30", "2023-12-31 09:00")
pat <- "(\\d{4})-(\\d{2})-(\\d{2}) (\\d{2}):\\d{2})"
m <- regexec(pat, dates)
parts <- regmatches(dates, m)
do.call(rbind, lapply(parts, \(p)
  setNames(p[-1], c("Y", "M", "D", "h", "m"))))
```

```
#>      Y      M      D      h      m
#> [1,] "2024" "01" "15" "14" "30"
#> [2,] "2023" "12" "31" "09" "00"
#> ... [output truncated]
```

- Write `camel_to_snake(s)` converting "camelCase" → "camel_case"
- Write `snake_to_camel(s)` converting "snake_case" → "snakeCase"
- Test both on "myHTTPRequest" / "my_http_request"

```
camel_to_snake <- function(s) tolower(gsub("([A-Z])","_\\1",s))
camel_to_snake("camelCase")
```

```
#> [1] "camel_case"
```

```
camel_to_snake("myHTTPRequest")
```

```
#> [1] "my_h_t_t_p_request"
```

```
snake_to_camel <- function(s) {
  parts <- strsplit(s, "_")[[1]]
  paste0(parts[1], paste0(toupper(substring(parts[-1],1,1)),
    substring(parts[-1],2), collapse=""))
}
snake_to_camel("my_http_request")
```

```
#> [1] "myHttpRequest"
```

```
cols <- c("First Name", "Last.Name", "Age (years)", "Income$").
```

- Replace all non-alphanumeric chars with _
- Remove leading/trailing _
- Collapse consecutive _ to single _
- Convert to lowercase

```
cols <- c("First Name","Last.Name","Age (years)","Income$")  
x <- gsub("[^a-zA-Z0-9]", "_", cols)  
x <- gsub("^_|_+$", "", x)  
x <- gsub("_+", "_", x)  
tolower(x)
```

```
#> [1] "first_name" "last_name"  "age_years"  "income"
```


Date and Time

Date Objects

```
today <- Sys.Date(); today; class(today)
```

```
#> [1] "2026-04-01"
```

```
#> [1] "Date"
```

```
as.Date('2024-01-15')
```

```
#> [1] "2024-01-15"
```

```
as.Date('15/01/2024', format = '%d/%m/%Y')
```

```
#> [1] "2024-01-15"
```

```
as.Date('Jan 15, 2024', format = '%b %d, %Y')
```

```
#> [1] NA
```

```
today + 30
```

```
#> [1] "2026-05-01"
```

```
as.Date('2024-12-31') - as.Date('2024-01-01')
```

```
#> Time difference of 365 days
```

Date Formatting

```
d <- as.Date('2024-07-15')  
format(d, '%Y'); format(d, '%B'); format(d, '%A')
```

```
#> [1] "2024"
```

```
#> [1] " "
```

```
#> [1] " "
```

```
format(d, '%d %b %Y')
```

```
#> [1] "15 7 2024"
```

```
weekdays(d); months(d); quarters(d)
```

```
#> [1] " "
```

```
#> [1] " "
```

```
#> [1] "Q3"
```

```
now <- Sys.time(); now; class(now)
```

```
#> [1] "2026-04-01 14:03:55 CST"
```

```
#> [1] "POSIXct" "POSIXt"
```

```
as.POSIXct('2024-01-15 14:30:00', tz = 'Asia/Taipei')
```

```
#> [1] "2024-01-15 14:30:00 CST"
```

```
# POSIXlt: list-based access to components
```

```
lt <- as.POSIXlt(now)
```

```
lt$hour; lt$min; lt$wday      # 0=Sunday
```

```
#> [1] 14
```

```
#> [1] 3
```

```
#> [1] 3
```

```
d1 <- as.Date('2024-01-01'); d2 <- as.Date('2024-12-31')
difftime(d2, d1, units = 'weeks')
```

#> Time difference of 52.14286 weeks

```
seq(as.Date('2024-01-01'), as.Date('2024-06-01'), by = 'month')
```

#> [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01"

#> [5] "2024-05-01" "2024-06-01"

Code	Meaning	Code	Meaning
%Y	4-digit year	%H	hour (00–23)
%m	month (01–12)	%M	minute (00–59)
%d	day (01–31)	%S	second (00–59)
%B/%b	month name/abbrev	%A/%a	weekday name/abbrev

strftime() and strptime()

```
strftime(Sys.time(), '%Y-%m-%d %H:%M:%S %Z')
```

```
#> [1] "2026-04-01 14:03:56 CST"
```

```
strptime('15-Jan-2024 14:30', format = '%d-%b-%Y %H:%M')
```

```
#> [1] NA
```

- Create today's date with `Sys.Date()`. Add 30 days
- Days between 2024-01-01 and 2024-12-31
- Generate monthly sequence Jan–Jun 2024
- Extract weekday name and month name from a date

```
today <- Sys.Date(); today + 30
```

```
#> [1] "2026-05-01"
```

```
as.Date("2024-12-31") - as.Date("2024-01-01")      # 365
```

```
#> Time difference of 365 days
```

```
seq(as.Date("2024-01-01"), as.Date("2024-06-01"),  
     by="month")
```

```
#> [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01"
```

```
#> [5] "2024-05-01" "2024-06-01"
```

```
d <- as.Date("2024-07-15")  
weekdays(d); months(d); format(d, "%A %B %d, %Y")
```

```
#> [1] " "
```

```
#> [1] " "
```

```
#> [1] "      15, 2024"
```


- Create as `POSIXct("2024-01-15 14:30:00", tz="Asia/Taipei")`
- Extract hour, minute via `POSIXlt`
- Difference between two timestamps in hours
- Format with `strftime`

```
t1 <- as.POSIXct("2024-01-15 14:30:00", tz="Asia/Taipei")  
lt <- as.POSIXlt(t1); lt$hour; lt$min
```

```
#> [1] 14
```

```
#> [1] 30
```

```
t2 <- as.POSIXct("2024-01-15 18:45:00", tz="Asia/Taipei")  
difftime(t2, t1, units="hours")
```

```
#> Time difference of 4.25 hours
```

```
strftime(t1, "%Y/%m/%d %H:%M %Z")
```

```
#> [1] "2024/01/15 14:30 CST"
```

File I/O

```
df <- read.csv('data.csv')
df <- read.csv('data.csv', header = TRUE, stringsAsFactors = FALSE)
df <- read.delim('data.tsv')           # tab-separated
df <- read.table('data.txt', header = TRUE, sep = '|')
```

```
write.csv(df, 'output.csv', row.names = FALSE)
write.table(df, 'output.tsv', sep = '\t',
            row.names = FALSE, quote = FALSE)
```

```
# save multiple R objects (binary, preserves types)
save(x, y, file = 'my_data.RData')
load('my_data.RData')

# save/load a single object
saveRDS(df, 'my_df.rds')
df <- readRDS('my_df.rds')
```

readLines, writeLines, scan

```
lines <- readLines('file.txt')
lines <- readLines('file.txt', n = 10)      # first 10 lines
writeLines(c('line 1', 'line 2'), 'out.txt')
nums  <- scan('numbers.txt')                # numeric
words <- scan('words.txt', what = '')        # character
```

readline() and interactive()

```
name <- readline(prompt = "Enter your name: ")
cat("Hello,", name, "\n")
n <- as.numeric(readline("Enter a number: "))
```

```
if (interactive()) {
  cat("Running interactively\n")
} else {
  cat("Running in batch mode\n")
}
```

- `charToRaw("Hello")` — what type?
- Convert back with `rawToChar`
- `as.integer(charToRaw("ABC"))` — what are the byte values?
- Write raw bytes to a temp file with `writeBin` and read back


```
r <- charToRaw("Hello"); r; typeof(r)           # raw
```

```
#> [1] 48 65 6c 6c 6f
```

```
#> [1] "raw"
```

```
rawToChar(r)                                     # "Hello"
```

```
#> [1] "Hello"
```

```
as.integer(charToRaw("ABC"))                     # 65 66 67
```

```
#> [1] 65 66 67
```

```
tmp <- tempfile()
writeBin(charToRaw("test123"), tmp)
rawToChar(readBin(tmp, "raw", 20))
```

```
#> [1] "test123"
```

- Write a function that uses `readline` to ask for user's name (use `eval=False`)
- Use `interactive()` to branch: `interactive` → `readline`, `batch` → `default`
- Why does `readline` always return character?

```
ask_name <- function() {  
  if (interactive()) {  
    name <- readline("Your name: ")  
  } else {  
    name <- "default_user"  
  }  
  cat("Hello,", name, "\n")  
}  
# readline reads raw text from console; R has no way  
# to know the intended type, so it always returns character
```

Error Handling

tryCatch() — Structured Error Handling

```
# note: tryCatch catches the FIRST condition raised
result <- tryCatch({
  log('a')          # error: non-numeric argument
}, warning = function(w) {
  cat("Warning:", conditionMessage(w), "\n"); NA
}, error = function(e) {
  cat("Error:", conditionMessage(e), "\n"); NA
}, finally = {
  cat("(cleanup code runs always)\n")
})
```

```
#> Error: non-numeric argument to mathematical function
#> (cleanup code runs always)
```

```
result
```

```
#> [1] NA
```

```
result <- try(log('a'), silent = TRUE)
inherits(result, 'try-error')
```

```
#> [1] TRUE
```

```
# useful in loops: skip failing inputs
inputs <- list(1, 'a', 4, NULL, 9)
results <- list()
for (i in seq_along(inputs)) {
  results[[i]] <- try(sqrt(inputs[[i]]), silent = TRUE)
}
Filter(\(r) !inherits(r, 'try-error'), results)
```

```
#> [[1]]
```

```
#> [1] 1
```

```
#>
```

```
#> ... [output truncated]
```

Common Error-Handling Idioms

```
# silently return NULL on failure
safe_parse <- function(x) {
  tryCatch(as.numeric(x), warning = function(w) NULL,
           error = function(e) NULL)
}
safe_parse("3.14"); safe_parse("abc"); safe_parse(NULL)
```

```
#> [1] 3.14
```

```
#> NULL
```

```
#> numeric(0)
```

```
# wrap risky operations with a default value
safe_log <- function(x, default = NA) {
  tryCatch(log(x), error = function(e) default)
}
sapply(list(10, -1, 'a', NULL), safe_log)
```

```
#> [1] 2.302585      NaN      NA      NA
```

```
stop(), warning(), message()
```

```
safe_div <- function(a, b) {  
  if (b == 0) stop("Division by zero!")  
  a / b  
}  
safe_div(10, 2); safe_div(10, 0)
```

```
#> [1] 5
```

```
#> Error in safe_div(10, 0): Division by zero!
```

```
check_val <- function(x) {  
  if (x < 0) warning("Negative value!")  
  sqrt(abs(x))  
}  
check_val(-4)
```

```
#> [1] 2
```



```
my_mean <- function(x) {  
  stopifnot(is.numeric(x), length(x) > 0)  
  sum(x) / length(x)  
}  
my_mean(1:5)
```

```
#> [1] 3
```

```
my_mean('hello')
```

```
#> Error in my_mean("hello"): is.numeric(x) is not TRUE
```

- Run `system2("echo", args=c("hello","world"))`
- Capture output with `stdout=TRUE`
- Run in background with `wait=FALSE`
- Cross-platform check with `.Platform$OS.type`

```
system2("echo", args=c("hello","world"))
out <- system2("date", stdout=TRUE); out
system2("sleep", args="1", wait=FALSE)
if (.Platform$OS.type == "unix") {
  system2("uname", args="-a", stdout=TRUE)
}
```

Useful Utilities

```
do.call()
```

```
dfs <- list(  
  data.frame(x = 1:2, y = 3:4),  
  data.frame(x = 5:6, y = 7:8),  
  data.frame(x = 9:10, y = 11:12))  
do.call(rbind, dfs)
```

```
#>      x  y  
#> 1   1  3  
#> 2   2  4  
#> ... [output truncated]
```

```
args <- list(1:10, na.rm = TRUE)  
do.call(mean, args)
```

```
#> [1] 5.5
```

```
f <- function(x, y) {  
  if (x > y) x - y else x + y  
}  
f_vec <- Vectorize(f)  
f_vec(1:3, 2)
```

```
#> [1] 3 4 1
```

invisible() — Silent Return

```
# invisible: returns value without printing
f <- function(x) invisible(x^2)
f(5)                # nothing printed
y_val <- f(5); y_val # but value is captured
```

```
#> [1] 25
```

```
# common pattern: side-effect functions return invisible(NULL)
my_print <- function(x) {
  cat("Value:", x, "\n")
  invisible(NULL)
}
result <- my_print(42); is.null(result)
```

```
#> Value: 42
```

```
#> [1] TRUE
```

on.exit() — Guaranteed Cleanup

```
safe_read <- function(path) {  
  con <- file(path, 'r')  
  on.exit(close(con))      # guaranteed cleanup  
  readLines(con)  
}
```

```
# add = TRUE: stack multiple on.exit calls  
process <- function(path) {  
  tmp <- tempfile()  
  on.exit(unlink(tmp), add = TRUE)      # cleanup 1  
  con <- file(path, 'r')  
  on.exit(close(con), add = TRUE)      # cleanup 2  
}
```


Benchmarking: `system.time()` and `proc.time()`

```
system.time({  
  x <- matrix(rnorm(1e6), 1000, 1000)  
  y_mat <- crossprod(x)  
})
```

```
#>      user  system elapsed  
#>  0.103    0.071    0.038
```

```
pt <- proc.time()  
x <- sum(1:1e7)  
proc.time() - pt
```

```
#>      user  system elapsed  
#>  0.000    0.018    0.001
```

```
system2("echo", args = c("hello", "world"))
out <- system2("date", stdout = TRUE); cat(out, "\n")
system2("sleep", args = "5", wait = FALSE)      # background
```

```
# cross-platform branching
if (.Platform$OS.type == "windows") {
  system2("taskkill", args = c("/F", "/IM", "myapp.exe"),
          stdout = NULL, stderr = NULL)
} else {
  system2("pkill", args = "myapp",
          stdout = NULL, stderr = NULL)
}
```

Advanced Topics

```
person <- list(name = 'Alice', age = 30)
class(person) <- 'Person'
print.Person <- function(x, ...) {
  cat(sprintf("Person: %s (age %d)\n", x$name, x$age))
}
person
```

```
#> Person: Alice (age 30)
```

Formula Objects

```
f <- y ~ x1 + x2  
class(f); typeof(f); all.vars(f)
```

```
#> [1] "formula"
```

```
#> [1] "language"
```

```
#> [1] "y" "x1" "x2"
```

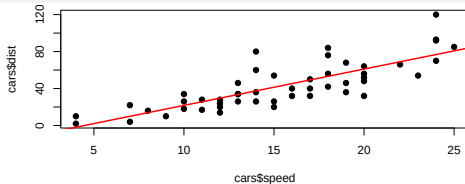
```
y ~ x           # simple regression  
y ~ x1 + x2     # multiple regression  
y ~ x1 * x2     # with interaction (= x1 + x2 + x1:x2)  
y ~ .           # all other columns  
y ~ poly(x, 3)  # polynomial  
y ~ log(x)      # transformed
```

Linear Regression Example

```
model <- lm(dist ~ speed, data = cars)
model$coefficients
```

```
#> (Intercept)      speed
#>  -17.579095    3.932409
```

```
plot(cars$speed, cars$dist, pch = 19)
abline(model, col = 'red', lwd = 2)
```



Random Number Generation

```
set.seed(42)  
runif(5); rnorm(5, mean = 0, sd = 1)
```

```
#> [1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455  
#> [1] 0.04788474 -1.10459944 0.53902380 0.58020632  
#> [5] -0.65750284
```

```
rbinom(10, size = 1, prob = 0.5)
```

```
#> [1] 1 1 0 0 1 1 0 1 1 0
```

```
rpois(10, lambda = 3)
```

```
#> [1] 3 2 5 3 5 4 4 2 4 0
```

```
rexp(5, rate = 1)
```

```
#> [1] 0.6658322 4.9959685 0.2235573 1.2113841 0.7190924
```

- Create S3 class Circle: `list(radius=r)` with `class="Circle"`
- Write `print.Circle` and `area.Circle`
- Create instance; dispatch `print` and `area`


```
Circle <- function(r) structure(list(radius=r), class="Circle")
print.Circle <- function(x, ...)  
  cat(sprintf("Circle(r=%.2f)\n", x$radius))  
area <- function(x, ...) UseMethod("area")  
area.Circle <- function(x, ...) pi * x$radius^2  
c1 <- Circle(5); c1
```

```
#> Circle(r=5.00)
```

```
area(c1)
```

```
#> [1] 78.53982
```

- `f <- y ~ x`; show `class()`, `typeof()`, `all.vars()`
- Fit `lm(dist ~ speed, data=cars)`. Extract coefficients
- Plot data with regression line

```
f <- y ~ x; class(f); typeof(f); all.vars(f)
```

```
#> [1] "formula"
```

```
#> [1] "language"
```

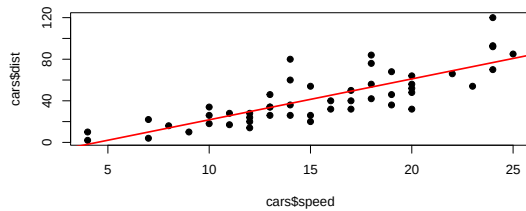
```
#> [1] "y" "x"
```

```
model <- lm(dist ~ speed, data=cars)  
coef(model)
```

```
#> (Intercept)      speed
```

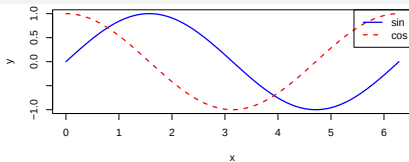
```
#> -17.579095    3.932409
```

```
#> ... [output truncated]
```



- Plot $\sin(x)$ and $\cos(x)$ for x from 0 to 2π with legend
- `par(mfrow=c(1,2))`: histogram and boxplot side by side
- Add annotation with `text()` and horizontal line with `abline(h=...)`

```
x <- seq(0, 2*pi, length=100)
plot(x, sin(x), type="l", col="blue", lwd=2, ylab="y")
lines(x, cos(x), col="red", lwd=2, lty=2)
legend("topright", c("sin","cos"), col=c("blue","red"),
      lty=1:2, lwd=2)
```



- `set.seed(42)` then generate 5 uniform, 5 normal, 10 Bernoulli
- Simulate 1000 dice rolls and tabulate
- Estimate π via Monte Carlo: sample (x,y) uniform in $[0,1]^2$, count proportion inside unit circle quadrant

```
set.seed(42)
runif(5); rnorm(5); rbinom(10, 1, 0.5)
```

```
#> [1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455
```

```
#> [1] 0.04788474 -1.10459944 0.53902380 0.58020632
```

```
#> [5] -0.65750284
```

```
#> [1] 1 1 0 0 1 1 0 1 1 0
```

```
table(sample(1:6, 1000, replace=TRUE))
```

```
#>
```

```
#> 1 2 3 4 5 6
```

```
#> 169 191 163 157 153 167
```

```
#> ... [output truncated]
```

```
n <- 1e5; x <- runif(n); y <- runif(n)
4 * mean(x^2 + y^2 <= 1) #
```

```
#> [1] 3.14176
```


- Save current digits option, set to 15, print pi, restore
- Time `crossprod(x)` for `x <- matrix(rnorm(1e6),1000)` with `system.time`
- Use `proc.time` for manual timing of `sum(1:1e7)`

```
old <- options(digits=15); pi
```

```
#> [1] 3.14159265358979
```

```
options(old)
system.time({
  x <- matrix(rnorm(1e6), 1000, 1000)
  y <- crossprod(x)
})
```

```
#>      user  system elapsed
#>  0.088    0.073    0.035
```

```
pt <- proc.time(); s <- sum(1:1e7); proc.time()-pt
```

```
#>      user  system elapsed
#>  0.000    0.014    0.001
```

Write `binary_search(x, target)` that returns the index of `target` in sorted vector `x`, or `NA` if not found. Use a `while` loop with `lo/hi` pointers.

- Test: `binary_search(c(2,5,8,12,16,23,38,56,72,91), 23) → 6`
- Test: `binary_search(c(2,5,8,12,16,23,38,56,72,91), 10) → NA`
- What is the time complexity?

```
binary_search <- function(x, target) {  
  lo <- 1; hi <- length(x)  
  while (lo <= hi) {  
    mid <- (lo + hi) %/% 2  
    if (x[mid] == target) return(mid)  
    if (x[mid] < target) lo <- mid + 1 else hi <- mid - 1  
  }  
  NA  
}  
binary_search(c(2,5,8,12,16,23,38,56,72,91), 23)
```

```
#> [1] 6
```

```
binary_search(c(2,5,8,12,16,23,38,56,72,91), 10)
```

```
#> [1] NA
```

```
# O(log n) time complexity
```

Implement `merge_sort(x)` recursively:

- Base case: length 1 $\rightarrow$ return as-is
- Split in half, sort each half, merge
- Write `merge_two(a, b)` helper that merges two sorted vectors
- Test: `merge_sort(c(38, 27, 43, 3, 9, 82, 10))`

```
merge_two <- function(a, b) {  
  res <- numeric(length(a) + length(b))  
  i <- j <- k <- 1  
  while (i <= length(a) && j <= length(b)) {  
    if (a[i] <= b[j]) { res[k] <- a[i]; i <- i+1 }  
    else { res[k] <- b[j]; j <- j+1 }  
    k <- k + 1  
  }  
  if (i <= length(a)) res[k:(k+length(a)-i)] <- a[i:length(a)]  
  if (j <= length(b)) res[k:(k+length(b)-j)] <- b[j:length(b)]  
  res  
}
```

```
merge_sort <- function(x) {  
  if (length(x) <= 1) return(x)  
  mid <- length(x) %/% 2  
  merge_two(merge_sort(x[1:mid]),  
            merge_sort(x[(mid+1):length(x)]))  
}  
merge_sort(c(38, 27, 43, 3, 9, 82, 10))  
  
#> [1] 3 9 10 27 38 43 82
```

Write `compose(...)` that takes n functions and returns their composition: $\text{compose}(f, g, h)(x) = f(g(h(x)))$.

- Test: `compose(sqrt, abs, sum)(c(-4, -5, -7)) → 4`
- Compose `toupper`, `trimws`, `paste0` to build a string cleaner
- Show it works with `Reduce`


```
compose <- function(...) {  
  fns <- rev(list(...))  
  function(x) Reduce(\(v, f) f(v), fns, init = x)  
}  
compose(sqrt, abs, sum)(c(-4, -5, -7)) # sqrt(16) = 4
```

```
#> [1] 4
```

```
clean <- compose(toupper, trimws)  
clean(" hello world ")
```

```
#> [1] "HELLO WORLD"
```

Write `curry(f, ...)` that partially applies arguments to `f`, returning a new function that accepts the remaining arguments.

- `add <- function(a, b) a + b; add5 <- curry(add, 5); add5(3) → 8`
- `curry(paste, sep = "-")("a", "b", "c") → "a-b-c"`
- Use currying to create `log2 <- curry(log, base = 2)`

```
curry <- function(f, ...) {  
  captured <- list(...)  
  function(...) do.call(f, c(captured, list(...)))  
}  
add <- function(a, b) a + b  
add5 <- curry(add, 5); add5(3)
```

```
#> [1] 8
```

```
dash_paste <- curry(paste, sep = "-")  
dash_paste("a", "b", "c")
```

```
#> [1] "a-b-c"
```

```
log2 <- curry(log, base = 2); log2(8)
```

```
#> [1] 3
```

Implement a stack using closures with operations: `push(x)`, `pop()`, `peek()`, `size()`, `is_empty()`.

- Push 10, 20, 30; peek $\rightarrow$ 30; pop $\rightarrow$ 30; size $\rightarrow$ 2
- Handle `pop()` on empty stack gracefully (return NULL + warning)
- Use the stack to reverse a string

```
make_stack <- function() {  
  items <- list()  
  list(  
    push = function(x) items[[length(items)+1]] <- x,  
    pop = function() {  
      if (length(items) == 0) {  
        warning("empty stack"); return(NULL) }  
      val <- items[[length(items)]]  
      items[[length(items)]] <- NULL; val },  
    peek = function() items[[length(items)]],  
    size = function() length(items),  
    is_empty = function() length(items) == 0)  
}
```

```
s <- make_stack()
s$push(10); s$push(20); s$push(30)
s$peek(); s$pop(); s$size()
# reverse string
chars <- strsplit("Hello", "")[[1]]
rs <- make_stack()
for (ch in chars) rs$push(ch)
paste0(replicate(rs$size(), rs$pop()), collapse = "")
```

```
#> [1] 30
```

```
#> [1] 30
```

```
#> [1] 2
```

```
#> [1] "olleH"
```

Implement a queue (FIFO) using closures: `enqueue(x)`, `dequeue()`, `front()`, `size()`.

- Enqueue "A", "B", "C"; dequeue → "A"; front → "B"
- Use the queue to implement breadth-first traversal of a nested list
- What is the time complexity of your dequeue? How could it be improved?

```
make_queue <- function() {  
  items <- list()  
  list(  
    enqueue = function(x)  
      items[[length(items)+1]] <- x,  
    dequeue = function() {  
      val <- items[[1]]; items[[1]] <- NULL; val },  
    front = function() items[[1]],  
    size = function() length(items),  
    is_empty = function() length(items) == 0)  
}
```



```
q <- make_queue()
q$enqueue("A"); q$enqueue("B"); q$enqueue("C")
q$dequeue(); q$front(); q$size()
# dequeue is O(n) due to list shifting
# improvement: use two-stack or index-based approach

#> [1] "A"
#> [1] "B"
#> [1] 2
```

Ex 147: Matrix Chain Multiplication Order

Given dimensions of n matrices, find minimum scalar multiplications using dynamic programming. Matrices A_i has dimension $p_{i-1} \times p_i$.

- Write `mcm(p)` for dimension vector `p` of length $n + 1$
- Test: `mcm(c(10, 30, 5, 60))` $\rightarrow$ 4500
- Return both the minimum cost and the optimal split

```
mcm <- function(p) {  
  n <- length(p) - 1  
  m <- matrix(0, n, n)  
  for (len in 2:n)  
    for (i in 1:(n - len + 1)) {  
      j <- i + len - 1; m[i,j] <- Inf  
      for (k in i:(j-1))  
        m[i,j] <- min(m[i,j],  
          m[i,k] + m[k+1,j] + p[i]*p[k+1]*p[j+1])  
    }  
  m[1, n]  
}
```

```
mcm(c(10, 30, 5, 60))      # 4500
```

```
#> [1] 4500
```

```
mcm(c(40, 20, 30, 10, 30)) # 26000
```

```
#> [1] 26000
```

Write `bisection(f, a, b, tol=1e-8)` that finds a root of f in $[a, b]$.

- Require $f(a) \cdot f(b) < 0$ (else stop)
- Test: root of $x^3 - x - 2$ near $[1, 2]$
- Compare with `uniroot` result

```
bisection <- function(f, a, b, tol = 1e-8) {  
  stopifnot(f(a) * f(b) < 0)  
  n <- 0  
  while (b - a > tol) {  
    mid <- (a + b) / 2; n <- n + 1  
    if (f(mid) == 0) return(list(root=mid, iter=n))  
    if (f(a) * f(mid) < 0) b <- mid else a <- mid  
  }  
  list(root = (a + b) / 2, iter = n)  
}
```

```
f <- function(x) x^3 - x - 2  
bisection(f, 1, 2)
```

```
#> $root  
#> [1] 1.52138  
#>  
#> ... [output truncated]
```

```
uniroot(f, c(1, 2))$root
```

```
#> [1] 1.521386
```

Write `my_rle(x)` that returns a list with lengths and values (like `rle()`).

- Test: `my_rle(c(1,1,1,2,2,3,3,3,3,1))` → lengths: 3,2,4,1; values: 1,2,3,1
- Write `my_inverse_rle(r)` that reconstructs the original vector
- Verify: `my_inverse_rle(my_rle(x))` equals `x` for various inputs


```

my_rle <- function(x) {
  n <- length(x); vals <- x[1]; lens <- 1L; k <- 1
  for (i in 2:n) {
    if (x[i] == vals[k]) lens[k] <- lens[k] + 1L
    else { k <- k+1; vals[k] <- x[i]; lens[k] <- 1L }
  }
  list(lengths = lens, values = vals)
}

my_inverse_rle <- function(r) rep(r$values, r$lengths)
x <- c(1,1,1,2,2,3,3,3,3,1)
my_rle(x)

```

```

#> $lengths
#> [1] 3 2 4 1
#> ... [output truncated]

```

```

identical(my_inverse_rle(my_rle(x)), x)

```

```

#> [1] TRUE

```

Write `env_info(fn)` that prints the enclosing environment chain of function `fn` (up to `.GlobalEnv` or `emptyenv()`).

- Test on a closure created by `make_counter()`
- Show `environment(fn)` differs from `parent.env(environment(fn))`
- List all objects in the closure's enclosing environment using `ls(envir = ...)`

```
env_info <- function(fn) {  
  e <- environment(fn)  
  while (!identical(e, emptyenv())) {  
    cat(environmentName(e), ": ",  
        paste(ls(e), collapse=" ", "\n"))  
    e <- parent.env(e)  
    if (identical(e, globalenv())) {  
      cat("(global)\n"); break }  
  }  
}
```

```
make_counter <- function() {  
  n <- 0  
  list(inc = function() { n <- n+1; n })  
}  
ctr <- make_counter()  
env_info(ctr$inc)  
ls(envir = environment(ctr$inc))
```

```
#> : n  
#> (global)  
#> [1] "n"
```

R has no TCO. Implement a trampoline: a loop that keeps calling thunks (zero-arg functions) until a non-function result is returned.

- Write `trampoline(f, ...)`
- Rewrite `factorial` in trampolined form
- Show it can handle `tramp_fact(10000)` without stack overflow

```
trampoline <- function(f, ...) {  
  result <- f(...)  
  while (is.function(result)) result <- result()  
  result  
}  
  
tramp_fact <- function(n, acc = 1) {  
  if (n <= 1) return(acc)  
  function() tramp_fact(n - 1, acc * n)  
}  
  
trampoline(tramp_fact, 10)    # 3628800
```

```
#> [1] 3628800
```

```
# trampoline(tramp_fact, 10000) works without stack overflow  
# (result is Inf due to numeric overflow, but no crash)
```

Write `retry(fn, max_attempts = 3, base_delay = 0.1)` that:

- Calls `fn()`; on error, waits `base_delay * 2^(attempt-1)` seconds and retries
- Returns the result on success, or re-throws the last error after `max_attempts`
- Test with a function that fails randomly: `function() { if (runif(1) < 0.7) stop("fail"); 42 }`

```
retry <- function(fn, max_attempts = 3, base_delay = 0.1) {  
  for (i in seq_len(max_attempts)) {  
    result <- tryCatch(fn(), error = function(e) e)  
    if (!inherits(result, "error")) return(result)  
    if (i < max_attempts)  
      Sys.sleep(base_delay * 2^(i - 1))  
  }  
  stop(result$message)  
}  
set.seed(42)  
flaky <- function() { if (runif(1) < 0.5) stop("fail"); 42 }  
retry(flaky, max_attempts = 5)
```

```
#> [1] 42
```


Implement a singly linked list using nested lists: `node(value, next_node)`.

- Write `ll_push(ll, val)`, `ll_to_vector(ll)`, `ll_length(ll)`, `ll_reverse(ll)`
- Build list $1 \rightarrow 2 \rightarrow 3$, reverse it, convert to vector
- Why are linked lists rarely used in R?

```
node <- function(val, nxt = NULL) list(val = val, nxt = nxt)
ll_push <- function(ll, val) node(val, ll)
ll_length <- function(ll) {
  n <- 0; while (!is.null(ll)) { n <- n+1; ll <- ll$nxt }; n
}
ll_to_vector <- function(ll) {
  v <- c(); while(!is.null(ll)) { v<-c(v,ll$val); ll<-ll$nxt }; v
}
```

```

ll_reverse <- function(ll) {
  prev <- NULL
  while (!is.null(ll)) {
    nxt <- ll$nxt; ll$nxt <- prev; prev <- ll; ll <- nxt }
  prev
}

ll <- Reduce\(acc, x) ll_push(acc, x), 3:1, init = NULL)
ll_to_vector(ll)           # 1 2 3
ll_to_vector(ll_reverse(ll)) # 3 2 1
# R copies on modify; linked lists lose their O(1) advantage

```

```
#> [1] 1 2 3
```

```
#> [1] 3 2 1
```

Write `make_validator(...)` that takes named validation rules and returns a function checking all rules.

- Rules: `list(name = is.character, age = \(x) is.numeric(x) && x > 0)`
- Validator returns `TRUE` if all pass, else a character vector of failures
- Test with valid and invalid inputs

```
make_validator <- function(rules) {  
  function(data) {  
    fails <- character()  
    for (nm in names(rules)) {  
      if (!nm %in% names(data))  
        fails <- c(fails, paste(nm, "missing"))  
      else if (!rules[[nm]](data[[nm]]))  
        fails <- c(fails, paste(nm, "invalid"))  
    }  
    if (length(fails) == 0) TRUE else fails  
  }  
}
```

```
v <- make_validator(list(name = is.character,  
  age = \(x) is.numeric(x) && x > 0))  
v(list(name = "Alice", age = 30))      # TRUE  
v(list(name = 42, age = -1))           # failures  
v(list(name = "Bob"))                  # age missing
```

```
#> [1] TRUE
```

```
#> [1] "name invalid" "age invalid"
```

```
#> [1] "age missing"
```

Write `flatten(x)` that recursively flattens arbitrarily nested lists into a single flat list.

- Test: `flatten(list(1, list(2, list(3, 4)), 5)) → list(1, 2, 3, 4, 5)`
- Compare with `rapply(x, identity, how = "unlist")`
- Handle mixed types: `list("a", list(1, list(TRUE)))`

```
flatten <- function(x) {  
  if (!is.list(x)) return(list(x))  
  result <- list()  
  for (item in x) result <- c(result, flatten(item))  
  result  
}  
str(flatten(list(1, list(2, list(3, 4)), 5)))
```

#> List of 5

#> ... [output truncated]

```
rapply(list(1, list(2, list(3, 4)), 5), identity,  
       how = "unlist")
```

#> [1] 1 2 3 4 5

#> ... [output truncated]

```
str(flatten(list("a", list(1, list(TRUE)))))
```

#> List of 3

#> ... [output truncated]

Write `memoize(f)` that wraps any single-argument function with a hash-map cache.

- Test with expensive `slow_square <- function(x) { Sys.sleep(0.1); x^2 }`
- Show second call is instant
- Add `cache_info()` method returning hit/miss counts

```
memoize <- function(f) {  
  cache <- new.env(parent = emptyenv())  
  hits <- 0L; misses <- 0L  
  wrapped <- function(x) {  
    key <- as.character(x)  
    if (exists(key, envir = cache)) {  
      hits <- hits + 1L; return(get(key, envir = cache))  
    }  
    misses <- misses + 1L
```

```

    val <- f(x); assign(key, val, envir = cache); val
  }
  attr(wrapped, "cache_info") <-
    function() c(hits = hits, misses = misses)
  wrapped
}
sq <- memoize(\(x) x^2)
sq(5); sq(5); sq(3)
attr(sq, "cache_info")()      # hits:1, misses:2

```

```
#> [1] 25
```

```
#> [1] 25
```

```
#> [1] 9
```

```
#>   hits misses
```

```
#>     1      2
```

```
#> ... [output truncated]
```

Implement a finite state machine for a turnstile: states {locked, unlocked}, events {coin, push}.

- coin in locked $\rightarrow$ unlocked; push in locked $\rightarrow$ locked (+ “blocked” message)
- coin in unlocked $\rightarrow$ unlocked (+ “thank you”); push $\rightarrow$ locked (+ “open”)
- Use closure to track state. Return a list with event(e) and state() methods

```
make_turnstile <- function() {  
  st <- "locked"  
  event <- function(e) {  
    msg <- if (st == "locked" && e == "coin") {  
      st <<- "unlocked"; "unlocked"  
    } else if (st == "locked" && e == "push") {  
      "blocked"  
    } else if (st == "unlocked" && e == "push") {
```

```
      st <- "locked"; "open"
    } else { "thank you" }
    msg
  }
  list(event = event, state = function() st)
}
ts <- make_turnstile()
ts$event("push")           # blocked
ts$event("coin"); ts$state() # unlocked
```

```
#> [1] "blocked"
```

```
#> [1] "unlocked"
```

```
#> [1] "unlocked"
```

Write functions to rotate an $n \times n$ matrix:

- `rotate_90(m)`: 90° clockwise
- `rotate_180(m)`: 180°
- `rotate_270(m)`: 270° clockwise (= 90° counter-clockwise)
- Verify: `rotate_90` applied 4 times returns the original

```
rotate_90 <- function(m) t(m)[, ncol(m):1]
rotate_180 <- function(m) m[nrow(m):1, ncol(m):1]
rotate_270 <- function(m) t(m)[nrow(m):1, ]
m <- matrix(1:9, 3, 3)
m
```

```
#>      [,1] [,2] [,3]
#> [1,]     1     4     7
#> ... [output truncated]
```

```
rotate_90(m)
```

```
#>      [,1] [,2] [,3]
#> [1,]     3     2     1
#> ... [output truncated]
```

```
identical(Reduce\(x, f) f(x), rep(list(rotate_90), 4),
          init = m), m)    # TRUE
```

```
#> [1] TRUE
```


Write `hanoi(n, from="A", to="C", aux="B")` that prints all moves.

- Test `hanoi(3)` — should produce 7 moves
- Count total moves for n disks (verify: $2^n - 1$)
- Return moves as a data frame with columns `disk`, `from`, `to`

```
hanoi <- function(n, from="A", to="C", aux="B") {  
  moves <- data.frame(disk=integer(), from=character(),  
                      to=character(), stringsAsFactors=FALSE)  
  solve <- function(n, from, to, aux) {  
    if (n == 0) return()  
    solve(n-1, from, aux, to)  
    moves[nrow(moves)+1,] <- list(n, from, to)  
    solve(n-1, aux, to, from)  
  }  
  solve(n, from, to, aux); moves  
}
```

```
h <- hanoi(3); cat("Moves:", nrow(h), "\n") # 7 = 23-1
```

```
#> Moves: 7
```

```
h
```

```
#>   disk from to  
#> 1     1     A  C  
#> 2     2     A  B  
#> ... [output truncated]
```

Write `reduce_right(f, x, init)` that folds from the right: $f(x_1, f(x_2, f(x_3, \text{init})))$.

- Test: `reduce_right("-", 1:4, 0)` vs `Reduce("-", 1:4, 0)`
- Write `scan_left(f, x, init)` that returns all intermediate results (like `Reduce` with `accumulate=TRUE`)
- Use `scan_left` to compute running balance from transactions

```
reduce_right <- function(f, x, init) {  
  Reduce(f, rev(x), init, right = TRUE)  
}  
Reduce("-", 1:4, 0)                # (((0-1)-2)-3)-4)
```

```
#> [1] -10
```

```
reduce_right("-", 1:4, 0)          # 1-(2-(3-(4-0)))
```

```
#> [1] 2
```

```
scan_left <- function(f, x, init)  
  Reduce(f, x, init, accumulate = TRUE)  
txns <- c(100, -30, 50, -20, -10)  
scan_left("+", txns, 1000)         # running balance
```

```
#> [1] 1000 1100 1070 1120 1100 1090
```

Write `make_pipeline(...)` that accepts functions and returns a pipeline function. Each step is named for debugging.

- `pipe <- make_pipeline(abs, sqrt, round); pipe(-16) → 4`
- Add `describe()` method listing all steps
- Add `remove_step(name)` and `add_step(fn, name)` for dynamic modification

```
make_pipeline <- function(...) {  
  steps <- list(...)  
  nms <- vapply(substitute(list(...))[-1], deparse,  
               character(1))  
  names(steps) <- nms  
  pipe <- function(x) Reduce(\(v, f) f(v), steps, init=x)  
  pipe_env <- environment(pipe)  
  attr(pipe, "describe") <- function()  
    cat("Steps:", paste(names(steps), collapse=" -> "), "\n")  
  attr(pipe, "steps") <- function() names(steps)  
  pipe  
}
```

```
p <- make_pipeline(abs, sqrt, round)
p(-16)
attr(p, "describe")()
```

```
#> [1] 4
```

```
#> Steps: abs -> sqrt -> round
```


Write `safe(fn)` that wraps any function: on success returns `list(ok = TRUE, value = result)`, on error returns `list(ok = FALSE, error = msg)`.

- Chain safe operations: `safe(log)` then `safe(sqrt)`
- Write `and_then(result, fn)` that applies `safe(fn)` only if `result$ok`
- Build: `safe(as.numeric)("3.14") |> and_then(log) |> and_then(sqrt)`

```
safe <- function(fn) {  
  function(...) tryCatch(  
    list(ok = TRUE, value = fn(...)),  
    error = function(e) list(ok = FALSE, error = e$message))  
  }  
  and_then <- function(result, fn) {  
    if (!result$ok) return(result); safe(fn)(result$value)  
  }  
  safe(as.numeric)("3.14") |> and_then(log) |> and_then(sqrt)
```

```
#> $ok  
#> ... [output truncated]
```

```
safe(as.numeric)("abc") |> and_then(log) |> and_then(sqrt)
```

```
#> $ok  
#> ... [output truncated]
```

Write three accumulator closures:

- `make_running_mean()`: accepts new values, always returns current mean
- `make_running_sd()`: online standard deviation (Welford's algorithm)
- `make_ema(alpha)`: exponential moving average

Test all three with the stream `c(10, 12, 8, 15, 11, 9, 13)`.

```
make_running_mean <- function() {  
  total <- 0; n <- 0  
  function(x) { total <-< total + x; n <-< n + 1; total/n }  
}  
make_ema <- function(alpha = 0.3) {  
  val <- NULL  
  function(x) { val <-< if (is.null(val)) x  
    else alpha*x + (1-alpha)*val; val }  
}  
rm <- make_running_mean(); ema <- make_ema(0.3)  
stream <- c(10, 12, 8, 15, 11, 9, 13)  
round(sapply(stream, rm), 2)
```

```
#> [1] 10.00 11.00 10.00 11.25 11.20 10.83 11.14
```

```
round(sapply(stream, ema), 2)
```

```
#> [1] 10.00 10.60 9.82 11.37 11.26 10.58 11.31
```

Implement a trie for string lookup using nested environments:

- `trie_insert(trie, word)`: insert a word
- `trie_search(trie, word)`: exact match $\rightarrow$ TRUE/FALSE
- `trie_starts_with(trie, prefix)`: prefix match $\rightarrow$ TRUE/FALSE
- Test with words: "apple", "app", "application", "bat"

```
make_trie <- function() new.env(parent = emptyenv())
trie_insert <- function(trie, word) {
  node <- trie
  for (ch in strsplit(word, "")[[1]]) {
    if (!exists(ch, envir=node))
      assign(ch, new.env(parent=emptyenv()), envir=node)
    node <- get(ch, envir = node) }
  assign("$end", TRUE, envir = node)
}
```

```

trie_search <- function(trie, word) {
  node <- trie
  for (ch in strsplit(word, "")[[1]]) {
    if (!exists(ch, envir=node)) return(FALSE)
    node <- get(ch, envir=node) }
  isTRUE(tryCatch(get("$end", envir=node), error=\(e) FALSE))
}

tr <- make_trie()
for (w in c("apple", "app", "bat")) trie_insert(tr, w)
c(app=trie_search(tr, "app"), ap=trie_search(tr, "ap"))

```

```

#>   app   ap
#> TRUE FALSE
#> ... [output truncated]

```

Implement `my_map(x, f)`, `my_filter(x, pred)`, `my_reduce(x, f, init)` without using any apply functions or `Reduce`.

- Use only for loops
- Test: pipe `1:20` through `filter(even) → map(square) → reduce(sum)`
- Compare results with `sum(Filter(\(x) x%%2==0, 1:20)^2)`


```
my_map <- function(x, f) {  
  out <- vector("list", length(x))  
  for (i in seq_along(x)) out[[i]] <- f(x[[i]])  
  out  
}
```

```

my_filter <- function(x, pred) {
  out <- list()
  for (item in x) if (pred(item)) out[[length(out)+1]] <- item
  out
}

my_reduce <- function(x, f, init) {
  acc <- init; for (item in x) acc <- f(acc, item); acc
}

result <- my_filter(1:20, \(x) x %% 2 == 0) |>
  my_map(\(x) x^2) |> my_reduce(`+`, 0)
result  # 2^2 + 4^2 + ... + 20^2 = 1540
sum(Filter(\(x) x%%2==0, 1:20)^2)  # same

```

```
#> [1] 1540
```

```
#> [1] 1540
```

Write `calc(expr)` that evaluates simple arithmetic strings: `"3 + 5 * 2" → 13`.

- Support `+`, `-`, `*`, `/` with standard precedence
- Handle parentheses: `"(3 + 5) * 2" → 16`
- Hint: tokenize with `strsplit` and `gsub`, then use recursive descent or parse

```
calc <- function(expr) {  
  # Safe: only allow digits, operators, parens, spaces, dots  
  stopifnot(grepl("[0-9+\\-*/().^\\s]+$", expr, perl=TRUE))  
  eval(parse(text = expr))  
}  
calc("3 + 5 * 2")          # 13
```

```
#> [1] 13
```

```
calc("(3 + 5) * 2")        # 16
```

```
#> [1] 16
```

```
calc("10 / 3")             # 3.333...
```

```
#> [1] 3.333333
```

```
calc("2^10")               # 1024
```

```
#> [1] 1024
```

Without using any packages, write:

- `my_select(df, ...)`: select columns by name (like `dplyr::select`)
- `my_mutate(df, ...)`: add/modify columns (like `dplyr::mutate`)
- `my_summarize(df, by, ...)`: group-by summarize
- Test all on `mtcars`

```

my_select <- function(df, ...) {
  cols <- as.character(substitute(list(...)))[-1]
  df[, cols, drop = FALSE]
}

my_mutate <- function(df, ...) {
  exprs <- as.list(substitute(list(...)))[-1]
  for (nm in names(exprs))
    df[[nm]] <- eval(exprs[[nm]], df, parent.frame())
  df
}

head(my_select(mtcars, mpg, cyl, wt), 3)

```

```

#>           mpg cyl  wt
#> Mazda RX4   21.0   6 2.620
#> ... [output truncated]

```

```

head(my_mutate(mtcars, kpl = mpg * 0.425), 3)[,"kpl"]

```

```

#> [1] 8.925 8.925 9.690

```

Write `permutations(x)` returning all permutations of vector `x` as a list.

- Test: `permutations(1:3)` $\rightarrow$ 6 permutations
- Use recursion: pick each element as first, permute the rest
- How many permutations of 8 elements? Can your function handle it?

```
permutations <- function(x) {  
  if (length(x) <= 1) return(list(x))  
  result <- list()  
  for (i in seq_along(x)) {  
    rest <- permutations(x[-i])  
    result <- c(result, lapply(rest, \(p) c(x[i], p)))  
  }  
  result  
}  
perms <- permutations(1:3)  
length(perms)                                # 6
```

```
#> [1] 6
```

```
perms[[1]]; perms[[6]]
```

```
#> [1] 1 2 3
```

```
#> [1] 3 2 1
```

```
factorial(8)                                # 40320 permutations
```

```
#> [1] 40320
```


Build an event system using closures:

- `emitter$on(event, callback)` — register a listener
- `emitter$emit(event, ...)` — fire all listeners for that event
- `emitter$off(event, callback)` — remove a listener
- Test with “click” and “hover” events

```
make_emitter <- function() {  
  listeners <- list()  
  list(  
    on = function(event, cb) {  
      listeners[[event]] <- c(listeners[[event]], list(cb)) },  
    emit = function(event, ...) {  
      for (cb in listeners[[event]]) cb(...) },  
    off = function(event) {  
      listeners[[event]] <- NULL }  
  )  
}
```

```
em <- make_emitter()
em$on("click", \(x) cat("Click at", x, "\n"))
em$on("click", \(x) cat("Log:", x, "\n"))
em$emit("click", 42); em$off("click")
```

```
#> Click at 42
```

```
#> Log: 42
```

Write `topo_sort(edges)` for a DAG given as list of `c(from, to)` pairs.

- Test: `list(c("A","B"), c("A","C"), c("B","D"), c("C","D"))` → A, B or C, then D
- Detect cycles: `list(c("A","B"), c("B","A"))` → error
- Use Kahn's algorithm (in-degree based)

```
topo_sort <- function(edges) {  
  nodes <- unique(unlist(edges))  
  indeg <- setNames(rep(0L, length(nodes)), nodes)  
  adj <- setNames(vector("list", length(nodes)), nodes)  
  for (e in edges) {  
    adj[[e[1]]] <- c(adj[[e[1]]], e[2])  
    indeg[e[2]] <- indeg[e[2]] + 1L }  
  queue <- names(indeg[indeg == 0]); result <- character()  
  while (length(queue) > 0) { n <- queue[1]; queue <- queue[-1]  
    result <- c(result, n)  
    for (m in adj[[n]]) { indeg[m] <- indeg[m] - 1L  
      if (indeg[m] == 0) queue <- c(queue, m) } }  
  if (length(result) != length(nodes)) stop("cycle!")  
  result }  
topo_sort(list(c("A","B"),c("A","C"),c("B","D"),c("C","D")))
```